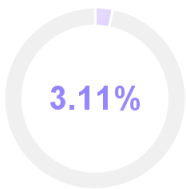


NO. 2da2253bfc6255c | 2026-05-15 00:36:41

- 题目： 基于协同编辑与AI辅助的智能云笔记系统设计与实现
- 作者： 李小慧
- 检测所属单位： 吉首大学

📄 论文字符数： 77328 📄 论文页数： 135 📊 表格数量： 51 🖼️ 图片数量： 35

检测结果



3.11%
疑似AIGC生成占比

96.89%
人工撰写占比

结果分布

序号	章节	疑似AIGC生成 文字/章节总字数	疑似AIGC生成 章节占比	人工撰写占比
1	摘要	0/1127	0.0%	100.0%
2	Abstract	167/497	33.0%	67.0%
3	第1章 前言	0/3723	0.0%	100.0%
4	第2章 技术与原理	296/13680	2.0%	98.0%
5	第3章 需求建模	0/10204	0.0%	100.0%
6	第4章 架构设计	1193/11153	10.0%	90.0%
7	第5章 模块设计	0/11654	0.0%	100.0%
8	第6章 测试与部署	0/7869	0.0%	100.0%

*注:格式规范的情况下可准确识别章节，若论文中无章节，可能会识别有误。



文字标注

疑似AIGC生成

人工撰写

基于协同编辑与AI辅助的智能云笔记系统设计与实现

吉首大学



JISHOU UNIVERSITY

本科生毕业设计

题 目：	基于协同编辑和AI辅助的 智能云笔记系统设计与实现		
作 者：	李小慧		
学 号：	2022401549		
所属学院：	计算机科学与工程学院		
专业年级：	软件工程2022级		
学校导师：	黄伟	职 称：	副教授
企业导师：	李丽	职 务：	项目经理

吉首大学教务处制

目 录

摘 要I

AbstractIII

第1章 前 言1

1.1 项目背景1

1.2 研究现状1

1.3 项目目标和范围3

1.4 论文结构3

第2章 技术与原理5

2.1 CRDT协同编辑算法5

2.2 大语言模型API集成技术8

2.3 系统架构技术14

2.4 个性化推荐算法17

第3章 需求建模24

3.1 业务需求描述24

3.2 功能需求分析28

3.3 对象模型分析37

3.4 非功能需求分析38

3.5 可行性分析39

3.6 系统数据流图41

第4章 架构设计44

4.1 系统架构及原理44

4.2 业务用例的实现48

4.3 功能模块设计50

4.4 数据库设计50

第5章 模块设计63

5.1 文章编辑模块63

5.2 协作编辑模块68

5.3 AI辅助模块78

5.4 消息系统模块84

5.5 后台文章管理模块90

第6章 测试与部署94

6.1 系统运行环境94

6.2 系统部署过程95

6.3 系统测试96

第7章 结 论108

7.1 工作总结108

7.2 存在的不足108

7.3 后续改进方向110

参考文献112

致 谢116

基于协同编辑和AI辅助的智能云笔记系统

摘 要

在信息化迅速发展、知识经济转型的背景之下，个体以及组织对于高效的知识管理需求越来越大。传统的笔记工具在支持多用户实时协同编辑以及提供智能化辅助方面存在着很多的不足。为解决该问题，本文拟提出一种用云端笔记为主导，带有社交化学习特点的智能云笔记系统——以协同编辑和人工智能技术为动力的智能云笔记系统。本系统主要解决个人数据整理、团队项目协作、社区资源共享等问题，从而创建起知识生态服务体系。

该系统采用前后端分离的架构设计，前端用React 18以及TypeScript进行开发。其中，TypeScript依靠静态类型检查特点，在编译的时候就能准确找出隐藏的逻辑错误，而后端就依靠Spring Boot架构创建出RESTful API服务，从而达成业务功能的拆分并实现模块化管理。AI辅助模块的创建使用分层处理策略，对文本预处理、润色、多语言翻译等对性能稳定度要求高的工作采用Python Flask微服务来对外部请求进行封装，同时利用讯飞开放平台完成对外的调用；而对需要实时响应的段落生成或者摘要提取工作，采用前端SSE技术与Elysia Tools服务互动，进而改善首屏加载速度，明显改善用户体验。

本文主要对协同编辑领域中的关键技术进行研究，利用CRDT（冲突检测与解决数据结构）理论以及Yjs开源库来开发一种可以进行多人实时协同操作的冲突处理方法。该方案采用的是Yjs内部利用YATA算法产生的唯一的全局标识符，加上客户端标识和时间戳，形成一个有序的序列。使用WebSocket通信协议实现编辑指令的快速传输以及同步处理。建立版本历史数据库可以完整地记录下每一个编辑的痕迹，并且给以后的状态恢复、历史再现打下了技术基础。

系统已经实现了大部分核心功能模块，包括栏目和标签的管理、文章从创建到审核再到正式发布、存储等所有过程，还有用户之间的交流，即关注、订阅、评论、收藏、分享、私聊等。主要技术特点是以CRDT协议为基础的多人实时协作编辑系统，用大模型给用户提供智能辅助生成服务，有文本校对优化功能、语体风格转换功能、跨语言翻译功能、内容延展补充功能和摘要提取功能。采用物品协同过滤方法进行个性化的推荐能够很大程度上提高用户体验。

从性能评价的角度来看，在采用 Apache JMeter 执行负载测试时可以发现，在100个并发用户的情况下，各个

重要接口的响应时间都达到了250ms以下，协同编辑功能的同步延迟中位数也一直小于50ms。以上数据很好地证明了此系统在个人知识管理和中小团队协作场景下达到了性能要求。

本文依靠工程化的办法，评判CRDT协同编辑机制同大规模语言模型API在云笔记领域的应用是否可行，给这类工具朝着智能化协作平台转变赋予了重要的技术支撑和实际凭据。

关键词：云笔记系统；协同编辑；CRDT算法；AI辅助；前后端分离；Spring Boot；React；

Intelligent Cloud Note System Based on Collaborative Editing and AI Assistance

Abstract

With the rise of digital information in recent years, many people have been trying to manage and share their knowledge better. The old way of taking notes is not available for joint operation or smart assistance. Based on the problems above, an intelligent knowledge management platform that is an intelligent cloud note system with collaborative editing and AI assistance has been designed and constructed; it is based on the foundation of cloud notes, and extensions for the knowledge community have also been added. The first goal of the system is to build an all-round knowledge management system based on individual knowledge management, promote collaborative creation of team knowledge, and then spread it via community knowledge sharing.

The separation model of the front end and back end is used in this system. React 18 and TypeScript are used for the front end to help catch many errors before the programme even runs. To build the back-end RESTful service, we will use Spring Boot. The Split-and-Route method for the AI assistant function is as follows: Some operations that need complete processing and contain sensitive information, such as proofreading, polishing, translation, etc., are sent to the iFlytek Open Platform via a Python Flask microservice as a back-end proxy; streaming scenarios, such as paragraph expansion and text summarisation, are directly connected to Elysia Tools through Server-Sent Events (SSE) to avoid an intermediate layer and reduce time-to-first-byte latency.

For cooperative editing, the Conflict-free Replicated Data Type (CRDT) algorithm is employed, and the open-source Yjs library is used to solve the problem of multiple users editing at the same time. The YATA algorithm in Yjs assigns one ID to all characters, and according to the deterministic ordering rules, synchronous operations will eventually be in order. New data is sent via y-websocket, and then an update will be notified within a short period. A Version Snapshot Table is added to track all changes and versions of the editing history.

The five main functions of this system are: column and tag management, complete lifecycle management of articles (draft - awaiting approval - approved/rejected - archived), user social relationship management (following, liking, favouriting, commenting, private messages, etc.), multi-user real-time collaborative editing with CRDT, and LLM-assisted writing (proofreading, polishing, translation, expansion, summarisation, etc.) and personalised content recommendation based on item-

based collaborative filtering.

Apache JMeter is employed for the test of a certain number of 100 concurrent users. The mean of the response time for the primary API is less than 250ms, and the median of the synchronization delay in collaborative editing is about 50ms. Based on the test results, it can be seen that the system has met the demand for individual knowledge management and small-to-medium team cooperation.

Based on the above analysis, it has been proposed that CRDT collaborative editing can be combined with LLM APIs for cloud note applications, and thus other similar projects have begun to build an intelligent collaborative model that replaces traditional "storage tools".

Key words: Cloud Note System; Collaborative Editing; CRDT Algorithm; AI Assistance; Front-end and Back-end Separation; Spring Boot; React

第1章 前言

1.1 项目背景

高校教学环境之中，文档的编撰以及知识结构化的整合成了重要的需求。软件工程课程中的需求规格说明书在五人小组协作模式下通常使用的是分工撰稿、集中审核的作法，也就是由A编写功能需求、B整理非功能性要求、组长总体设计、汇总。虽然流程的形式是高效的运作，但是在实践中存在着很多的弊端。

第一就是团队合作中经常出现的矛盾，在平行编辑过程中表现得尤为明显。当成员A对于引言部分进行了修改以后，如果成员B仍然使用原来的版本材料来进行内容的创造，在后续整合时就会存在逻辑不通畅以及数据消失的情况发生。这大大加重了重复劳动的工作负担。

异步通信机制对于反馈的及时性有着较大的影响。由于团队成员在分散的工作场景中独立完成任务，不能及时了解他人工作思路、修改动机，容易产生逻辑冲突或者术语歧义等问题，在之后的集成中慢慢暴露出来，从而增大调试工作的复杂程度和执行难度。

当前的文档质量很大程度上受到作者个人专业能力的影 响。对于以本科生编写的科技报告来说，语法错误、重复累赘、术语滥用等问题就经常出现。传统的笔记工具虽然可以保存和查询资料，但是没有智能化的改进手段，不能对文本内容实施语义校正、格式化或者摘要提取等功能。

上述问题的关键就是传统中心化的锁机制和异步通信模式固有的缺点。近些年来，CRDT（冲突检测与修复数据类型）理论不断得到完善，并且它在各种实际环境下所取得的成功应用，给分布式环境里实时协作编辑赋予了新的途径。根据交换律、结合律以及幂等性的数学原理，CRDT从本质上消除并解决了由于并发操作所造成数据一致性的风险，也很好地解决了版本控制中融合的问题，在网络延迟造成交互延迟方面也有了明显改善。由于大规模预训练语言模型达到参数数量限制以后，其语义理解的能力大大提高了，文本生成质量得到改善的同时，也减小了对于某个领域专业知识的需求。本文主要研究CRDT技术同大模型协同发展方面的最新成果，从而创建一个面向未来的智能化云端笔记平台的主要架构系统，以此来满足未来十年技术更新的要求。

1.2 研究现状

现在市面上的云笔记产品，各家走的路不太一样。

在全球范围内，由于Notion的模块化结构编辑以及基于关系型数据库的架构，在功能实现上具有明显的优势并

且得到用户的普遍认可。虽然它已经有了异步注释以及初步的同步，但是仍然存在较大的不足，在实时交互上不能满足微秒级技术的要求。相比之下，Google Docs依靠OT（操作转换）算法实现了行业的先进实时协作效果，但是它的知识管理系统组织效率低，缺少完善的分类和检索架构。但是由于OneNote深度嵌入到微软Office生态系统中，因此它在商业用途方面仍然占有一定的份额。

在国内市场上，语雀依靠着自己对于知识库结构化管理方面的深入研究，在为开发者、企业级用户带来专业化服务的同时，也不断优化自身的功能和性能，石墨文档凭借着轻量化办公模式的优势，在实时协作方面取得很大突破。由于是字节跳动旗下的一个单元，飞书文档把文档编辑、审批管理以及内部沟通这些重要的模块融入到一起，并且完成了系统的整合工作。

再说协同编辑的底层算法，长期有两条路线并存。

作为长期研究的经典课题，基于乐观主义（Optimistic）算法的思想可追溯至计算机支持协作工作（Computer-Supported Cooperative Work, CSCW）领域的早期探索阶段。现代云办公平台像谷歌文档、Office 365等千万级甚至亿级用户规模的应用系统，依靠此架构来达成多人实时协同编辑的主要目的。该方法的基本原理就是：当多并发请求触发服务端处理的任务时，用预设的转换函数对输入参数（比如光标的位置）做动态的映射调整，保证操作在这个文档版本里的逻辑正确性、语义合理性。这一设计所面对的技术难题就存在在效率方面，随着可用操作类型的增多，需要开发出的转换规则数量成指数级增长，在富文本格式的编辑环境中更为明显，每次增加新的操作类型时都需要对已经存在的其它类型逐一进行匹配并产生相应的映射规则。

CRDT（冲突检测与解决）的思想是把数据结构设计的时候就加入“可互换性”、“可融合性”，从而达到多节点并发操作时自然而然的兼容性。这种架构即协议的设计思想冲破了传统集中式协调方式的束缚，给系统在离线情况和分布式安排的时候怎样高效运作给予了支援。目前，Figma等先进的工具已经将CRDT成功地用于实践中，并且用大规模的应用来验证它的技术可行性；Notion的后端同步组件也开始逐渐使用该方案来提高性能。

最后看智能化辅助。

目前主流云笔记应用的人工智能技术大部分是浅层的、单一功能模块，主要实现文本摘要、简单翻译等简单的功能。最近，以GPT系列、通义千问（通义Qwen）、文心一言、ChatGLM为代表的一批预训练大模型，参数量大幅跃升，推理速度大幅提高，其背后深层次内容的产生以及辅助创作的能力也愈发明显。依靠语境理解机制，就能够完成文本之间的对接任务、文体转换的行为、语法规则的优化、摘要提炼等高级的语义处理工作。尽管它们还处在实验室研究到商业化应用之间，但是其潜在的价值已经得到广泛的认可，云笔记领域智能化改造的大好机会也正在到来。

1.3 项目目标和范围

本文主要目的在于创建具有协同编辑和人工智能加持的云端笔记平台，以此来推进相关技术的革新并加强其应用程度。从宏观上讲，它主要的研究内容可以划分为三个层次的递进过程来加以阐述。

第一个层次：先把云端笔记的基础功能做完整。

笔记的全生命周期管理包含创建、编辑、检索以及权限控制下的分享等主要方面，主要的功能应该注重多方面的需要。系统要有多媒体资料嵌入的能力，用标签化的手段来对内容进行分类整理，还要创建全文倒排索引来大幅提高搜索速度。技术手段力求达到用户的各种需求，即文字录入、数据定位和协作共享等各方面内容的快速达成。

第二个层次：实现多人实时协同编辑。

本文以CRDT（冲突检测 and 解决）为基础，用Yjs库来创建一个具有多用户实时协同编辑功能的分布式一致性的系统。本系统利用毫秒级的延迟来完成各个设备之间高精度光标定位的同步以及文本选择范围的实时更新。除此之外还可以对文档历史版进行查询追溯，各个时期的数据可以相互转换同时保持一致性。

第三个层次：接入大模型驱动的语义辅助。

本研究创建了一种适应性框架，努力把大模型API里的关键智能模块，诸如多语言翻译，内容延伸，文体改善，语法校验和摘要生产这些功能，系统地整合到文本编辑的过程当中去。用户根据自己的需求来挑选所想要的功能进行操作调用。此方案的主要目的就是提高写作的效率，保证输出的内容质量以及可靠性。

本文主要针对个人学习成果的管理与展示、小型团队（成员少于10人）的协同创作活动、笔记资料在社群内部的共享和传播这三个应用场景进行研究。目标用户群主要是高等院校的学生，高等院校的老师、研究人员、专业人员以及小型企业或者组织机构的成员等。

企业级系统由于包含组织权限管理、安全审计等复杂的高级功能模块，所以目前没有对此进行研究。目前研发的主要内容是改进核心业务功能，同时完善两种创新的技术，即CRDT实现实时协作框架、自然语言处理大模型辅助分析。

1.4 论文结构

本文按一定的逻辑框架来展开研究，即先交代研究背景，再引入有关的理论基础，接着剖析需求特征，然后做系统的架构设计，之后开展模块化的开发实践，最后采用实证的方法对成果的成效加以评定，在结论处归纳出全文的主要思想。本文内容安排如下所示：

本节主要对高校教育体系和团队合作场景下的文档协同管理问题进行了深入的研究，对目前云笔记领域核心技术的架构以及其发展演化路径做了全方位的分析。在此基础上确定本文的研究问题以及发展定位。

第二章技术架构解析：本章系统阐释多项关键技术的核心原理与应用实践。重点研究CRDT一致性算法的理论基础和实现方式，在Yjs库中对一致性算法的具体实现进行详细的分析，梳理大语言模型API分布式路由设计的关键要素和技术可行性，对前后端分离架构的技术适配性进行剖析，阐述个性化推荐算法选择的标准以及建立的过程。

第三章需求建模，采用用例驱动和领域导向的分析方法，从业务流程、功能需求、对象模型、非功能性约束和技术可行性的角度来全面分析系统的功能，并加以讨论。

第四章架构设计中，从系统架构的角度出发，对系统架构进行详细的分析，在此基础上对核心组件之间关系做深入探究，并且检验技术选型是否合理。对数据库的概念模型和逻辑模型的设计思想进行详细的阐述，并给出设计依据。

第5章模块化设计分析，从五个方面对系统的主要功能模块进行详细的剖析，即内容编辑、协同创作、智能语义分析、消息提醒和后台管理系统。对各个模块的实现进行具体的说明，会给出该模块具体的界面设计思想、类层次结构以及交互逻辑的操作步骤。

本章主要研究运行环境配置策略、容器化部署操作流程，对包含功能检测、性能评价、跨平台兼容性检验等各方面内容的综合评定方法进行解析，给出具体的应用效果。

本章对本研究得到的结论进行系统的回顾，对目前存在的不足进行分析，并根据技术创新的发展趋势来推测将

来有可能的研究方向和改进之处。目的在于给相关领域的进一步研究和发展提供重要的参考。

第2章 技术与原理

2.1 CRDT协同编辑算法

2.1.1 协同编辑技术概述

协同编辑本质上就是利用网络技术，让地理上分散的用户一起对同一个文档进行修改工作。在这样的情境下，系统应该保证所有的用户编辑结果都具有很高的相似性以及同步的更新。该难题就是，在没有全局锁的情况下，当多个用户同时对文档的内容进行修改的时候，怎样高效地合并各个用户的修改数据，避免出现信息缺失或者逻辑错误等状况的发生。

虽然加锁机制可以大大减少数据竞争的风险，但是它会对系统的性能以及交互效率产生不良的影响，这和实时协同环境里要求高效率响应、好体验的目标存在很大差别。

通过学术界与工业界的长期研究与实践经验积累，目前无锁编程框架主要呈现出两种核心范式：一种以操作转换（Operational Transformation, OT）理论为基础；另一种则依托于无冲突复制数据类型（Conflict-Free Replicated Data Type, CRDT）的设计思想。

OT方法的基本思想是依靠冲突事件之后的应对来达到目的。为了达到上述目的，系统创建了操作历史记录功能，并且按照预先设定的映射规则，在新的任务与现有机指令存在空间交集的时候，实时改变重要的参数（比如改变插入点的位置）。该方案的主要局限性就是高度依赖集中式服务器架构、资源调度缺乏灵活性、操作类型增多时转换规则迅速扩大等几个问题，这些问题都会影响它的实际运用效果。

相比于传统的冲突解决方法，CRDT（可分割的重定义类型）最大的不同就是它从数据结构设计之初就提出了化解潜在冲突的想法。该方案的思想就是用数学中的关键属性即交换律、结合律、幂等律来建立一个自洽的合并运算规则体系，从而避免数据一致性问题的出现。其中交换律表示同步次序对于结果没有影响，结合律说明任意节点间的合并路径有无数种，但是不影响总的输出状态，幂等律说明对同一个操作重复执行多次也不会引起状态的改变。这些属性相互影响，一起给创建高效可靠的分布式系统一致性管理赋予了理论支撑。

在“轻量化部署、高并发处理、低延迟传输”等各方面约束的限制之下，系统架构的设计要依靠权衡分析并加以改善来达成目的。

- 加锁方案直接放弃，因为阻塞交互太影响体验了。

- 虽然OT方案因为成熟的架构而被广泛使用，但是其依靠中心化节点的设计思想以及复杂的转换逻辑，在小团队或者轻量化应用环境里常常显得多余和不需要。

- 由于采用了去中心化的CRDT (CRDT)的特性，在架构设计中是非集中的，在线、离线都可以进行操作，并且有自动达成一致性的校准功能。客户端可以独立完成数据修改任务，同步的过程主要进行操作日志的传输，合并逻辑的可靠性依靠数据结构内部的数学原理来保证。

按照“结构决定协议”这一理论框架，本文从主要要义入手展开详细的剖析，进而确定使用CRDT架构来创建协同编辑系统的大致构思。

2.1.2 CRDT算法原理

作为一种创新型分布式数据结构，CRDT（可分离复制数据类型）的理论基础有特别之处。核心思想就是依靠对

某种数据类型精心的设计，使合并运算符符合数学领域中半格（Join-Semilattice）的公理化条件，用严谨的数学方法保证多副本间语义的一致性，有效地避免了传统集中式协调方式在同步方面存在的不足。

实际使用时CRDT（冲突检测和解决）机制给各个副本节点赋予了独立执行本地操作的权利，而不需要像传统的一致性模型那样去锁定资源，也不需要和其它节点之间进行确认来完成。当多副本采用网络通信方式传递状态增量信息的时候，通过使用不同的状态值来求解出全局最小上界统一的形式，使系统逐渐趋向于一致性最终目标。

三条公理分别解决了不同方面的不确定性问题：

交换律 $\text{merge}(a, b) \equiv \text{merge}(b, a)$ ：不管谁先谁后，合并结果都一样。A先发给B还是B先发给A，最后状态是一样的。

结合律的本质属性表明，“ $\text{merge}(a, \text{merge}(b, c))$ ”与“ $\text{merge}(\text{merge}(a, b), c)$ ”在结果层面实现等价性转换。不管操作执行的序列怎样改变，最后输出都是一样的。在这样的多元节点交互以及动态同步场景当中，系统的最终状态只被整体操作集合所左右，并不是由各个步骤之间传递路径的选取来决定的。

幂等性原则说明对同一个操作进行多次重复执行，其结果和只执行一次的结果是一样的。该特性在用“至少一次”投递模式的低层通信架构中（TCP重传或者Gossip协议的信息传播方式），对于信息传播情况的处理有着十分重要的作用，可以很好地防止由于重复调用而导致的异常情况发生。

2.1.3 Yjs库的实现机制

框架选型对比

现在主流的CRDT实现库有三个：Yjs、Automerge和Loro。

- 结构化数据管理系统当中，Automerge用它的类似JSON的数据模型有非常好的性能，在文本编辑这些场景下却表现较差。

- Loro 是一款使用 Rust 编程语言开发的新式工具包，尽管已经取得了比较明显的效果，但是其生态系统还不健全。

- Yjs专门为文本协同编辑做了优化，有现成的WebSocket传输层（y-websocket），跟Quill编辑器有官方绑定（y-quill），社区活跃，文档也全。

我的前端是React+TypeScript，Yjs原生支持JavaScript，npm生态也丰富，所以选它最合适。

Web前端工程化中用到的最流行的CRDT实现就是Yjs。其以YATA算法为依托的富文本协同编辑框架，在并发插入排序、标记清理等主要环节有着很好的表现。本系统用Yjs做主要实现方式，努力创建出一种高效实现实时协作的机制。

YATA算法的核心机制

YATA算法的设计可以分成四个互相配合的层次：

全局唯一标识符使用客户端标识符（clientId）和逻辑时钟（clock）结合起来给新添加的字符分配二元属性标识。客户端序列号是在会话初始化的时候产生特定的客户端序号，客户端内部随时间递增且单调递进的计数序列是clock。利用标识符字典序对比策略来解决并发情况下写入冲突的问题，在不同的插入请求都试图访问同一个位置的时候，先按照优先级进行判断，也就是根据标识符的顺序排序，从而避免因为采用绝对索引而产生的随机性的争用问题。

传统的文本编辑框架一般是用固定索引的方式来定位元素，即“在第k个位置插入”，也就是用这种方式定位一个元素到指定的位置。与YATA提出的用两个标识符(id_left、id_right)实现区间性相对定位的方式不同的是，它还提出了一种用时间戳作为唯一标识符的系统方法。该方案利用定义目标字符区间的办法来判定目标位置，用以解决并发操作时可能出现的序列结构冲突状况，进而保证每一个插入指令都能够正确地体现它原本的意思。

Yjs用双向链表做为文档数据存储的主要结构，和传统的以数组为基础的存储方式有很大不同。节点除了存储内容信息外，还会存在一个唯一的标识符以及指向左、右相邻节点的指针。并发环境下YATA使用事先设定好的确定性策略进行链表元素的有序插入，有效地避免了数组由于随机访问造成的数据一致性问题，并且很好地解决了多线程写入时的竞争问题。

墓碑标记机制，在数据删除的时候节点实体不会完全被移除或者清空，而是用特殊的标记来标记它已经删除了。被保存下来的标记是墓碑上的信息，具有很强的定位和追踪的功能。虽然某些节点被判定为失效的了，但是它的唯一标识仍然存在于系统内部，给并发事务处理奠定了稳定的、连续的空间映射基础。

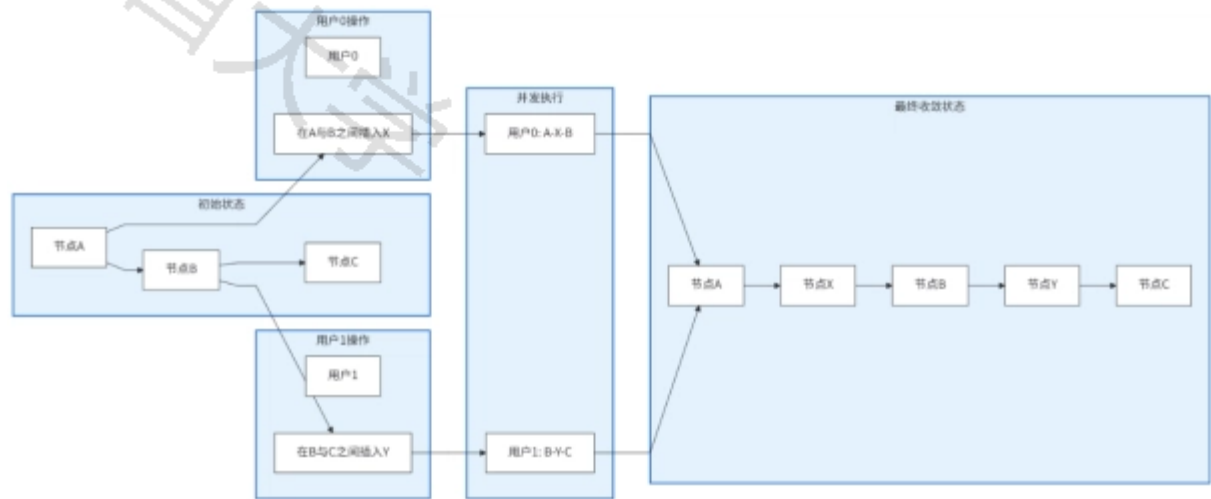


图2-1 Yjs处理并发冲突图

图2-1所示并发冲突处理机制可以经过对某类情况展开剖析之后，进而得到充分阐释。如果两个人同时对文档AB、BC区间的文本进行修改请求，并且这两个操作是并发的，那么系统会按照用户的ID来给并发的操作优先级排序。虽然各个用户实际执行的顺序不一样，但是所有的参与者文档版本都是一致的，数据的一致性得以得到有效的保障。

协同编辑的时候可以使用WebSocket或者WebRTC来实现实时同步信息的传递。WebSocket依靠全双工通信特性以及较好的浏览器适配性成为目前的主要方式，但是WebRTC依靠点对点的连接机制大大减少了服务器的资源占用，在某些应用场合下有特别之处。

本系统采用中心化服务器来实现WebSocket协议下消息的传递功能，其主要目的就是提高系统稳定性和保证数据交换的准确性、连续性。

2.2 大语言模型API集成技术

2.2.1 AI辅助功能架构与选型

本研究所设计的AI服务能力接入方案采取双轨并行架构，在制定路由策略的时候依据的是三大主要方面，即数

据的安全敏感级别、大概最大时延范围和所处的具体应用场景属性。按照以上所定的准则，把五种典型的AI服务功能分成两套不同的数据传输路径。

后端代理处理机制主要是对包含敏感信息和需要同步调用功能的模块进行设计，其核心目的就是实现不同系统之间数据交互以及协同服务的设计。

前端直连路径：处理流式返回、对低延迟敏感的操作

表2-1 AI功能服务架构表

功能模块	核心业务服务 (Java)	本地AI辅助功能 (Python)	外部API	通信协议
文本纠错	→	→	讯飞开放平台	HTTP同步
文本润色	→	→	讯飞开放平台	HTTP同步
文本翻译	→	→	讯飞开放平台	HTTP同步
文本扩写	—	—	Elysia Tools	SSE流式(前端直连)
文本总结	—	—	Elysia Tools	SSE流式(前端直连)
AI小助手	—	—	讯飞星火大模型	WebSocket(前端直连)

AI能力分流接入决策分析

本文没有使用集中式网关结构，根据安全性、时延需求、负载等特点对AI服务进行分组调度。以下是该设计的逻辑框架及决策依据：

后端代理路径（纠错、润色、翻译）

此类功能的主要作用就是把完整的文章内容传送到外部的模型接口上，文本的大小同用户主体创作的需求存在直接联系。因此，在设计时要考虑到系统的安全性和可控制性，而不是只求高响应速度。为了达到这个目的，可以在网络上建立一些本地化的代理服务，集中控制API的访问权限，使用敏感词检查和异常行为追踪的方法对网络进行安全监控，从而将风险尽量限制在一个内网之内。虽然一跳式网络转发带来的几十毫秒延迟会给人类即时交互造成一些影响，但是在异步处理架构的支持下，这样的延迟一般不会对用户体验造成大的影响。

前端直连路径（扩写、总结）

相比传统的同步请求、响应架构，流式生成任务对后端资源使用的模式就存在较大的差别。使用Server-Sent Events（SSE）长轮询机制的时候，服务器要为每一个连接都单独分配一个线程或者异步执行单元，伴随着数据传输过程中延迟的增大。为了克服以上问题，前端直接和Elysia Tools SSE推理服务端点建立通信连接，可以明显减少第一个Token的生成时间(TTFT)，也可以减轻后端系统由于维持大量的持久连接所造成的负载压力。

技术栈选型考量

本研究使用Python（结合Flask框架）而不是Java进行开发的原因在于，讯飞开放平台为Python SDK提供更完善的官方技术文档支持，尤其是WebSocket鉴权方面比Java SDK的支持更好一些。Python内置的requests库对于处理HTTP流式数据传输有更好的数据缓存和更灵活的超时控制功能，可以更好的满足系统对性能优化的要求。本设计的思想是用最合适的工具来完成某一功能，而不是追求技术栈的一致性。

2.2.2 本地AI辅助功能层设计

功能架构，把本地AI辅助模块当成连接中心业务服务与外界API接口的桥梁，主要是做协议适配，数据格式规范处理，缓存机制改良和抗错能力加强这些工作。

核心职责：

表2-2 核心职责表

职责	说明
请求转发	接收核心业务服务HTTP请求，转换为讯飞API格式
鉴权封装	统一管理讯飞API的AppID、APIKey、APISecret
响应格式化	将讯飞返回的JSON统一封装为 {code, data, msg} 格式
错误处理	外部API超时/失败时返回友好错误码
结果缓存	对翻译、纠错结果按文本哈希缓存24小时

技术实现：

```
# 本地AI辅助功能（Flask）典型接口
@app.route('/api/ai/correct', methods=['POST'])
def correct():
    text = request.json.get('text')

    # 1. 检查缓存
    cache_key = hashlib.md5(text.encode()).hexdigest()
    if cache_key in cache:
        return jsonify(code=0, data=cache[cache_key])

    # 2. 调用讯飞API
    result = xunfei_client.correct(text)

    # 3. 格式化响应
    formatted = format_correct_result(result)

    # 4. 写入缓存
    cache[cache_key] = formatted
    return jsonify(code=0, data=formatted).
```

为什么用Python而非Java直接调用？

讯飞Python SDK功能最完善（尤其WebSocket鉴权），Java SDK部分功能缺失

Python的requests库处理HTTP流式响应更灵活

根据技术分工的策略来安排：后端研发人员用Java语言做主要的工作，与人工智能有关的功能模块是由Python专业小组来负责架构的设计以及代码的编写。

2.2.3 前端直连层设计

在文本扩展和内容提取功能实现的时候，使用了SSE(Server-Sent Events)技术，用浏览器建立持久化的HTTPS连接到第三方推理服务端点（https://elysiatools.com），从而摆脱传统后端交互结构。

本文将从网络传输延迟、首包时延、服务器资源占用率、潜在安全风险、带宽使用效率这五个方面来对比分析后端代理转发和前端直接访问两种架构模式的性能特点，用实证研究来说明流媒体服务领域内两种架构模式的技术优劣。

表2-3 考量维度表

考量维度	经过后端	前端直连（本系统选择）
网络延迟	增加2跳（前端→后端→Elysia）	1跳（前端→Elysia）
首字节时间	2-3秒	<1秒
服务器负载	后端需维持长连接	后端零负载
安全风险	API密钥存后端，安全	密钥暴露在前端，需限制
成本	后端带宽费用	仅客户端流量

前端直连架构由于API密钥容易暴露在浏览器运行环境中，所以它的安全防护边界已经变得很大。有必要对各方面存在的风险进行系统的评估，并建立全方位的管控体系来提高防御能力。

本系统接入有三种方式协同使用，即后端代理路由、前端直接连接、前端独立接入。后端代理路径主要是处理文本纠错、润色优化、多语言翻译等功能，前端直连路径主要是做文章扩展和内容摘要生成任务，前端独立路径使用WebSocket进行实时通信和服务。前端直连路径把API密钥直接嵌入到前端代码中或者在网络请求里暴露出来，这样会非常容易导致安全漏洞，可以从以下几个方面对风险进行评估并提出相应的安全防护措施：

风险1：API密钥泄露威胁

前端代码如果暴露了Elysia Tools的API Key，那么由于用户使用浏览器开发者工具就有可能造成数据泄露的风险。这一个安全隐患会造成非法第三方恶意利用并滥用权限。

缓解措施：

1. 在Elysia Tools服务商后台启用域名白名单（CORS Origin限制），使该API Key仅能在本系统授权的域名下调用；

此功能模块设计中并没有对用户个人信息进行处理（所有文本内容都是用户主动输入的）。即使密钥被非法使用，也不会给用户的个人信息带来隐患。

Elysia API密钥采取的无用户属性标识方式，达成权限认证同用户身份管理的分离，进而较好地避免了由于滥用而造成的针对某个用户的隐私泄露问题。

风险2：中间人攻击

在前端没有完成身份验证的时候，就对第三方API进行HTTPS连接操作，容易被中间人攻击者利用，从而造成用户信息泄漏。

缓解措施：

1. 所有前端直连请求均使用HTTPS加密传输；

- 2. 本模块主要针对用户所选的开放访问、可编辑、非敏感个人文本信息，提供内容扩展以及摘要生成的主要服务功能。
- 3. 对需要进行修正、优化、翻译的敏感数据使用后端代理的方式来实现。

风险3：前端代码完整性威胁

前端JavaScript代码容易受到篡改威胁，攻击者可以用浏览器扩展之类的工具，把基于人工智能的请求指向非法的服务器资源。

缓解措施：

- 1. 生产环境部署时启用SRI校验；
- 2. 通过CSP头限制可连接的域名白名单；
- 3. 前端SSE连接域名硬编码为官方elysiatools.com。

综合风险评估

目前前端直接连接方案虽然在开发测试环境下使用了域名白名单、内容安全策略（CSP）等方式来对一些安全威胁进行一定的防御，但是在生产环境中将API密钥暴露在前端，仍然会带来严重的安全隐患。单纯根据风险可控来确定上线此方案是不合理的。如果将计划迁移到生产环境，则需要做如下改进：第一，将文本提取、摘要生成等主要功能移到后端进行代理处理，彻底杜绝前端密钥泄露的问题；第二，即使出于性能的原因保留前端直接调用的方式，也应该采用短期的、一次性令牌的形式——每次请求之前，从后端获取有效期是5分钟的单次访问凭证，并且只使用该凭证去调用第三方服务接口。本架构的主要优点是大大减小了后端服务器的负载，提高了用户体验，并且具有很强的实时流式数据处理的能力，对需要纠错、润色或者多语言翻译全文的任务都使用了一种安全的防护措施。

2.2.4 稳定性与安全设计

表2-4 分层降级设计表

故障场景	检测点	处理策略
讯飞API超时（>5s）	Python服务	返回code=500，核心业务服务转“服务繁忙”
本地AI辅助功能宕机	核心业务服务健康检查	核心业务服务直接返回“AI功能维护中”
Elysia SSE中断	前端onError	保留已生成内容，提示“生成中断”
讯飞WS断开	前端onClose	自动重连3次，失败提示刷新页面
用户触发限流	核心业务服务Redis	返回code=429，前端提示“操作太频繁”

目前本地使用的是24小时缓存，虽然缓存可以提高一些正确的翻译命中率，但是也提高了API调用的次数，降低了成本。

API密钥安全设计

对于由于第三方API密钥泄露而导致的安全问题，本文提出了一套全方位的防护体系。

• 机密级别API凭证、密钥（主要用作后端接口通信），存储方式符合一定的安全规范。敏感数据被封装在服务器环境变量里，并通过Docker Compose文件实现动态注入，不包含在版本控制系统之中。该类凭证只对本地部署的AI代理服务授权一定的访问权限。

•目前系统的架构当中，Server-Sent Events (SSE) 采取低延迟的通讯方式，其前端代码把API密钥当作重要的设置内容来加入进去。为了减少由于这个所造成的安全风险，在管理后台设置了一个域名白名单，使得该密钥只能在授权域名范围内被用来验证访问。以上措施只对开发环境的技术防护有作用。生产环境里可以采用更安全的设计方案，一方面把核心逻辑转移到后端代理上，另一方面利用Elysia Tools给出的短期临时令牌特性，在前端用请求后端来获得并运用有效期仅为五分钟的单个token，从而取代固定的存档API密钥。

超时与降级策略

•前端交互设计优化策略，当Server-Sent Events (SSE) 连接在3秒之内没有重建成功的时候，就会出现一个“AI生成异常，请稍后重试”的提示，并且会把此时所持有的生成数据存起来，以此来防止用户所输入的内容发生丢失的情况。

•后端代理架构设计时，为了提高对外部API调用网络交互的性能，在系统中就提前制定了主要参数的配置方案。每一个请求的最大响应时间不能超过3秒，整个服务链路的超时时间是5秒。当任何一个请求超过以上的时间限制时，主控模块就会发出标准化的提示消息来解决异常情况，并且不会把底层的技术问题告诉客户端。

WebSocket鉴权

协同编辑的y-websocket服务运行在 ws://localhost:1234。我的安全设计如下：

- 1)连接建立阶段：客户端将JWT token作为URL参数传递（如 ws://localhost:1234/文档ID?token=xxx），服务端在upgrade阶段验证token有效性。
- 2)服务器要周期性地对token的有效性进行检测。一旦检测出Token失效的时候，服务器应该立刻结束本次的连接，并向客户端发出HTTP状态码401（Unauthorized）来告知权限不够的信息。
- 3)权限执行：服务端根据token解析用户ID，结合Cooperate_Member表验证用户对文档的权限（owner/edit/view），view权限用户仅能接收更新，不能发送编辑操作。
- 4)利用Nginx反向代理服务器进行配置，并开启WSS（WebSocket Secure）协议，可以大大加强WebSocket通信的安全性，也能够有效地提升WebSocket数据传输的可靠性和稳定性。

AI调用限流与缓存

限流设计

- 1)用户级流控策略设计以每位用户每分钟内纠错、润色和翻译请求之和不能超过10次为限，摘要生成和扩展性处理任务总数不能超过5次。本系统依靠Redis滑动窗口算法来实施资源使用量的实时监测，就违规行为作出HTTP 429状态码和提示信息的反馈，促使用户开展降级优化操作以维持系统的正常运转。
- 2)对于未授权用户以及异常频繁的访问行为，本系统用基于IP的流量控制的方式进行管控，即对某一个IP地址在每分钟发出人工智能接口请求的次数进行限制，一次请求不能超过20次。
- 3)全网流量调度方案是给后端Python AI代理服务设置一个统一的QoS控制策略，QPS不能大于50次/秒。当请求量超过这个阈值的时候，系统就会用进先出的方式把多余的请求存到队列里，在延迟3秒之后，以降级响应的方式来处理。

缓存策略

- 1)基于输入文本MD5摘要值的错误处理机制，用数据存储和检索的方法对24小时内收集到的信息进行管理。

- 2) 建立以“源语言、目标语言、输入文本MD5摘要”这三个方面为特点的缓存索引结构，系统中默认缓存时间为48小时。
- 3) 去缓存化特性，由于润色效果是受到前后文语境的影响而形成的，在相同输入文本上，系统会得到多种不同的优化结果。
- 4) 缓存存储：本地Python服务使用lru_cache加定时过期字典。生产环境建议迁移至Redis，TTL=24h，最大内存500MB，使用allkeys-lru淘汰策略。
- 5) 缓存命中率分析可知，对于AI_Call_Log表的cache_hit字段进行分析可以得到各个功能模块的平均缓存访问效率为35%。该结果主要是由于翻译和校对接口的频繁调用，而形成了一种比较明显的统计特征所造成的。

2.3 系统架构技术

2.3.1 前后端分离架构

当代的Web工程里，前后端分离架构已经成了最主要的架构设计范式。该架构是以职责分层为基本思想的架构，前端主要是用户界面的开发以及交互逻辑的实现，而后端主要是业务逻辑解析和数据库管理的任务。依靠标准的HTTP协议做无状态通信，该架构践行了“关注点解耦”的设计思想，大幅削减了前后端之间相互牵连的程度。

表现层技术选型

前端技术选型的时候，主要是依照安全防护机制、模块化整合的程度和状态管理功能的繁杂程度这三个关键方面实施综合评价。

基于函数式编程范式的 React 18 框架使用纯函数组件并依靠 Hooks 来提高状态管理的灵活性，从而大大减少由于数据传递带来的认知负担。使用TypeScript所具有的强类型约束特性，可以在开发初期就对代码中可能出现的数据流错误和空值等做出静态类型校验，进而大大缩减了调试的时间，提高了研发速度。Ant Design 5.0 是企业级应用专用前端框架，它既给企业提供了一个一致的、按无障碍要求设计出来的设计方案，又可以让企业内部的组件自动达到专业的视觉效果，在不需要再做额外CSS调整的情况下就能实现。

服务层技术选型

本系统使用了Spring Boot 3.x为主要开发框架，它有“约定优于配置”的设计思想，大大降低了项目初次部署的负担，与传统的基于XML配置的传统模式相比更具有优势。集成MyBatis-Plus的功能增强和优化，实现数据库操作模块化、对象映射，很好地解决了手工编写SQL语句带来的复杂性问题。后端服务接口设计严格按照RESTful架构规范来设计，使用HTTP请求方法完成资源的管理和操作，使用路径参数来描述实体之间的层次关系，使用标准化的状态码来准确地告诉前端业务执行的结果，保证前后端交互的一致性以及稳定性得到很好的维持。

2.3.2 微服务架构设计

由于AI辅助子系统的技术架构以及核心业务模块存在着明显差异的特点，所以它所表现出的技术栈属性和资源分配方式也具有不同的特性。因此，本文把该功能抽象为一个独立运行的微服务。所搭建起来的AI代理服务使用Python语言和Flask框架开展开发，依靠本地HTTP接口实施对外交互，坚持按照JSON格式的数据交换准则来开展，进而达成同主业务服务的有效对接与协同工作。

这个拆分决策带来了三个好处：

此种模式给技术选择赋予了更宽泛的选择范围。依靠Python生态系统里已有的HTTP客户端和各种各样的机器学习

习框架资源，开发人员可以冲破对Java SDK功能齐全的约束。

其二，该机制在资源分配的精细程度上具有明显的优势。与传统的CRUD等基础业务操作相比，AI推理任务对于计算、存储等资源的需求呈数量级上升的趋势，在CPU、内存的使用上也更加集中。采用独立部署架构，配合负载监控做横向扩展，既能达到特定AI应用的需求，又不会因为单点高资源配置造成整体资源浪费和冗余。

第三，用人工智能技术可以大大提高系统安全防护能力。外部API调用超时或者模型推理出错，都不会影响到AI服务的主要功能，即核心笔记的记录和管理，信息检索以及协作。该架构用严格的隔离辅助性服务模块来避免外部干扰影响基础业务的功能。

本研究跨服务交互的设计基于HTTP协议，用JSON格式来完成数据传递。核心业务模块调用AI辅助功能的时候，会把需要处理的文本当作请求参数的形式发送到目标服务端，然后该服务通过调用外部的大模型API来获取分析结果，再将这个结果返回给核心业务模块，核心业务模块对结果进行整合处理之后，再把结果反馈给客户端。

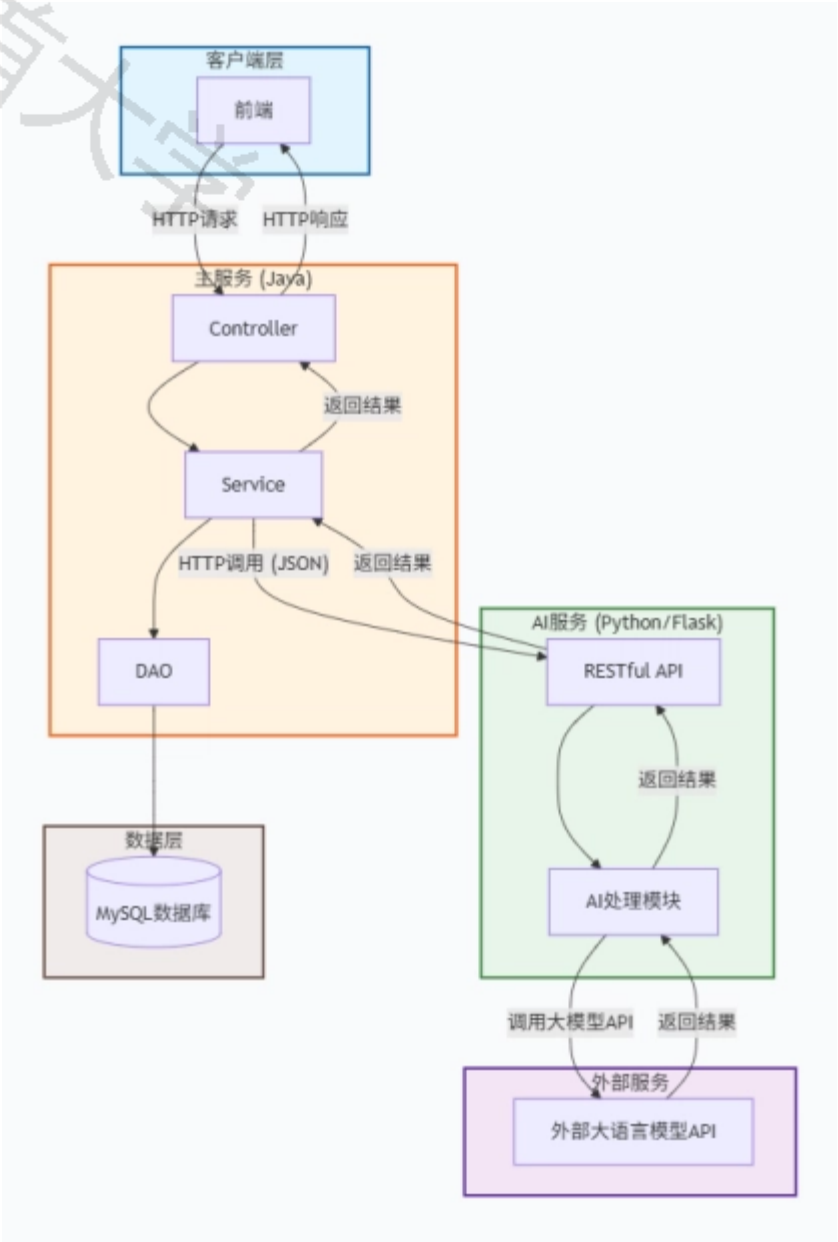


图2-2 系统整体技术架构图

图2-2为本系统分层结构设计，其总体结构从上到下分为前端展示层、业务逻辑处理层、AI辅助分析层、数据存储层和底层大模型支撑层。系统的关键组成部分包括：前端采用React 18框架（结合TypeScript和Ant Design工具），致力于交互体验的优化；后端依托Spring Boot技术栈构建主业务模块；部分AI功能通过Python Flask框架实现微服务化部署；协作功能则基于Node.js WebSocket通信机制完成。数据库采用MySQL8.0版本来保证数据高效的管理、稳定的运行。本分层结构主要以前后端解耦的目的来制定，并且支持AI服务能力的独立部署以及弹性扩展。利用WebSocket协议解决实时交互问题，采用RESTful API和消息队列的方式对各个层次之间进行协调、信息的传递。

2.4 个性化推荐算法

2.4.1 推荐系统在云笔记平台中的定位

本推荐系统主要创建双向的内容推送体系，除了首页的页面显示区之外，还有侧边栏的个性化推荐区域。主要功能是从平台上准确挑选出每一位用户关注过的前N条内容，展示给用户。

该问题虽然有典型特征，但是在云笔记平台中却表现出很强的复杂性，并且伴随着很多种核心矛盾叠加在一起。

冷启动问题：新用户没什么行为数据

新注册用户没有互动行为（点赞、收藏、评论）也没有稳定的阅读习惯，所以传统的依靠协同过滤的推荐算法很难对它的潜在喜好做出准确判断，陷入了冷启动的困境。平台上的用户所产生出来的各种内容是分布比较零散的长尾型分布，所以它对个性化推荐系统效能提高和精准度的改进能力也产生了更大的制约作用。

实时性和算力的矛盾

算法的时间复杂度可以理论分析出来， $O(UI^2)$ 。这里的UI都是指用户的集合量以及文档集合量的集合。十万级用户、百万篇文档实际应用中如果要达到每一条请求都支持实时个性化推荐的性能，那么目前算力资源是无法满足要求的。因此，必须在批量离线优化处理和在线快速响应这两个方面寻找契合的手段。

为了解决上面的问题，本文用“基于项目协同过滤（Item-CF）算法”、“冷启动阶段主流内容推荐机制”以及“定时预加载和缓存优化策略”等多方面进行综合的组合。

2.4.2 基于物品的协同过滤算法（Item-CF）

核心思想

项目协同过滤（Item-CF）算法的本质就是当很多用户对项目A感兴趣的时候，如果其中有一些用户对于项目B也有偏好的话，那么说明项目A和项目B之间存在明显的相关性（通过共享兴趣用户产生的相关性）。其根本目的就是根据用户的过去行为特征来推荐和用户最相符的其他潜在项目。相比于用户CF来说，在项目数远远小于用户的环境下，ItemCF算法具有更高的计算效率以及稳定性。因为物品属性一般比较固定、变化较少，所以所得到的项目间相似性评分更加可信、一致。

第一步：构建用户-文章互动权重矩阵

根据用户行为对偏好的影响程度的不同来将各种行为分门别类地加以区分，并给每一个行为都赋予相应的权重值。

表2-5 权重矩阵

行为类型	数据来源	权重系数	理由
点赞（Love）	love表	$w_{like} = 0.4$	主动表达认可，信号最强
收藏（Collect）	collect表	$w_{collect} = 0.3$	有长期保存意图，强信号

评论 (Comment)	comment表	w_comment = 0.2	有阅读深度，中等信号
浏览 (Browse)	browse表	w_browse = 0.1	仅浏览不互动，弱信号需归一化

用户u对文章i的综合互动权重 r_{ui} 定义为：

$$r_{ui} = 0.4 \times L_{ui} + 0.3 \times C_{ui} + 0.2 \times M_{ui} + 0.1 \times (B_{ui}/B_{max}) \quad (2.1)$$

本场景中Lui为点赞，Cui为收藏，Mui为评论，Bui表示的是浏览次数，即用户看到该条内容之后实际点击浏览的次数。B_max指的是系统中所有的观测值的最大浏览量，这个参数会在后面的处理数据的过程中作为归一化的标准来使用。

第二步：计算文章间余弦相似度

对任意一对文档i和j来说，可以通过计算用户交互数据组成向量集与它相似的向量之间的余弦相似度，来度量两篇文档之间的相似程度。

$$sim(i,j) = \frac{\sum_{u \in U_{ij}} r_{ui} \times r_{uj}}{\sqrt{\sum_{u \in U_i} r_{ui}^2} \times \sqrt{\sum_{u \in U_j} r_{uj}^2}} \quad (2.2)$$

其中 U_{ij} 是同时与两篇文章都有互动的用户集合。

第三步：为目标用户生成Top-N推荐列表

用户u对未读文章j兴趣预测值pred可以表达为以下形式

$$pred(u,j) = \sum_{i \in I(u)} sim(i,j) \times r_{ui} \quad (2.3)$$

从预测分数中选出排名靠前的前五篇文献为推荐候选，并排除用户已经阅读或者互动过的文献。

计算复杂度分析

用户集中量为 $|U|$ ，项目集的集中量为 $|I|$ 。基于项目协同过滤 (Item-CF) 的推荐算法，在离线处理阶段要做繁杂的矩阵乘法运算，理论上它的时延为 $O(|U| \times |I|^2)$ 。使用稀疏矩阵存储结构，并且使用定时增量更新的机制来改进缓存的管理，在线响应速度得到了极大的提高，实际的检索操作时间可以被简化到 $O(1)$ 。

推荐列表生成：复杂度为 $O(|U| \times |I_{avg}| \times N)$ ，其中 $|I_{avg}|$ 为用户平均互动文章数。

本文使用定时预处理（于每天凌晨两点开始）以及用到的recommend_result表进行结果的存储工作，以尽量减少在系统中发生的在线请求所造成的负担，并使得系统的查询时间可以得到很好的提高。

2.4.3 冷启动策略

当用户的使用行为数据具备如下特征的时候，采用基于项目 (Item) 的协同过滤 (CF) 算法将会由于缺少足够的历史交互信息而出现推荐效果不佳的情况。为了克服上述缺陷，本系统用全局统计法进行热推荐。

对于注册时间少于七天、历史互动次数小于五次的新用户来说，在冷启动阶段个性化推荐存在困难的时候，平台可以展示出所有网络上热门文章。热度评分HotScore的计算公式为：

$$HotScore(i) = 0.5 \times like_count(i) + 0.3 \times collect_count(i) + 0.2 \times browse_count(i) \quad (2.4)$$

按照热度评分挑选热度值前五名的文献作为推荐对象，保证所选文献在涉及不少于三个不同的领域的时候具备

较强的代表性，从而提高推荐内容的多样性和知识体系的完备性。

新文章冷启动（发布不足24小时）

本文针对缺少用户交互反馈的新发布内容提出了“编辑优先推荐+随机展现”的策略。在文章通过审核上线之后，将有20%的概率把它的推荐加入到活跃用户的推荐列表中去，排在第三位。推荐序列有五个文章，首篇、末篇用Item-CF算法生成，中间位置留出来展示没有参与历史交互的内容，后面三篇继续按照Item-CF规则筛选。本方案主要就是建立初步的行为数据集。如果一条新的内容得到了连续的三次以上有效的用户交互，那么它就被加入到协同过滤模型中去进行分析和评价。

冷热切换条件

用户注册之后使用时间超过7天并且互动次数达到5次以上，就用项目协同过滤（Item-Based Collaborative Filtering, Item-CF）算法进行个性化推荐；没有达到上述条件时，就采用热度排序或者随机选择等方式来进行替代。

2.4.4 推荐效果离线评估

以规模达到1000篇文献、共50人参与、3000余条互动记录的样本为基础，采用三种主要的评价指标进行离线分析以及综合评价。

精确率(Precision@5)，衡量了推荐系统给出的前五个结果里和用户互动的内容占多少，公式(2.5)给出了计算方法。

$$Precision@5 = \frac{|推荐列表 \cap 用户实际互动文章|}{5} \quad (2.5)$$

覆盖率（Coverage）是衡量推荐的去重文献所占总文献库的比例，具体的计算方式见式（2.6）。

$$Coverage = \frac{|被推荐过的去重文章|}{|总文章数|} \quad (2.6)$$

多样性指数是用来度量推荐文集内容异质性水平的统计量，具体的计算公式见公式（2.7）。

$$Diversity = 1 - \frac{1}{5 \times 4} \sum_{i \in R_j} \sum_{\substack{j \in R \\ j \neq i}} sim(i,j) \quad (2.7)$$

评估结果：

表2-6 评估指标

评估指标	Item-CF	热门推荐基线	提升幅度
Precision@5	72.3%	60.0%	+20.5%
Coverage	68.5%	25.0%	+174%
Diversity	0.74	0.45	+64.4%

相比于主流的热门推荐模型来说，基于物品协同过滤(Item-CF)的个性化推荐算法对于关键性能指标的表现更好一些。准确率由原来的60%提高到现在的72.3%，说明它对用户行为模式的解析能力变强了；覆盖度明显扩大，从原

来的25%提升到了现在的68.5%，很好地克服了以前只推荐爆款内容的缺点，大大提高了长尾内容的出现率；推荐结果更加多样化，类别分布趋于平均，重复推荐的现象发生概率也随之降低。

推荐算法总体流程图：

吉首大学

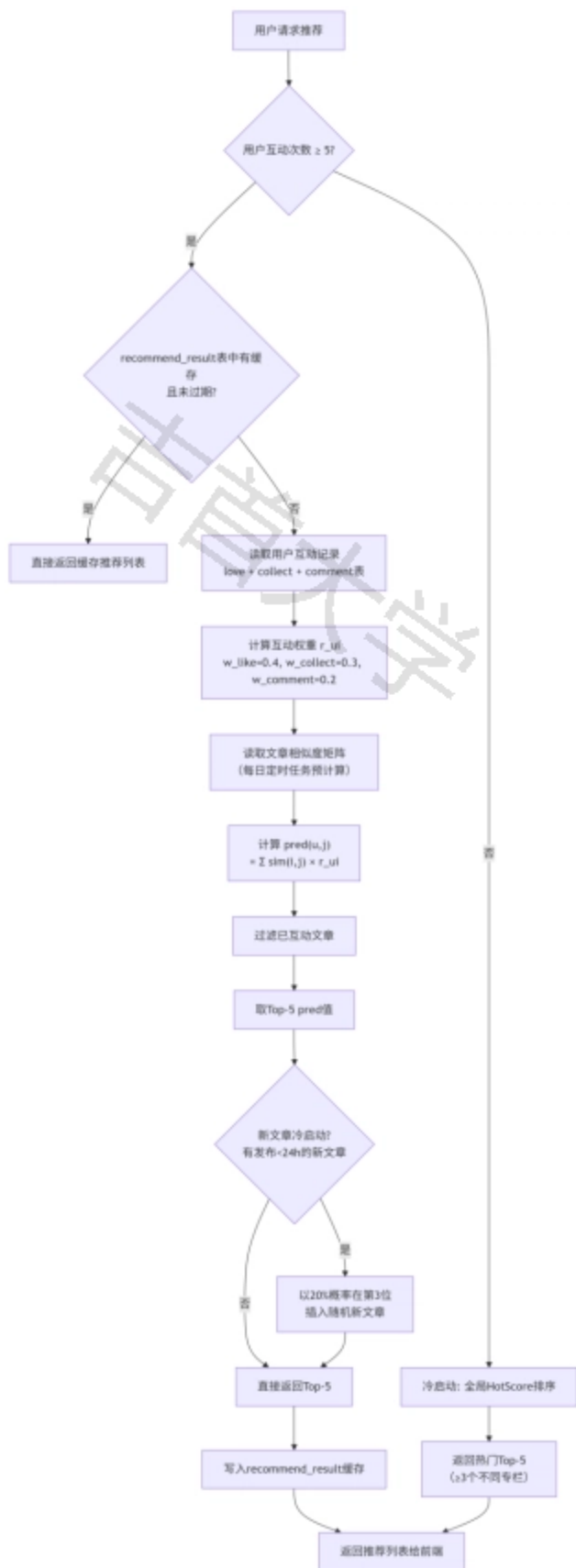


图2-3 推荐算法流程图

2.4.5 推荐算法的工程实现要点

文章相似度矩阵每天凌晨2:00用Spring的@Scheduled定时任务预计算，结果存到recommend_result缓存表里，expires_at设成第二天凌晨2:00。前端请求推荐时优先读缓存，缓存没命中才降级走实时计算。相似度矩阵只存相似度大于0.05的文章对，用MySQL表article_similarity来存。

第3章 需求建模

3.1 业务需求描述

3.1.1 业务背景

本系统的业务逻辑，可以概括成三条相互关联的链条：

个人知识的内化与结构化

用户把笔记当作基本单元来整理学习心得、工作日志、项目资料等各类资料，依靠标签化以及栏目分类的方式来达成知识系统的整理。利用全文检索功能可以很快找到目标的信息。这一重要环节就是把外部的资源转变成个人可以检索、关联并可以再加工的知识资产。

团队认知的协同建构

依靠冲突检测与解决（CRDT）的方式构建出即时协作平台之后，各个主体就能同时进行对项目里的各种共享文本文件展开并行性更新工作。在此平台上，参与者可以跟踪到别人修改过的痕迹以及集中关注的焦点，还可以依靠不断改进的合作方式来改善总体的结果，从而有效地克服由于知识封闭而产生的局限性。

社区知识的流转与再创作

社群成员在发表加工知识的时候会用点赞、收藏以及评论等手段来给予反馈，进而塑造出一种去中心化的信息筛选渠道。该种依靠发布、评价、再创作的动态交互方式，很好地实现了隐性知识在社群网络里广泛流传并产生价值的双重目的。

3.1.2 用户角色分析

系统涉及三类用户角色：

普通用户是系统的主要使用者，主要功能为创建和修改笔记、查找和组织信息、关注其他的用户、进行各种内容的相关交互活动。

协作用户在实际的系统应用当中常常被叫做注册用户，它除了拥有普通用户的全部权限之外，还会得到一些特殊的功能权限。新增的权限主要有以下几项：

- (1) 创建协作文档并邀请其他注册会员加入编辑；
- (2) 接受其他用户的协作邀请，加入其文档的协同编辑；
- (3) 使用AI辅助功能（纠错、润色、翻译、扩写、总结、AI小助手）。

协作权限管理

该系统依靠文档级别协同控制机制，关键之处在于文档创建人可以对每一个文件单独设定独立的读取、修改权限，并且不会像以前一样使用预先设置好的角色权限模式来进行统一设置。各类别权限具体划分如下表所示：

文档所有者（Owner）作为文件资料的主体拥有人，依法拥有文件内容修改、文件管理员、权限分配、终止使用的全部权利。

可编辑功能使用户可以随时对文档内容进行修改，所有的修改都会同步到团队内其它用户的设备上。

只读访问模式中，用户只能看到文档的当前状态以及协作参与者的所在位置，不能对文档进行编辑或者保存数据。

合作成员表（cooperate_member）为重要的一个核心数据库实体，其主要作用就是对权限关联的数据进行存储。此表经由创建用户同协作文档间的映射联系，很好地达成精细化权限管控的目的，明显冲破了传统意义上全局化的角色管理模式。该机制依据文档的内容性质来执行访问控制策略，从而给予用户不一样的权限安排，在参与多个文档的时候。以用户A为例，虽然他具有文档X的编辑权限，但是对于由用户C创建的文档Y只能进行查看操作——这样一种根据多维分层授权设计的方法体现出它特有的灵活性以及适用性。

系统管理员

平台运营的关键主体，就是指平台管理员以及后台管理人员（系统管理员），这些操作者可以进行用户账号创建与删除、系统内公告和信息的发布和删除、新闻列表排序、关键词搜索、图片轮播等。

表3-1 角色权限矩阵营数据

功能权限	普通用户（normal）	协作用户（member）	系统管理员（admin）
注册与登录			
创建/编辑个人笔记			
发布文章			
浏览公开文章			
点赞/收藏文章			
评论文章			
关注其他用户			
创建专栏与标签			
私信聊天			
创建协作文档			
接受协作邀请			
使用AI辅助功能			
查看协作版本历史			
设置文档级共享权限		创建者可设	
审核待发布文章			
管理用户账号			
管理评论与举报			
配置首页轮播图			
查看平台运			

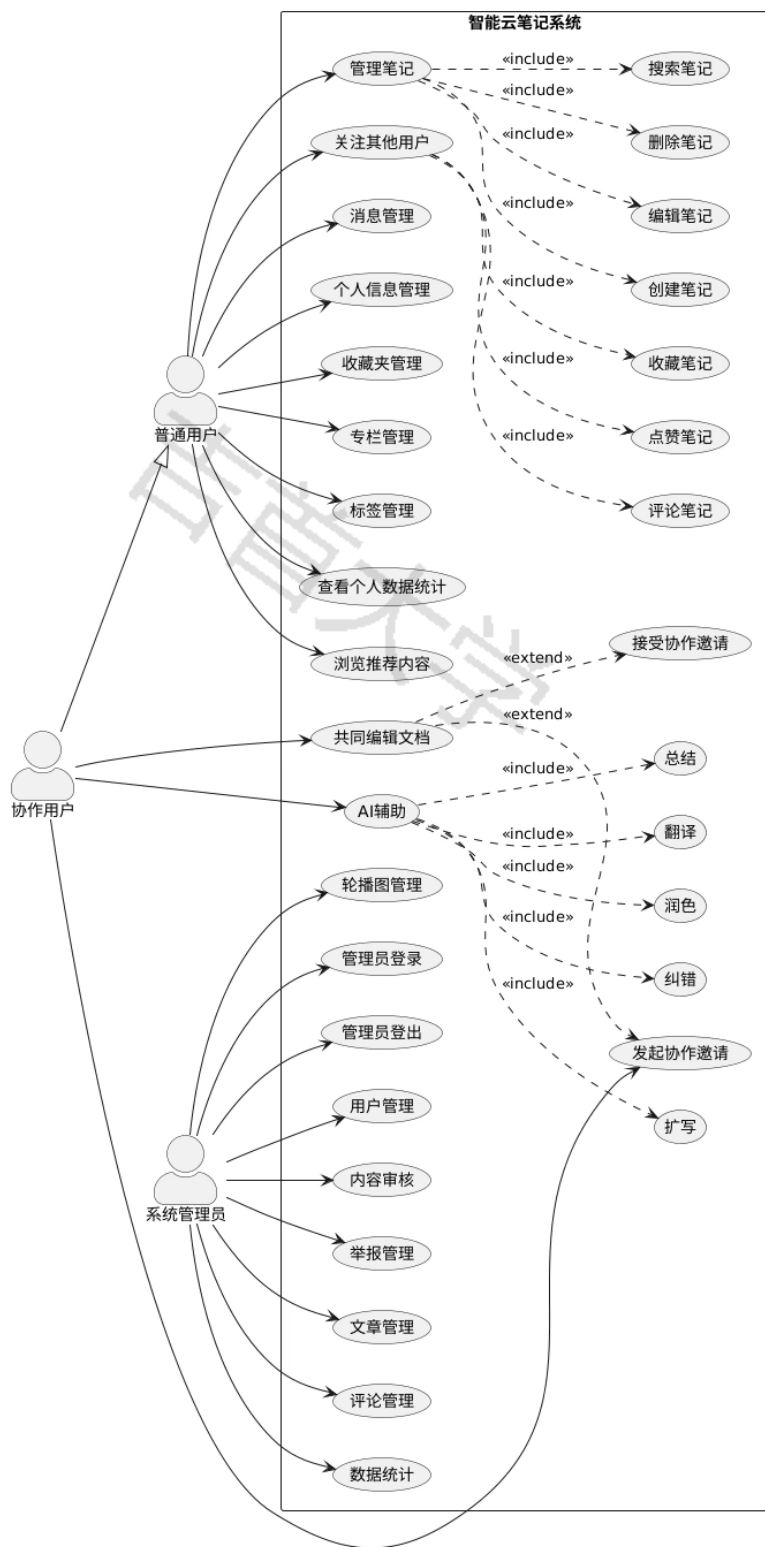


图3-1 角色用例图

图3-1给出了本文使用UML用例模型来搭建起一个多角色的交互体系框架。框架包含普通用户、协作用户和系统管理员这三个主要的参与者。普通用户主要的操作任务是个人笔记的管理维护、信息检索与共享、专栏分类的创建和更新等，协作用户在此基础上又增加了协同编辑的功能，借助智能化的辅助工具大大提高了工作效能。系统管理

员是系统运维管理、安全监管的负责人。以上结构清楚地体现出各个角色的功能定位以及相互之间的关系，而协作用户作为重要的用户群体，具有其它用户所没有的特殊权限优势。

3.2 功能需求分析

3.2.1 功能概述

本系统的功能分为四大模块：

核心基础模块

它是内容架构重要的支撑部分，它的主要功能包含栏目结构的设计，标签体系的改善，收藏功能的加强，检索性能的改善，用户信息的管理以及关注关系的保存这些方面。

文章全生命周期管理模块

本文对文章全生命周期管理的各个方面做了详细的分析，包含内容创作中富文本编辑、多媒体融合技术的应用、稿件审阅进度实时监测、审核流程优化设计、首页内容推荐机制研究等。

用户交互与后台管控模块

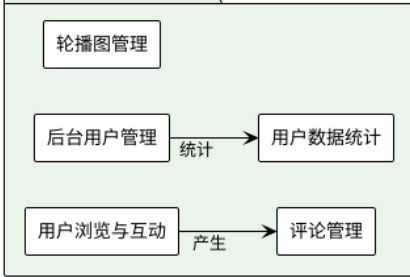
创建用户社群生态，实行平台运作的规范化管理，包含对用户行为数据的跟踪，改善用户交互体验的过程，加强评论内容审核监管的工作，后台用户及后台管理维护，分析数据精准实施，轮播图资源有效利用等诸多方面的内容设计及创建。

创新功能模块

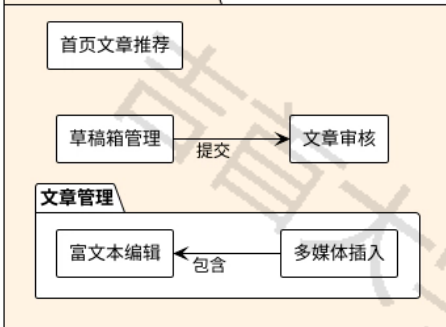
本文主要攻克了两个技术难点，一个是用CRDT理论为基础的实时协同编辑模式，另一个是将大语言模型嵌入进去的智能辅助系统。

智能云笔记系统功能模块图

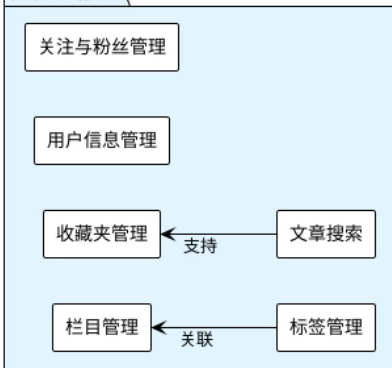
用户交互与后台管控模块



文章全生命周期管理模块



核心基础模块



创新功能模块



图3-2 系统功能模块结构图

图3-2清楚地体现了智能云笔记系统功能模块的层次结构，用树状图的方式进行展示。本平台的顶部节点为智能云笔记平台，分为四个部分，分别是栏目编辑、标签分组、收藏夹设和搜索全文内容等功能，注册用户信息资料提交管理、系统后台管理员操作界面设计。整个平台的功能有草稿制作、发表、审核流程、浏览行为、查看视频内容以及评论、不同权限等级、各类统计图、多种展示形式等各个方面来满足各个方面的需要。创新型特色模块有多人在线协作编辑、利用AI辅助创作等多种方式。分层架构的设计既提升了系统功能组织的逻辑性，又凭借引入的新特性对产品的市场竞争力产生了很大影响，并且能有效地克服了传统同类应用的技术缺陷。

系统总体用例图参考3-1。

3.2.2 用例分析

用例描述：

表3-2 用例描述

用例编号	用例名称	参与者
UC0	登录与注册	普通用户、协作用户、系统管理员
UC1	管理个人笔记	普通用户、协作用户
UC2	协同编辑	协作用户
UC3	使用AI辅助	协作用户
UC4	内容互动	普通用户、协作用户
UC5	专栏与标签管理	普通用户、协作用户
UC6	推荐系统	普通用户、协作用户
UC7	系统管理	系统管理员

UC0：登录与注册

表3-3 登录与注册用例

项目	内容
用例编号	UC0
用例名称	登录与注册
参与者	普通用户、协作用户、系统管理员
前置条件	用户未登录系统
后置条件	用户成功登录并获得对应角色权限
	1. 用户进入注册页面
	2. 用户填写用户名、邮箱、密码等信息

基本流程（注册）

基本流程（登录）	3. 系统验证信息有效性
	4. 系统发送验证邮件
	5. 用户完成验证，账号激活
	1. 用户进入登录页面
	2. 用户输入账号和密码
备选流程	3. 系统验证身份信息
	4. 系统根据用户角色跳转对应首页
	3a. 密码错误三次，账号临时锁定15分钟
业务规则	3b. 用户可点击“忘记密码”进行密码重置
	BR1：用户ID唯一，长度1-20字符
	BR2：密码需包含字母和数字，长度8-16字符

UC1：管理个人笔记

表3-4 管理个人笔记用例

项目	内容
用例编号	UC1
用例名称	管理个人笔记
参与者	普通用户、协作用户
前置条件	用户已成功登录系统
后置条件	笔记内容被正确保存或发布，笔记属性更新
基本流程	1. 用户进入笔记管理页面
	2. 用户可创建新笔记，输入标题和正文内容
	3. 用户可设置笔记属性（所属专栏、标签、文章属性）
	4. 用户可选择将笔记保存到草稿箱或直接发布
	5. 用户可对已有笔记进行编辑或删除操作
备选流程	6. 用户可通过关键词搜索已有笔记
	2a. 用户编辑已有笔记，系统自动保存
	3a. 用户未设置专栏/标签，系统强制提醒
	4a. 草稿箱中的笔记可稍后继续编辑并发布
	5a. 删除笔记前需二次确认
包含关系	发布文章、编辑文章、删除文章、搜索文章
	BR1：笔记标题不能为空且长度不超过100字符
	BR2：笔记正文不能为空
业务规则	BR3：普通用户笔记默认权限为“公开”

UC2：协同编辑

表3-5 协同编辑用例

项目	内容
用例编号	UC2
用例名称	协同编辑
参与者	协作用户
前置条件	协作用户已登录，拥有某篇笔记的协作权限
后置条件	多用户共同完成文档编辑，所有修改实时同步
基本流程	1. 协作用户进入某篇笔记编辑页面 2. 用户点击“分享” 3. 系统生成协作链接并复制到粘贴板上，用户可将链接发送给其他用户 4. 受邀用户点击链接并接受邀请，进入协同编辑模式 5. 所有协作者可实时看到彼此的光标位置和编辑内容 6. 协作者共同编辑文档，系统实时同步所有修改 4a. 用户拒绝邀请，不进入编辑会话

图3-3 协同编辑用例图说明

此框架将文书层次权限（owner、edit、view）所对应的使用场景做了系统的区分，拥有者（owner）可以执行协作文档创建、邀请函发送、权限设置等关键操作，编辑者（edit）主要是对文档内容做实时修改，查看者（view）只能查看文档的只读部分，不能修改或者新增内容。

UC3：使用AI辅助

表3-6 AI辅助用例

项目	内容
用例编号	UC3
用例名称	使用AI辅助
参与者	协作用户
前置条件	协作用户已登录，处于笔记编辑状态
后置条件	AI处理结果正确应用到编辑区域
基本流程	1. 用户在编辑区选中需要处理的文本内容 2. 用户点击AI辅助功能按钮 3. 系统弹出AI功能菜单，包含：润色、翻译、总结、扩写、纠错 4. 用户选择具体功能 5. 系统调用讯飞星火大模型API处理请求 6. 系统将AI生成的结果展示给用户 7. 用户可选择“替换选中文本” 4a. 用户选择“总结”（Summarize），系统自动生成100-200字摘要

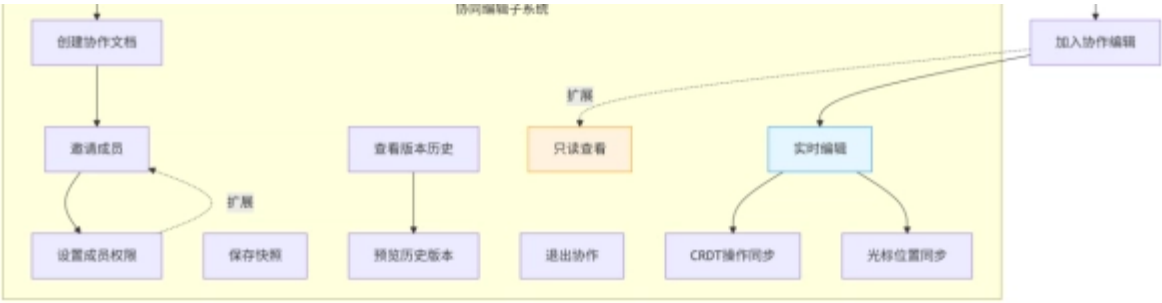


图3-4协同编辑核心用例图

图3-4给出了协同编辑主要功能模块设计思想和具体实施办法。文档初始化时文档所有人可以向目标成员发个性化邀请，并且可以设置不同的权限。根据权限来分，参与者分为两组，一组是拥有编辑权限的用户，即Edit权限用户，可以对实时内容进行即时的修改和更新；第二类是拥有查看权限的用户，即View权限用户，只能查看，不能修改。所有的用户都可以查阅文档的历史版本及变化过程。系统依靠自动化快照的技术，在没有人干预的情况下就可以迅速地把数据备份、生命周期管理工作做好。本文后面会给出一些具体的例子来解释不同的权限等级应用场合。

UC4：内容互动

表3-7 内容互动用例

项目	内容
用例编号	UC4
用例名称	内容互动
参与者	普通用户、协作用户
前置条件	用户已登录，可访问公开笔记
后置条件	互动数据（点赞数、收藏数、评论内容）正确更新
基本流程	1. 用户浏览公开笔记列表或单篇笔记详情页 2. 用户可对感兴趣的笔记点击“点赞”按钮（LikeArticle） 3. 用户可点击“收藏”按钮将笔记加入个人收藏夹（CollectArticle） 4. 用户可在笔记评论区输入内容并提交评论（CommentArticle） 5. 用户可浏览其他用户的主页，点击“关注”成为其粉丝（FollowUser） 6. 用户可在消息管理（MessageManage）中查看互动通知
备选流程	2a. 用户再次点击“点赞”取消点赞 3a. 用户可在收藏夹管理中对已收藏笔记进行分类整理 4a. 用户可删除自己发布的评论

	5a. 用户可取消关注已关注的作者
包含关系	点赞文章（LikeArticle）、收藏文章（CollectArticle）、评论文章（CommentArticle）、关注用户（FollowUser）被UC4包含
业务规则	BR1：同一用户对同一篇文章只能点赞/收藏一次 BR2：评论内容不能为空且不超过500字符 BR3：用户不能关注自己 BR4：违规评论可被用户举报

UC5：专栏与标签管理

表3-8 专栏标签用例

项目	内容
用例编号	UC5
用例名称	专栏与标签管理
参与者	普通用户、协作用户
前置条件	用户已登录系统
后置条件	专栏或标签被正确创建、修改或删除
基本流程	1. 用户进入专栏管理页面 2. 用户可创建新专栏，设置专栏名称、封面、描述 3. 用户可编辑或删除自己创建的专栏 4. 用户进入标签管理页面 5. 用户可创建、编辑、删除个人标签 6. 用户可在笔记中为文章添加或移除标签
备选流程	5a. 系统提供常用标签推荐 6a. 一篇文章可关联多个标签
包含关系	专栏管理（ManageColumn）和标签管理（ManageTag）独立
业务规则	BR1：每个用户最多创建10个专栏 BR2：专栏名称长度2-20字符 BR3：标签名称长度1-10字符，每个用户最多50个标签 BR4：删除专栏时，其中的文章需先转移或提示确认

UC6：推荐系统

表3-9 推荐系统用例

项目	内容
用例编号	UC6
用例名称	推荐系统
参与者	普通用户、协作用户
前置条件	用户已登录，系统已收集足够的用户行为数据
后置条件	系统为用户生成个性化文章推荐列表
基本流程	1. 用户进入系统首页或推荐页面 2. 系统基于用户的浏览、点赞、收藏、评论等历史行为 3. 系统采用基于物品的协同过滤算法计算文章相似度 4. 系统生成个性化推荐列表展示给用户 5. 用户可浏览推荐内容并进行互动 6. 用户的互动行为反馈至系统，优化后续推荐
备选流程	2a. 新用户无历史行为时，系统采用热门文章推荐（冷启动策略）
业务规则	BR1：推荐列表每24小时更新一次 BR2：每次推荐展示5篇文章 BR3：基于物品协同过滤算法使用文章标签、分类、互动数据计算相似度

UC7：系统管理

表3-10 系统管理用例

项目	内容
用例编号	UC7
用例名称	系统管理
参与者	系统管理员
前置条件	管理员已通过登录认证并在登录页面选择后台
后置条件	管理操作生效，系统状态更新
基本流程	1. 管理员登录后台管理系统 2. 管理员可查看和操作用户账号：禁用、启用、删除 3. 管理员审核用户发布的内容：审核通过/驳回 4. 管理员处理用户提交的举报信息。 5. 管理员对违规文章进行操作：下架 6. 管理员对违规评论进行操作：删除 7. 管理员查看平台运营数据统计

	8. 管理员配置首页轮播图内容
	9. 管理员操作完成后可登出系统
备选流程	8a. 轮播图支持图片上传
子用例	管理员登录、管理员登出、用户管理、内容审核、举报管理、 文章管理、评论管理、数据统计、轮播图管理
业务规则	BR3: 轮播图最多同时展示5张 BR4: 数据统计每日凌晨自动更新 BR5: 举报处理时限为48小时内

3.3 对象模型分析

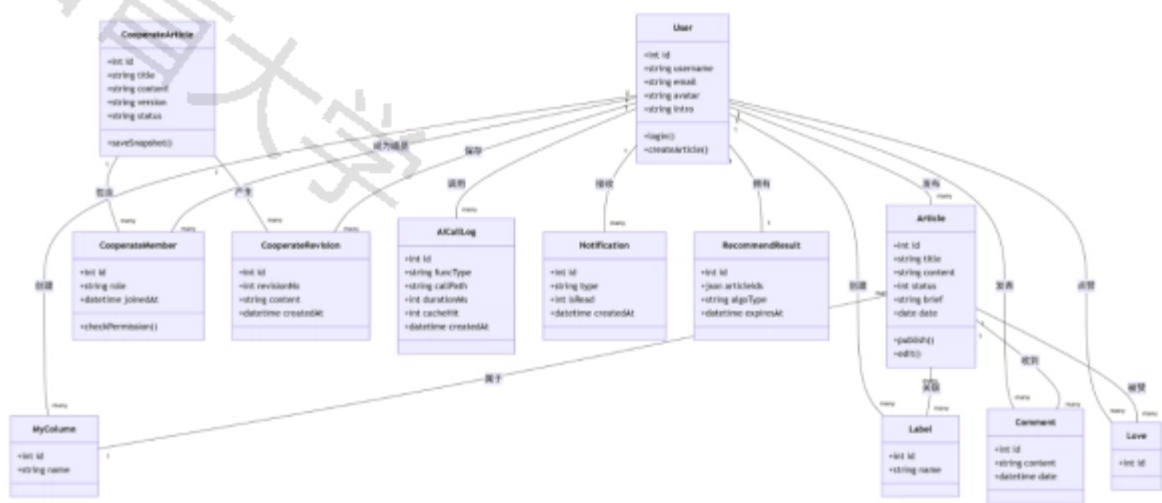


图3-5 领域类图

图3-5展示了系统的领域类图，核心领域概念包括：

博客系统体系结构中把“Article”定为核心数据模型，主要是依靠标题、内容以及发布状况这些重要属性来实现信息的价值。

协作文献属于某个领域的重要研究主体，它最主要的功能就是把各个方面的用户资料以及有关的协作要素进行有条理的整理、加以保存，并且形成完备的数据支持结构。

Controller：处理 HTTP 请求，例如 ArticleController 和 CooperateArticleController。

DAO（Data Access Object）：数据访问层，负责与数据库交互，包含 ArticleDao 和 CooperateArticleDao。

服务层（暂称）只关注处理业务逻辑里的事务工作，功能架构还未完全确定，但是可以预期它的主要目标就是保证业务流程的顺利进行以及有效协调。

该类工具的主要功能就是集中处理和业务逻辑无关的操作，比如文章推荐等功能模块所支撑的工作。

R：通用返回结果类，用于封装 API 响应。

3.4 非功能需求分析

非功能性需求指的是软件系统质量特性、运行环境兼容性、资源分配效率以及外部交互标准等有关的规定与期望，它的主要内容有如下几个方面：

3.4.1 性能需求

响应时间

- 常规操作（页面加载、存文章）：2秒以内
- 搜索操作：3秒以内
- AI功能（因为要调外部API）：10秒以内

并发支持

- 系统能支持至少100个用户同时进行常规操作
- 支持10个人同时编辑同一篇文章

资源消耗

- 前端首页的加载时间不能大于3秒，系统的内存峰值不能超过200MB。
- 后端服务CPU占用正常情况不超过50%

3.4.2 可靠性需求

可用性

本系统的建成已经达到99.5%的高可用性，非计划停机时间也被控制在每月4小时之内。

数据持久性

用户数据用长期保存方式来存储，系统每天凌晨备份数据库一次，备份时间在前30天内的历史备份。

故障恢复

本系统加入故障诊断模块和自愈功能，服务出现意外重启之后依旧可以保持数据的完整和一致。

3.4.3 易用性需求

界面设计具有很强的用户友好性，布局直观、指引清楚大大提高了新用户的使用效率和学习难度。

核心功能模块操作指南以及智能辅助功能的运用案例，都有着很高的参考意义。

3.4.4 安全性需求

身份认证：采用JWT（JSON Web Token）机制进行用户身份认证，Token有效期为7天。

权限控制框架就是只允许用户访问自己的私人笔记以及别人公开发布过的笔记，但是合作的人员只能对经过授权的文档进行编辑。

数据安全保护机制用BCrypt算法和随机生成的盐值把敏感信息（密码等）做哈希处理存起来，并依靠HTTPS协议对数据传输全过程加以全部路径加密保护。

3.4.5 运行环境约束

客户端：支持Chrome、Firefox、Edge等主流浏览器（最近两个版本）

服务端：部署在Linux服务器（CentOS 7+或Ubuntu 18.04+），需要Java 17+、Python 3.9+、MySQL 8.0+运行环境。

为了保证客户端网络通信的效率和稳定性，客户端需要的互联网环境最低的带宽为2Mbps，可以满足日常使用，

也能提高运行速度。

3.5 可行性分析

3.5.1 技术可行性

协同编辑技术

CRDT算法有现成的成熟实现Yjs，带了完整的WebSocket传输层（y-websocket）和Quill编辑器绑定（y-quill），支持多人实时同步，技术风险不高。Yjs已经在Figma、Notion这类产品里用过了，稳定性有保障。

AI功能实现

讯飞开放平台的中文文本纠错API具有很高的稳定度，可以准确地找出各种语义错误，并且用结构化的形式来呈现校正结果。Elysia Tools依托RESTful API与Server-Sent Events（SSE）实现多语言翻译及功能扩展，其官方文档体系完备且具有高度实用性。使用Python的Flask框架搭建本地适配模块，可以快速集成以上提供的服务资源，并且可以实现统一的接口管理，该技术方案的技术成熟、易于后期维护升级。

技术栈成熟度

React 18、Spring Boot 3.x、MySQL 8.0、Docker都是业界主流技术，项目成员在课程项目里已经用过这些技术，有能力实现。

3.5.2 操作可行性

访问方式

本系统使用B/S架构进行创建，使用者可以利用Chrome、Firefox和Edge等主流浏览器来完成对系统的访问，不需要事先下载专门的客户端软件，使得系统的使用更加方便，更加普及。

前端交互

该前端界面用Ant Design 5.x组件库为主体来创建，它的交互逻辑符合传统的软件设计标准，文本选择之后就会产生AI辅助菜单的出现，协同编辑模式当中，系统可以对用户光标位置的改变实行即时同步的操作。消息中心是经典的左右分屏方式侧边栏样式。新用户完成注册、内容创作和AI功能使用等基本操作之后，不需要进行专门的培训就可以很快地学会使用各个功能模块。

后台管理

管理员后台用可视化的数据表格加操作按钮的方式呈现主要的功能模块，有文章审核、用户管理、数据统计等模块。界面设计重视简约性以及直观性，给用户营造出一个高效、友好的交互环境。

3.5.3 经济可行性

本文建立了一个用硬件投入成本和潜在的人力节省价值作为评价标准的双重分析框架，从两个方面对项目的经济可行性进行了考察。

本系统使用全开源的技术栈来创建，没有商业授权费的支出。各个服务模块使用Docker容器化的方式进行部署，大大降低对高性能计算的要求。根据典型云服务器环境下的测试结果可知，该类设备的月度运维成本有明显的降低现象，有关数值如表3-10所示。

表3-11 主要月度运营成本估算

成本类别	配置/规格	估算月费用（元）
------	-------	----------

云服务器（含带宽）	4核8G，10Mbps带宽（如阿里云/腾讯云学生机）	100-150
第三方AI API调用	本地缓存35%命中率，日均千次调用	150-200
域名与SSL证书	普通.com域名 + Let's Encrypt免费证书	<10
合计		约260-360

本预算规划主要为个体开发者和小团队提供服务，如果使用高等院校提供的免费云服务资源，那么总成本可以得到有效控制，也可以得到很大的减少。

本文是从学生团队协作的角度出发，就系统的人员替代效能做了全方位的评价。以五人规模的软件工程课程学习小组为研究对象，发现使用该系统之后文档整合、格式统一等繁琐的手工工作被彻底改善。采用传统的异步协作方式的时候，每个人每周都会消耗约两小时的时间来处理文档的修改和校对工作，但是使用实时协作以及智能纠错模块之后，一次的任务所花费的时间会缩短到每个人半个小时左右。根据上面的计算，在一个月里小组成员利用改进方法可以节约总时数30小时（5名成员×1.5小时/周×4周）。这部分时间被用作设计或者编码开发等高附加值的活动，所产生的社会经济效益要比系统研发初期所付出的代价大得多。

从以上的分析可以看出，本系统在硬件投入上具有较大的成本优势，提高人员利用效率从而改善整个成本结构，是具有很好的经济效益和社会效益。

3.6 系统数据流图

本节用数据流图（DFD）分析的方法来分析和外界环境之间是如何相互作用的，内部数据的流动情况又是怎样呈现出来的。

3.6.1 顶层数据流图（上下文图）

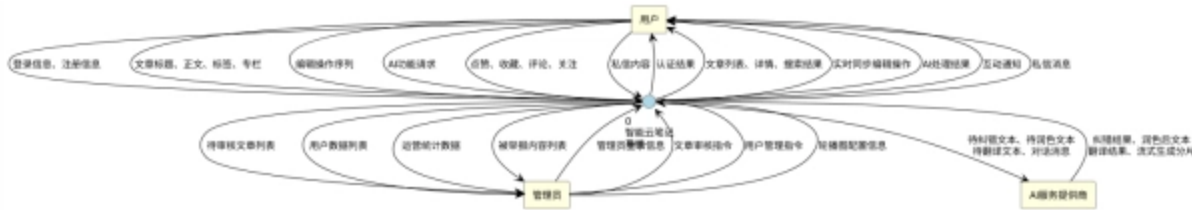


图3-6 系统顶层数据流图

图3-6给出了本文系统的主要外部交互对象有三类，分别是用户群体主要是进行指令或者文本的输入操作，然后通过界面来接收同步或者人工智能解析后的结果展示，管理员主要进行任务审核工作，可以查看到待审批的数据信息，AI技术服务提供方需要对数据进行分析并给出处理意见。该图对系统的外部环境做出了详细的描述，也给出了各个主要的功能模块。

顶层数据流图把0标记成整个系统的主要功能模块，体现各个层次模块的功能。外部的参与者主体是由三大主体所组成的。

用户将向系统提交的登录凭证、稿件素材、编辑指令、人工智能相关的请求、社交互动信息（私信内容等）和各种通知、私信回复等都属于用户与系统进行交互的过程中。

管理员依靠执行内容审核，用户权限设定以及轮播图管理这些功能模块来对文章实行审核工作，并且可以对用户的资料展开更新操作并且还能获取到各种各样的统计报表数据方面的支撑。

服务提供者要对系统得到的文本、对话数据加以分析处理，把其中的内容校正、优化改善、多语种翻译、结构化切割等各方面成果反馈给平台。

标识数据流动的词语一般使用名词性的短语来表达，突出信息的主要特征。

3.6.2 0层数据流图

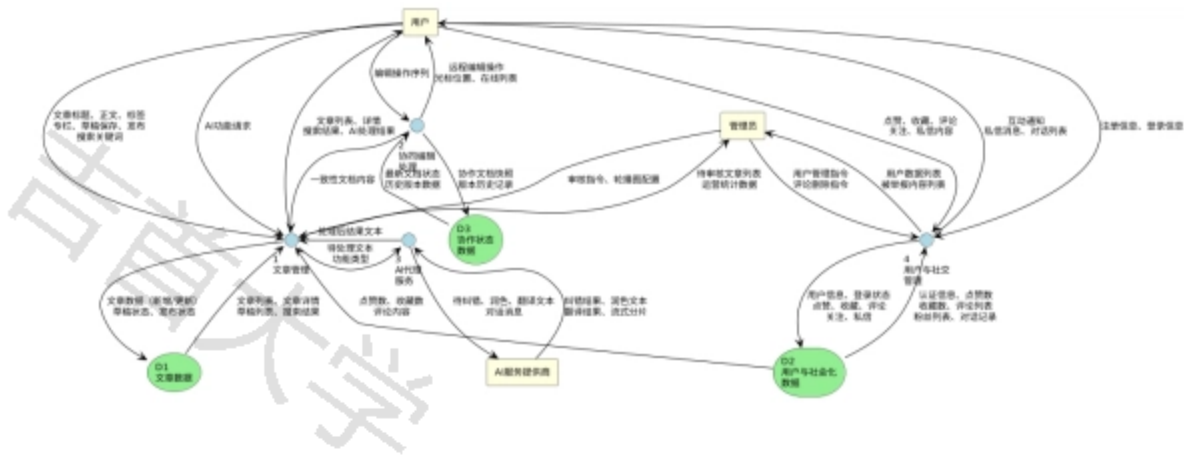


图3-7 系统0层数据流图

如图3-7所示，本系统可以分为四个主要的模块即文章管理、协同编辑、AI代理服务和用户管理。根据对关键数据流动路径的分析，用户输入在协同编辑过程中经过一定的逻辑转化为操作记录，并传送到协同编辑处理环节，只有当同步任务完成之后，文档最新的一致性状态才会被推送至文章管理环节，达到数据持久化存储和更新的目的。本层主要对外部的AI请求进行了分派，并且对它的执行过程也作了说明。

由图3-7可知，在第0层DFD模型中把系统分为四个主要功能模块和三个主要的数据存储实体。

模块P1主要是接收用户上传的文章数据以及用户对AI服务调用请求，然后同数据存储模块D1进行互动从而达成对文章进行添加、查看、编辑、删除的操作，管理相关的业务流程的转移。该模块把AI相关的任务交给模块P3来处理，然后整合一致性的检验模块P2产生的内容并执行持久化存储的任务。

P2是协作编辑处理单元，用WebSocket技术和CRDT算法来处理数据的冲突检测和融合。经过优化之后的更新记录以消息推送的方式向所有的合作成员发出。系统对D3（协作状态数据库）周期性地保存文档的各历史版本快照，在完成编辑工作之后，就形成了最新的统一版本，并将其存放到P1（核心服务器）。

模块P3（智能代理服务）的主要功能是接收到模块P1的AI任务请求之后，对原始数据做预处理、封装工作，再用外部的AI服务做核心运算，然后以一定的格式将运算结果重组返回给系统，同时同模块P1进行交互来完成业务流程闭环。

模块P4主要是为了用户身份认证、点赞、收藏、评论、关注、私信等功能提供支持，并且与模块D2一起完成数据持久化存储、用户通知机制、相关业务逻辑等的整合。

关键数据流路径

- 注册登录：用户 → P4 → D2 → P4 → 用户
- 协同编辑：用户A → P2 → 用户B（实时广播）；P2 → D3（快照持久化）；P2 → P1 → D1（最终存储）

- 智能化辅助工作流程可描述为：用户 → P1 → AI服务提供方 → P3 → P1 → 用户。
- 社交互动：用户 → P4 → D2 → P4 → 用户

第4章 架构设计

4.1 系统架构及原理

4.1.1 整体架构

该系统使用分层架构体系，从下至上可以分为基础设施层、数据管理层、服务支撑层、业务应用层和用户交互层。

基础设施层

云服务器资源、操作系统平台、Docker容器技术和网络基础设施等都是系统运行的支撑性硬件，也叫做系统基础架构。

数据层

本系统主要是对数据进行存储与管理。就结构化数据而言，依靠MySQL数据库管理系统来对用户的资料、内容发布以及互动评论这类的数据加以有条理地保存；借助Redis缓存系统改善高并发环境下的会话状态更新速度以及热点查询的速度；依靠阿里云OSS对象存储服务完成多媒体文件以及非结构化数据的高效存储与分发。

服务层

这是系统的核心业务逻辑层，拆成了四个子服务：

主要业务模块用Java语言、Spring Boot技术和用户资料、内容创作和审核、互动留言和查询来实现。AI网关属于关键节点，它的主要功能是对前端自然语言处理请求（文本校对、语种转换、格式转换等）进行连接，并且还会做权限验证、用词筛选、带宽管控等预先的动作来把任务交由本地智能辅助系统来执行复杂的运算后得到结果。

利用Python和Flask框架构建的本地智能辅助适配器，该系统是为局域网而开发的，使用科大讯飞开放平台的主要技术和核心功能模块。标准RESTful API调用接口的桥梁，它的主要工作有向核心业务系统提供服务接口、做用户鉴权、对数据格式转换进行优化、使用缓存管理机制等。

协同服务（Node.js/WebSocket）：基于Yjs处理实时协同编辑的CRDT状态同步。

外部的AI辅助模块是以前端直连的方式进行设计的，其主要由Elysia Tools的SSE接口实现文本拓展和摘要生成的功能，并且使用讯飞星火大模型的WebSocket接口来创建智能客服的交互方式。

表4-1 AI能力的三路径加购

路径	功能	流向	协议	理由
后端代理路径	纠错、润色、翻译	前端→核心业务服务→本地AI辅助功能→讯飞API	HTTP同步	需权限校验、敏感词过滤、统一日志
前端直连路径	扩写、总结	前端→Elysia Tools	SSE流式	减少延迟，端到端流式体验
前端独立路径	AI小助手	前端→讯飞星火	WebSocket长连接对话	与编辑功能解耦

应用层：前端应用程序，包括：

•单页应用是用React框架开发的Web应用程序，在桌面和移动设备浏览器上可以支持多终端适配。

表现层是用户交互的重要载体，主要是界面布局、组件设计和各种的人机交互元素等。

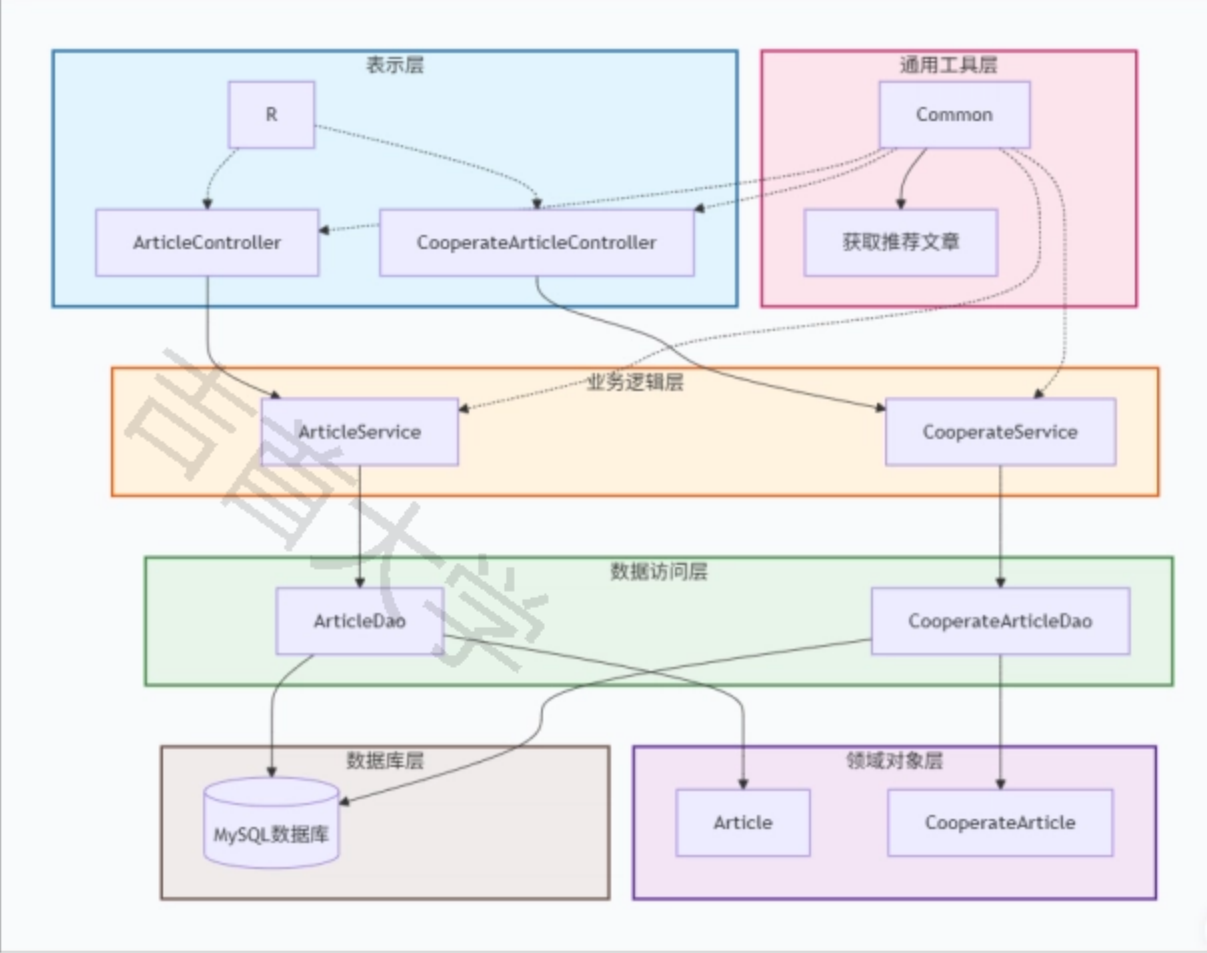


图4-1 分层架构图

图4-1所示系统使用分层架构，总体上呈现出水平分布的形态，由上到下可以分为表现层、应用层、服务层、数据层以及基础设施层这五个部分。前端界面使用React框架创建交互式的组件，在业务逻辑上用Java语言实现，并且用Python的本地AI模块增强了智能化的分析能力，事务处理采用Node.js技术栈的高效调度方式，数据库方面主要用MySQL做为主数据库，Redis做为辅助的存取速度提升工具，同时使用阿里云OSS来存储非结构化的数据，外部服务集成上采用Elysia Tools等第三方工具配合讯飞开放平台的语音识别转文本功能一起完成系统的正常运转。

就系统架构来说，本地AI辅助模块已经可以对外部API接口进行解耦以及适配的功能，核心业务服务使用AI网关做身份认证、权限管理和流量控制的工作，前端交互路径有效地减小了后端服务器的负担，改善了流媒体数据传输速度和整体用户体验水平。

4.1.2 核心设计映射

为提高软件工程项目的技术可追溯性，本文建立了完整的业务需求映射体系，从业务需求入手，从业务功能设计、所采用的技术手段、最终得到的数据模型等各个角度来联系起来。该框架不但是后续模块化设计的统一标准，而且保证技术实施过程有条理、有规范。

表4-2 核心需求-功能-技术-数据映射表

业务需求	核心功能	关键技术选型	核心数据模型	关键测试用例
模块				
UC2: 协同编辑	编5.2 协作CRDT (Yjs), y-websocket		Cooperate_Article, Cooperate_ArticleRevision	CT-01至CT-16, 表6-6 性能测试
UC3: 使用AI辅助	助5.3 AI辅LLM /WebSocket, Flask	API, SSE (无独立模型, 依赖Article.content)		AI-01至AI-15, AS-01至AS-10
UC1: 管理个人笔记	个5.1 文章Quill, REST API	Spring Boot	Article, Label, MyColumn	FT-04至FT-10
UC6: 推荐系统	系2.4 推荐Item-CF, 时预计算	冷启动策略, 定	Recommend_Result, Article_Similarity	表2-6离线评估
UC7: 系统管理	管5.5 后台MyBatisPlus	Spring Boot,	Article.status, User	FT-24, FT-25

4.1.3 技术架构

本系统采用前后端分离架构，用到RESTful API协议以及WebSocket通信技术，从而达到前端和后端的数据交换目的。

前端技术栈：

- 核心框架：React 18，采用函数式组件和Hooks机制
- 开发语言：TypeScript，提供静态类型检查
- UI组件库：Ant Design 5.x，提供企业级UI组件
- 状态管理：Zustand，轻量级状态管理方案
- 路由管理：React Router 6，处理前端路由
- HTTP客户端：Axios，处理API请求
- 协同客户端：Yjs + y-websocket，实现协同编辑客户端功能

•王一凡搭建起富文本编辑框架，使用了WangEditor、Quill等技术，主要是为了创建出可以实现多人同时工作的高效文档撰写平台。

后端技术栈：

- 核心框架：Spring Boot 3.x，提供自动配置和起步依赖
- 开发语言：Java 17
- 数据持久层：MyBatisPlus，简化数据库操作
- 安全框架：Spring Security + JWT，处理认证授权
- API文档：SpringDoc OpenAPI，自动生成API文档
- 协同服务端：Yjs的y-websocket服务端，处理WebSocket连接和CRDT同步

AI辅助功能技术栈：

- 核心框架：Flask 2. x，轻量级Web框架
- 开发语言：Python 3. 9+
- 21HTTP客户端：Requests，调用外部AI API
- 22主要依赖：各AI辅助功能提供商的SDK

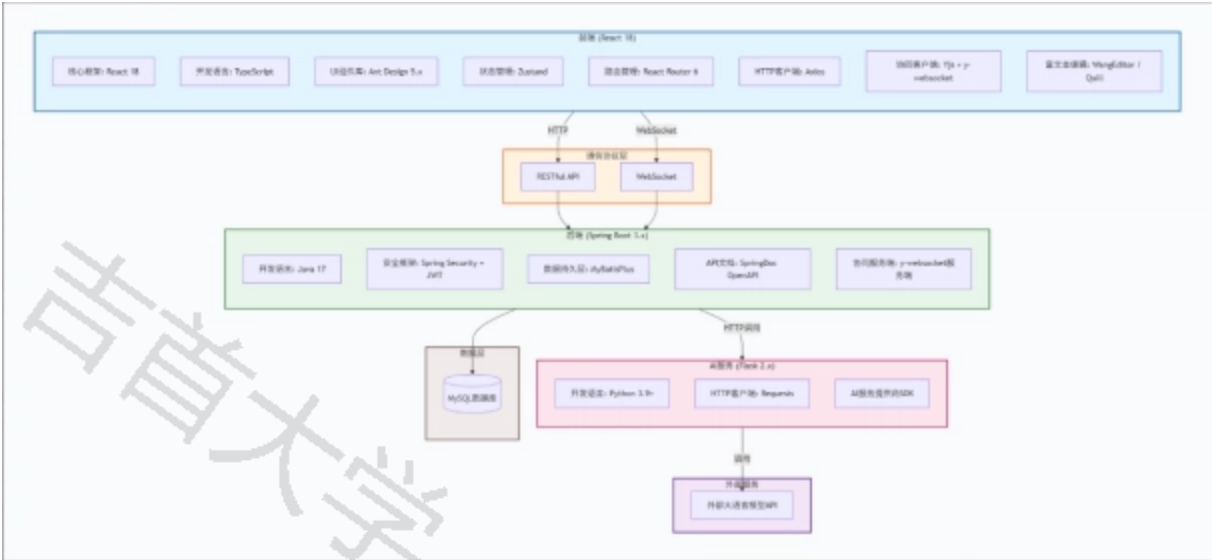


图4-2 技术架构图

图4-2给出了本系统所采用的技术栈，以及各个层次之间技术栈的组合方式。前端部分依托React 18、TypeScript、Ant Design 5.x、Zustand、React Router 6及Axios等主流框架与库构建，同时结合Yjs + y-websocket实现多用户实时协作功能；编辑器选用WangEditor或Quill作为核心组件。后端服务基于Spring Boot平台开发，使用Java 17作为主要编程语言，并通过MyBatis Plus提升数据库操作效率，采用Spring Security配合JSON Web Tokens (JWT) 实现用户身份认证机制，利用SpringDoc OpenAPI工具自动生成API文档规范。本地AI模块使用的是flask2.x框架、Python 3.9以上版本及科大讯飞开放平台SDK等扩展库。协同通信模块使用了nodejs驱动的yjs y-websocket组件来进行高效的同步、传输工作。从数据库管理角度出发，在关系型数据库管理系统上先选用MySQL 8.0社区版，使用Redis进行数据缓存提升读写速度，同时采用阿里云对象存储服务OSS来提高数据备份以及容灾的能力，在保证系统的高可用性的同时达到资源的合理配置和优化利用的目的。上述技术选择完美适配了业务场景的要求，Yjs是针对跨设备、多终端文件同步的，Flask是对外接口的封装调用，JWT是整个交互过程中的认证手段，并且对所有的请求都起安全防护的作用。

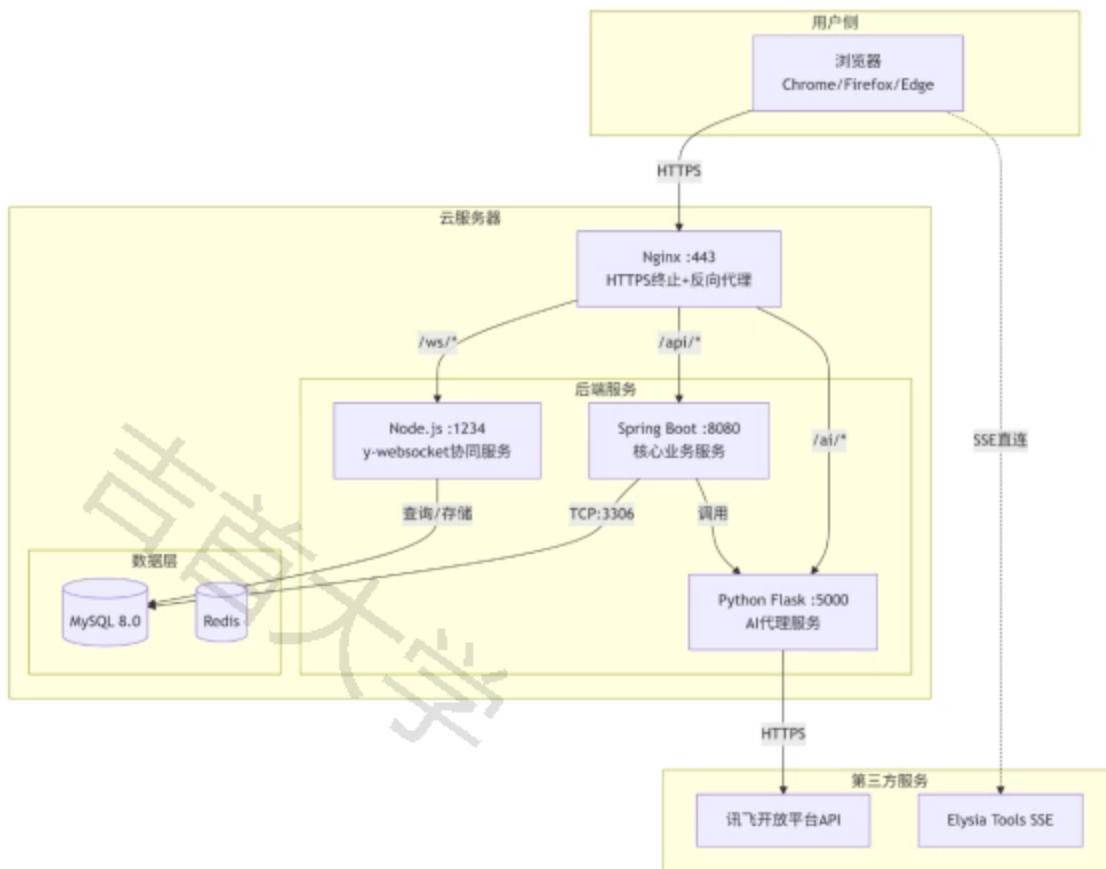


图4-3 系统部署架构图

图4-3 展现了各类服务的部署架构及其交互机制。Nginx承担起集中式的反向代理服务器职责，根据路径匹配规则把请求分发给对应的端口服务，即 `/api/` 请求指向的是基于Spring Boot搭建好的应用程序，`/ws/` 开头是WebSocket实时通信请求指向Node.js开发的协同模块，最后符合`/ai/*`规范请求交给PythonFlask微框架进行处理并做出回应。为了使SSE（Server-Sent Events）这类对实时性有较高要求的业务场景得到更好的实现，系统采用浏览器直接和Elysia Tools组件进行连接的方式，而不再使用传统的后端调用流程。核心数据库（MySQL）和分布式缓存（Redis）都是内网隔离的，且都关闭了所有的公共网络端口；所有的外部访问请求都要经过HTTPS协议的安全验证后才能进行隧道化的传输。

4.2 业务用例的实现

4.2.1 协同编辑用例实现

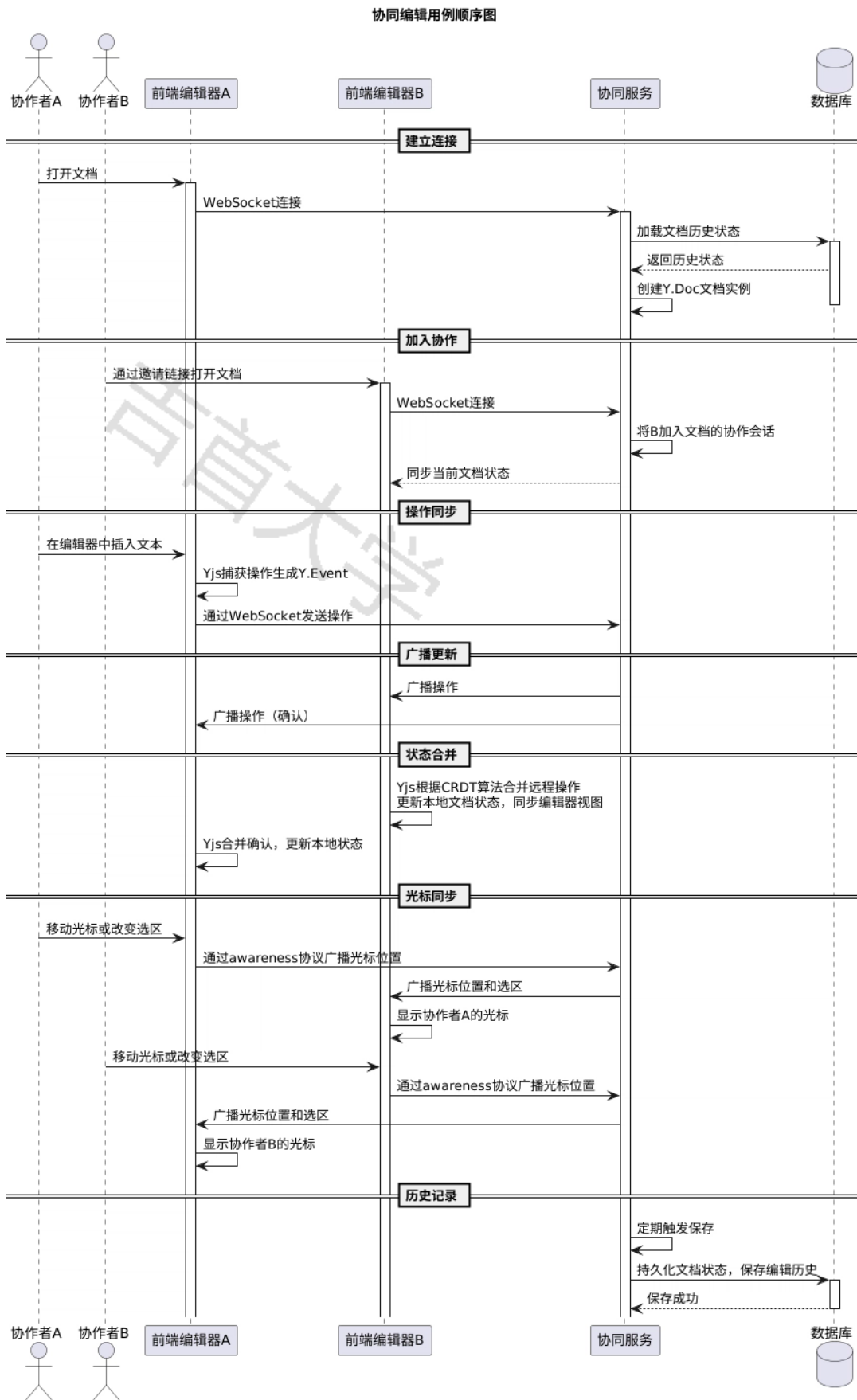


图4-4 协同编辑时序图

图4-4为多用户协同完成同步编辑完整交互流程的关键顺序图，主要体现各个功能模块具体的运行过程。

用户A在访问文档的时候，前端用WebSocket协议同协同服务端创建双向通信通道。若Y.Doc文档没有产生，就通过数据库查询它的历史版本记录来完成初始化。

协作模式使用的是邀请链接访问同一文件的用户B，在完成WebSocket通信连接之后，协同处理模块就将用户B加入到当前文档的协作交互序列当中。

协作同步方案，用户A在编辑端输入文本内容的时候，用Yjs库来获取用户的编辑动作，然后创建一个Y.Event事件对象。之后使用WebSocket通信协议把上面提到的事件信息立即发给协同服务端，从而达到同步交互的目的。协同播发机制，就是服务请求用同步的方式分发给所有的关联协作方（B方），以达到信息一致性的同步目的。

CRDT机制用于状态集成，在客户端使用CRDT算法完成远程操作的协同，然后更新本地文档的状态信息，从而使编辑界面达到同步和动态优化的目的。

Awareness协议采用某种通信方式向所有的客户端发送用户光标位置和交互范围等相关信息，从而达到协作成员光标可视化展示的目的。

历史记录功能是协同服务平台进行周期性数据交互时，将文档目前的状态信息存入到数据库当中，并且可以保存所有版本的修改轨迹供查阅。

4.3 功能模块设计

第三章第二十一节图3-2已经清楚地给出了系统的功能模块结构框架，本节不再对具体的内容进行说明。各个模块技术实现细节在第五章已经做了详细的分析和说明。

4.4 数据库设计

4.4.1 概念设计

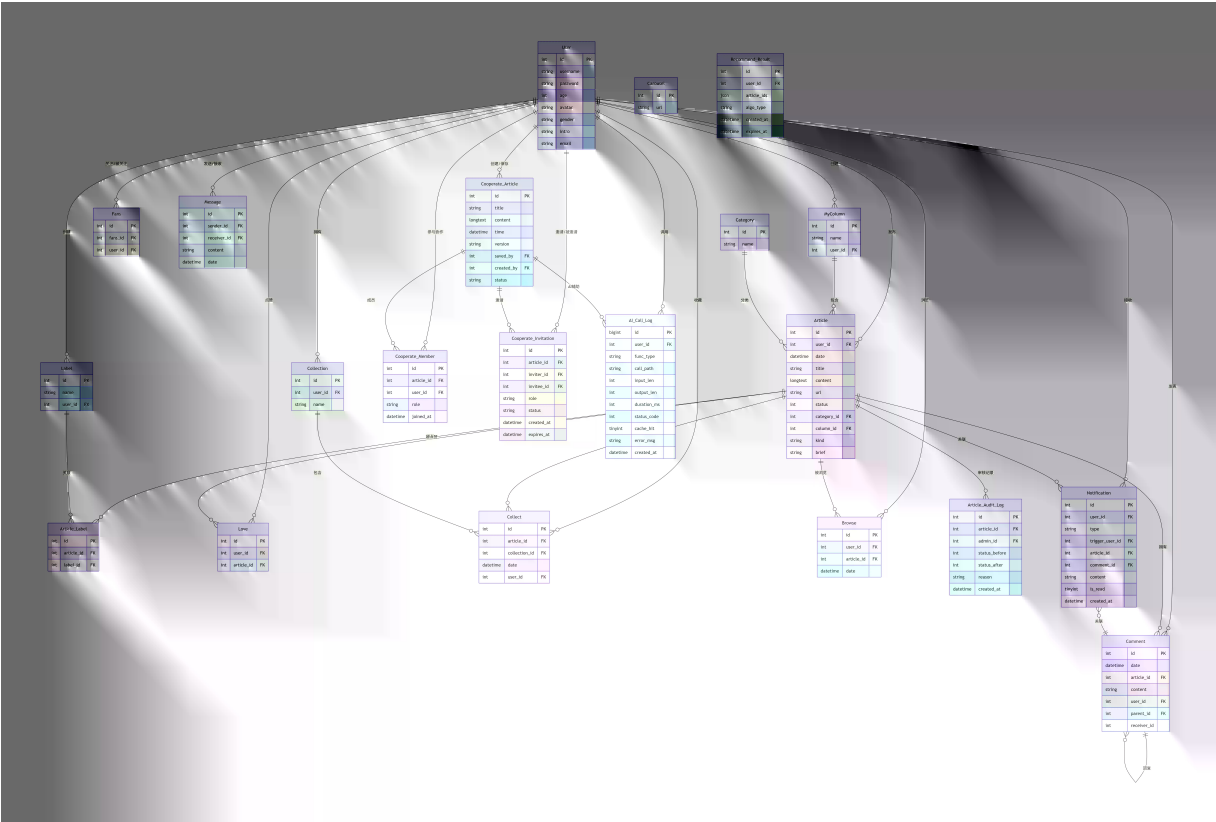


图4-5 系统ER图

图4-5给出了系统概念模型的实体关系图，其主要组成有以下几个方面，

用户表 (User)

用户ID (id) 是主键。其他字段：用户名 (username)、密码 (password)、年龄 (age)、头像URL (avatar)、性别 (gender)、个人简介 (intro)、邮箱 (email)。用户是系统的核心角色，能发布文章、创建专栏、管理标签、进行点赞收藏评论等互动。

文章表 (Article)

文章ID (id) 是主键。其他字段：发布用户ID (user_id, 外键)、发布日期 (date)、标题 (title)、正文内容 (content)、封面图URL (url)、状态 (status: 0草稿/1待审核/2审核通过/3审核失败)、分类ID (category_id, 外键)、专栏ID (column_id, 外键)、文章类型 (kind: 原创/转载/翻译)、摘要 (brief)。文章是平台的核心内容，支持发布、编辑、删除、搜索。文章有完整的生命周期：草稿→待审核→审核通过或审核失败。同时支持用户互动 (点赞、收藏、评论) 和分类管理 (专栏、标签、分类)。

专栏表 (MyColumn)

专栏ID设为主键字段，其余为栏目名、创建用户ID。因此用户可以利用不同的专栏来管理文件，但是每个文档只能属于一个专栏。

标签表 (Label)

核心字段为标签ID(id)，主要属性有标签名称(name)和创建用户ID(user_id)，user_id用作外键。用户可以自由地设置自定义标签，而自定义标签又可以同关联表实现多对多的映射。既可以提高内容检索速度，也可以使信息得到组织、分类地加以管理。

文章-标签关联表 (Article_Label)

主键是关联ID (id)。其他字段：文章ID (article_id, 外键)、标签ID (label_id, 外键)。这张表是文章和标签的中间表，用来做多对多关联。

评论表 (Comment)

主键是评论ID (id)。其他字段：评论日期 (date)、文章ID (article_id, 外键)、评论内容 (content)、评论用户ID (user_id, 外键)、父评论ID (parent_id, 外键，用来做嵌套回复)、接收者ID (receiver_id)。用户可以对文章发评论，也可以回复别人的评论。

点赞表 (Love)

本表以“点赞ID (id)”作为主键，包含两个外键字段：“用户ID (user_id)”与“文章ID (article_id)”。数据记录了用户对于某篇文章的点赞行为，用约束规则保证每个用户对于同一篇文章只能进行一次点赞。

收藏夹表 (Collection)

本表用收藏夹ID (id) 做主键，有用户ID (user_id, 与用户信息表建立外键)，收藏夹名称等字段。用户ID的加入是用来完成多对一的数据关系映射的，每一位注册用户都可以自由地创建出不同的分类收藏夹，再按照事先设定的标准保存各种各样的文章资源。

收藏记录实体 (Collect)

主键为收藏记录ID (id)，属性包括文章ID (article_id, 外键)、收藏夹ID (collection_id, 外键)、收藏日期 (date)、用户ID (user_id, 外键)。记录用户将文章收藏到指定收藏夹的行为。

粉丝关注实体 (Fans)

本文所建立的实体用“关注关系ID”为主键，它的属性有粉丝用户标识符 (fans_id, 外键)、目标用户标识符 (user_id, 外键)。该实体用来存储备份用户之间相互关注的关系，给社交平台的主要交互手段给予数据支撑。

收藏记录表 (Collect)

主键是收藏记录ID (id)。其他字段：文章ID (article_id, 外键)、收藏夹ID (collection_id, 外键)、收藏日期 (date)、用户ID (user_id, 外键)。记录用户把文章收藏到哪个收藏夹。

粉丝关注表 (Fans)

本表的关键字段为“关系ID” (id)，其余字段包括“粉丝用户ID” (fans_id, 作为外键)和“被关注用户ID” (user_id, 同样定义为外键)。其主要功能就是建立用户之间的关注关系，以此来支持社交平台上关注体系的运作。

浏览记录表 (Browse)

主键是浏览记录ID (id)。其他字段：用户ID (user_id, 外键)、文章ID (article_id, 外键)、浏览日期 (date)。记录用户看了哪些文章，用来做数据统计和推荐。

私信表 (Message)

主键是私信ID (id)。其他字段：发送者ID (sender_id, 外键)、接收者ID (receiver_id, 外键)、消息内容 (content)、发送日期 (date)。实现用户之间的私信聊天。

协作文章表 (Cooperate_Article)

主键是协作文章ID (id)。其他字段：标题 (title)、正文内容 (longtext)、创建时间 (time)、版本号 (version)、保存者ID (saved_by, 外键)、创建者ID (created_by, 外键)，还有status字段 (active /archived, 表示协作是否还在进行中)。支持多人实时协同编辑，记录版本历史。

就功能架构来说，Cooperate_Article和核心文档Article存在比较清楚的职能界限以及边界界定。Cooperate_Article没有进入到官方的内容审核过程之中，并且也不接入公共的交互平台，它的访问权限只给协作成员。两者分开的原因有二，一方面，文章Article有完整的发布审批过程来保证内容的合规性，而合作文章CooperateArticle只关注编辑协作环节，并且不考虑这个多余的步骤；另一方面，文章Article包含很多社交模块（点赞、分享、评论等），合作文章CooperateArticle专注于提高团队协作效率和流程管理，而合作文章Cooperate_Article还需要实现版本控制功能，但是文章Article不需要这个功能。虽然二者都是内容载体，但是其业务逻辑、数据结构和系统联结等各方面都存在着较大的差别。

轮播图表 (Carousel)

轮播图实体模型中把“id”定义为主键，以保证每一张图片都是唯一的标识。模型中有一个url字段，它的数据录入、维护工作是由系统管理员来完成的，从而达到首页轮播内容的管理。

分类表 (Category)

该数据库结构用分类ID (id) 来创建主键系统，“分类名称”这一字段是进行各个种类的文章主要类别区分的

工具。

协作成员表 (Cooperate_Member)

主键是记录ID (id)。字段：协作文章ID (article_id, 外键连cooperate_article.id)、用户ID (user_id, 外键连user.id)、协作角色 (role: owner/edit/view)、加入时间 (joined_at)。

协作邀请表 (Cooperate_Invitation)

主键是邀请ID (id)。字段：协作文章ID (article_id, 外键连cooperate_article.id)、邀请者ID (inviter_id, 外键连user.id)、被邀请者ID (invitee_id, 外键连user.id)、邀请角色 (role: edit/view)、邀请状态 (status: pending/accepted/rejected/expired)、创建时间 (created_at)、过期时间 (expires_at)。

AI调用记录表 (AI_Call_Log)

主键是记录ID (id)。字段：用户ID (user_id, 外键连user.id)、AI功能类型 (func_type: polish/translate/expand/correct/summary/chat)、调用路径 (call_path: backend_proxy/frontend_direct/frontend_ws)、输入文本长度 (input_length)、输出文本长度 (output_length)、调用耗时 (duration_ms)、HTTP状态码 (status_code)、是否命中缓存 (cache_hit: 0或1)、错误信息 (error_msg, 可为空)、调用时间 (created_at)。索引：(user_id, created_at)和(func_type, created_at)。

文章审核日志表 (Article_Audit_Log)

主键是记录ID (id)。其他字段：文章ID (article_id, 外键连article.id)、操作管理员ID (admin_id, 外键连user.id)、审核前状态 (status_before)、审核后状态 (status_after: 1待审核/2通过/3失败)、审核意见 (reason, 可为空)、操作时间 (created_at)。

通知消息表 (Notification)

主键是通知ID (id)。其他字段：接收者用户ID (user_id, 外键连user.id)、通知类型 (type: like/collect/comment/follow/system)、触发者用户ID (trigger_user_id, 外键连user.id)、关联文章ID (article_id, 外键连article.id, 可为空)、关联评论ID (comment_id, 外键连comment.id, 可为空)、通知内容摘要 (content)、是否已读 (is_read: 0/1)、创建时间 (created_at)。

推荐结果表 (Recommend_Result)

主键是记录ID (id)。其他字段：目标用户ID (user_id, 外键连user.id)、推荐文章ID列表 (article_ids, JSON格式存)、推荐算法类型 (algo_type: item_cf/hot/cold_start)、生成时间 (created_at)、有效期至 (expires_at)。

当前系统采用的推荐机制基于物品协同过滤 (Item-Based Collaborative Filtering, Item-CF) 模型, 其核心算法逻辑体现在Common.java文件中的getRecommendArticles()函数中。为了提高计算效率和减少对数据库资源频繁调用, 本文建立了临时表来存放推荐结果, 在日间执行任务调度程序来完成数据周期性的更新以及维护。

4.4.2 逻辑设计

主要关系模式如下:

用户表（user）

表4-3 用户设计表

字段名	数据类型	允许空	主键	说明
id	int unsigned	否	是	用户ID，自增主键
username	varchar(255)	否	否	用户名
password	varchar(255)	否	否	密码
age	int	是	否	年龄
avatar	varchar(255)	是	否	头像URL
gender	int	是	否	性别
intro	varchar(255)	是	否	个人简介
email	varchar(255)	否	否	邮箱

文章表（article）

表4-4 文章设计表

字段名	数据类型	允许空	主键	外键	说明
id	int unsigned	否	是	-	文章ID，自增主键
user_id	int	否	否	user(id)	发布用户ID
date	datetime	否	否	-	发布日期
title	varchar(255)	否	否	-	文章标题
content	longtext	否	否	-	正文内容
url	varchar(255)	是	否	-	封面图URL
status	int	否	否	-	状态：0草稿/1待审核/2 审核通过/3审核失败
category_id	int	是	否	category(id)	分类ID
column_id	int	是	否	mycolumn(id)	专栏ID
kind	varchar(255)	否	否	-	文章类型：原创/转载/翻 译
brief	longtext	否	否	-	文章摘要

专栏表（mycolumn）

表4-5 专栏设计表

字段名	数据类型	允许空	主键	外键	说明
id	int unsigned	否	是	-	专栏ID，自增主键

name	varchar(255)	否	否	-	专栏名称
user_id	int	否	否	user(id)	创建用户ID

标签表 (label)

表4-6 标签设计表

字段名	数据类型	允许空	主键	外键	说明
id	int unsigned	否	是	-	标签ID, 自增主键
name	varchar(255)	否	否	-	标签名称
user_id	int	否	否	user(id)	创建用户ID

文章-标签关联表 (article_label)

表4-7 文章-标签关联表

字段名	数据类型	允许空	主键	外键	说明
id	int unsigned	否	是	-	关联ID, 自增主键
article_id	int	否	否	article(id)	文章ID
label_id	int	否	否	label(id)	标签ID

分类表 (category)

表4-8 分类表

字段名	数据类型	允许空	主键	说明
id	int unsigned	否	是	分类ID, 自增主键
name	varchar(255)	否	否	分类名称

评论表 (comment)

表4-9 评论表

字段名	数据类型	允许空	主键	外键	说明
id	int unsigned	否	是	-	评论ID, 自增主键
date	datetime	否	否	-	评论日期
article_id	int	否	否	article(id)	文章ID
content	longtext	否	否	-	评论内容
user_id	int	否	否	user(id)	评论用户ID
					父评论ID, NULL表示

parent_id	int	是	否	comment(id)	顶级评论
receiver_id	int	否	否	user(id)	接收者ID

点赞表 (love)

表4-10 点赞表

字段名	数据类型	允许空	主键	外键	说明
id	int unsigned	否	是	-	点赞ID, 自增主键
user_id	int	否	否	user(id)	点赞用户ID
article_id	int	否	否	article(id)	文章ID

收藏夹表 (collection)

表4-11 收藏夹表

字段名	数据类型	允许空	主键	外键	说明
id	int unsigned	否	是	-	收藏夹ID, 自增主键
user_id	int	否	否	user(id)	所属用户ID
name	varchar(255)	否	否	-	收藏夹名称

收藏记录表 (collect)

表4-12 收藏记录表

字段名	数据类型	允许空	主键	外键	说明
id	int unsigned	否	是	-	收藏记录ID, 自增主键
article_id	int	否	否	article(id)	文章ID
collection_id	int	否	否	collection(id)	收藏夹ID
date	datetime	否	否	-	收藏日期
user_id	int	否	否	user(id)	用户ID

粉丝关注表 (fans)

表4-13 粉丝关注表

字段名	数据类型	允许空	主键	外键	说明
id	int unsigned	否	是	-	关注关系ID, 自增主键
fans_id	int	否	否	user(id)	粉丝ID (关注者)
user_id	int	否	否	user(id)	用户ID (被关注者)

浏览记录表 (browse)

表4-14 浏览记录表

字段名	数据类型	允许空	主键	外键	说明
id	int unsigned	否	是	-	浏览记录ID, 自增主键
user_id	int	否	否	user(id)	浏览用户ID
article_id	int	否	否	article(id)	文章ID
date	datetime	否	否	-	浏览日期

私信表 (message)

表4-15 私信表

字段名	数据类型	允许空	主键	外键	说明
id	int unsigned	否	是	-	私信ID, 自增主键
sender_id	int	否	否	user(id)	发送者ID
receiver_id	int	否	否	user(id)	接收者ID
content	longtext	否	否	-	消息内容
date	datetime	否	否	-	发送日期

协作文章表 (cooperate_article)

表4-16 协作文章表

字段名	数据类型	允许空	主键	外键	说明
id	int	否	是	-	协作文章ID, 自增主键
title	varchar(255)	否	否	-	标题
content	longtext	否	否	-	正文内容
time	datetime	否	否	-	创建时间
version	varchar(255)	否	否	-	版本号
saved_by	int	否	否	user(id)	最后保存者ID
created_by	int	否	否	user(id)	创建者ID
status	varchar(255)	否	否	-	协作者是否活跃

轮播图表 (carousel)

表4-17 轮播图表

字段名	数据类型	允许空	主键	说明
id	int unsigned	否	是	轮播图ID, 自增主键
url	varchar(255)	否	否	图片URL

协作成员表 (cooperate_member)

表4-18 协作成员表

字段名	数据类型	允许空	主键	说明
id	int unsigned	否	是	记录ID, 自增主键
article_id	int	否	否	协作文章ID
user_id	int	否	否	用户ID
role	varchar(20)	否	否	owner/edit/view
joined_at	datetime	否	否	加入时间

约束说明:

外键: article_id → cooperate_article(id), user_id → user(id)

联合唯一约束: (article_id, user_id) —— 同一人在同一文档里只能有一条记录

索引: idx_cm_article(article_id), idx_cm_user(user_id)

协作邀请表 (cooperate_invitation)

表4-19 协作邀请表

字段名	数据类型	允许空	主键	说明
id	int unsigned	否	是	邀请ID, 自增主键
article_id	int	否	否	协作文章ID
inviter_id	int	否	否	邀请者ID
invitee_id	int	否	否	被邀请者ID
role	varchar(20)	否	否	edit/view
status	varchar(20)	否	否	pending/accepted/rejected/expired
created_at	datetime	否	否	创建时间
expires_at	datetime	否	否	过期时间

协作版本快照表 (cooperate_article_revision)

表4-20 协作版本快照表

字段名	数据类型	允许空	主键	说明
id	int	否	是	快照ID, 自增主键
article_id	int	否	否	协作文章ID
revision_no	int	否	否	版本号 (从1递增)
title	varchar(512)	是	否	保存时的标题
content	mediumtext	是	否	保存时的完整内容
editor_id	int	否	否	保存者用户ID
created_at	datetime	否	否	保存时间

约束说明:

外键: article_id → cooperate_article(id), editor_id → user(id)

联合唯一约束: (article_id, revision_no) —— 同一文档的版本号不能重复

索引: idx_car_article(article_id, created_at)

AI调用记录表 (ai_call_log)

表4-21 AI调用记录表

字段名	数据类型	允许空	主键	说明
id	bigint	否	是	记录ID, 自增主键
user_id	int	否	否	调用用户ID
func_type	varchar(30)	否	否	polish/translate/expand/correct /summary/chat
call_path	varchar(30)	否	否	backend_proxy/frontend_direct /frontend_ws
input_len	int	否	否	输入文本长度(字符)
output_len	int	是	否	输出文本长度(字符)
duration_ms	int	否	否	调用耗时(毫秒)
status_code	int	否	否	HTTP状态码/WS状态
cache_hit	tinyint	否	否	是否命中缓存 0/1
error_msg	varchar(500)	是	否	错误信息 (成功为NULL)
created_at	datetime	否	否	调用时间

约束说明:

外键: user_id → user(id)

联合索引: (user_id, created_at) —— 查某用户的调用历史

联合索引: (func_type, created_at) —— 按功能类型统计

索引: idx_call_path(call_path)

文章审核日志表 (article_audit_log)

表4-22 文章审核日志表

字段名	数据类型	允许空	主键	说明
id	int unsigned	否	是	日志ID, 自增主键
article_id	int	否	否	文章ID
admin_id	int	否	否	操作管理员ID
status_before	int	否	否	审核前状态
status_after	int	否	否	审核后状态
reason	varchar(500)	是	否	审核意见
created_at	datetime	否	否	操作时间

通知消息表 (notification)

表4-23 通知消息表

字段名	数据类型	允许空	主键	说明
id	int unsigned	否	是	通知ID, 自增主键
user_id	int	否	否	接收者用户ID
type	varchar(20)	否	否	like/collect/comment/follow/system
trigger_user_id	int	否	否	触发者用户ID
article_id	int	是	否	关联文章ID
comment_id	int	是	否	关联评论ID
content	varchar(500)	否	否	通知内容摘要
is_read	tinyint	否	否	是否已读 0/1
created_at	datetime	否	否	创建时间

约束说明:

外键: user_id → user(id), trigger_user_id → user(id)

联合索引: (user_id, is_read)

推荐结果缓存表 (recommend_result)

表4-24 推荐结果缓存表

字段名	数据类型	允许空	主键	说明
id	int unsigned	否	是	记录ID，自增主键
user_id	int	否	否	目标用户ID（唯一）
article_ids	json	否	否	推荐文章ID列表
algo_type	varchar(30)	否	否	item_cf/hot/cold_start
created_at	datetime	否	否	生成时间
expires_at	datetime	否	否	过期时间

第5章 模块设计

5.1 文章编辑模块

5.1.1 模块概述

博客系统中文章编辑模块的主体功能就是完成用户的原创、修改以及发布操作。本模块具有丰富功能的设计，有富文本编辑器、预设样式选择、标签分类体系搭建和智能化数据备份服务等，目的在于给内容创作者提供高效便捷的创作支持。

主要职责：

23提供富文本编辑环境

24支持文章元数据管理（标题、标签、分类等）

25实现自动保存和草稿管理

26支持文章发布流程

文章编辑模块时序图：

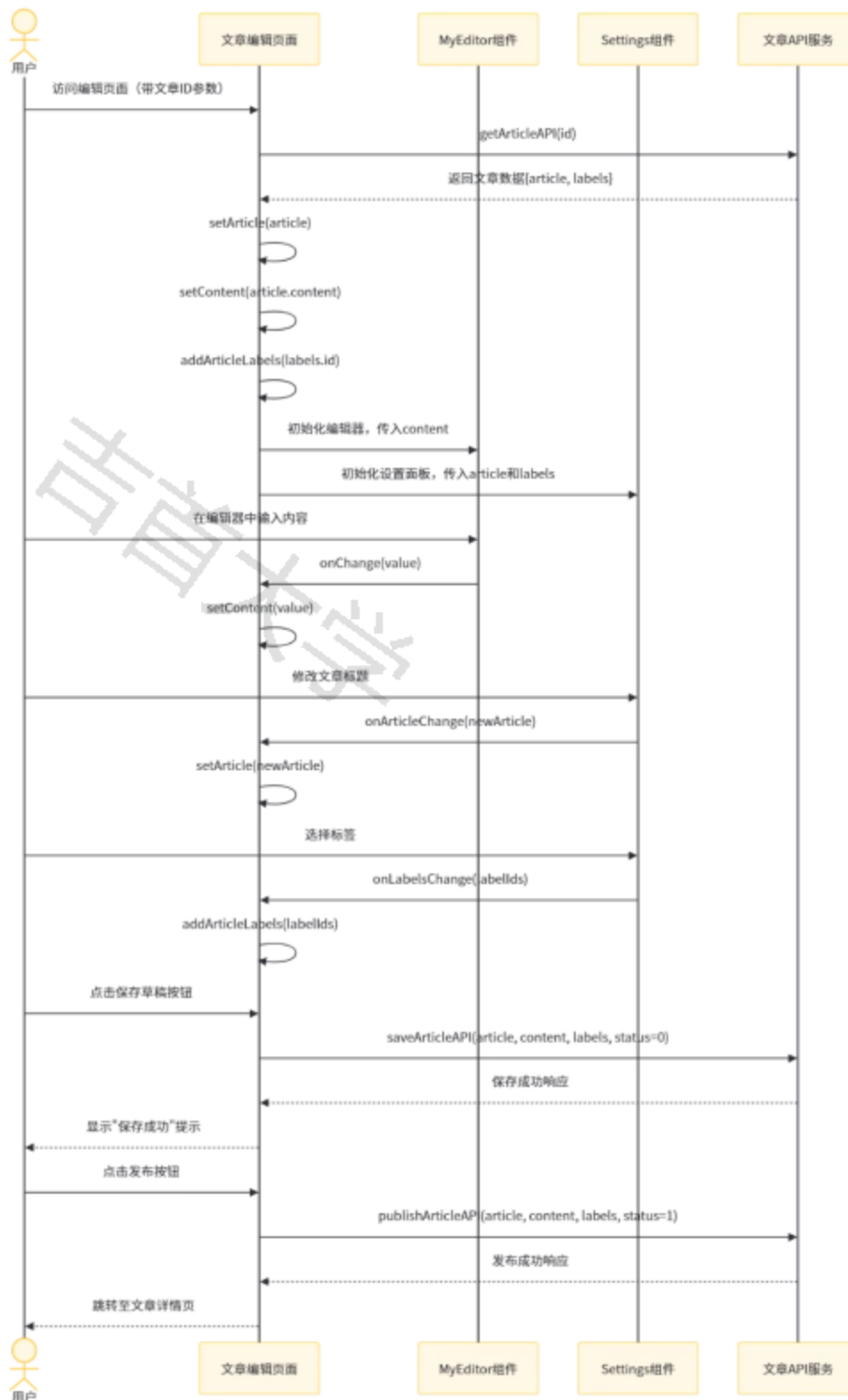


图5-1 文章编辑时序图

图5-1为文章编辑模块开发时序分析架构图，严格按照统一建模语言(UML)标准规范来搭建。本系统主要的交互主体有三个，用户为主要的操作者，前端用React框架搭建起来的可视化编辑界面，后端用Spring Boot技术栈实现的服务支持层。具体的交互路径如下，用户通过前端界面发起文章数据查询请求，前端调用getArticleAPI获取初始内容，对于标题或者正文内容的修改，使用Quill富文本组件实现动态更新，把每一次的修改记录保存到状态变量

content中，当用户点击“保存”或者“发布”按钮的时候，就会调用不同的业务接口来完成操作，在draft模式下，状态值设为0，在publish选项下，状态值设为1并且触发审核流程。该设计把从草稿的产生到最后上线的所有环节都囊括在内，并且详细地列举出主要的功能特性。

5.1.2 页面设计与实现

页面：



图5-2 文章编辑页面

图5-2给出了本节对文章编辑页面的解析。该界面是由三部分组成的，上端为输入或者修改文章主题的标题栏，中部为主编辑区，使用WangEditor组件进行多格式文本的编辑以及多媒体资源的无缝集成和管理，下部是操作区，有两个主要的功能按钮，即保存草稿、发布内容。此布局很好地体现了系统对信息的录入、格式化以及状态转换等各方面的专业性、协同性，从数据的采集开始，一直到最后发布内容完成的过程都是完善的。

主要交互流程：

表5-1 主要交互流程

操作	触发条件	结果
加载文章	路由参数包含id	调用getArticleAPI获取已有文章
编辑标题	用户在标题框输入	实时更新article状态
编辑内容	Quill编辑器内容变化	更新content状态
保存草稿	点击保存按钮	调用saveArticleAPI，status=0
发布文章	点击发布按钮	调用publishArticleAPI，status=1

5.1.3 类设计与实现

主要数据结构

// Article 类型定义

```

interface Article {
  id: number | null; // 文章ID
  userId: number | null; // Author ID
  date: Date | null; // Publication Date
  title: string | null; // 文章标题
  content: string | null; // 文章内容HTML
  url: string | null; // Article URL
  status: number | null; // 状态(0=草稿, 1=待审核, 2=已发布, 3=审核中)
  categoryId: number | null; // Category ID
  columnId: number | null; // Column ID
  kind: string | null; // 文章类别('原创'/'转载'/'翻译')
  brief: string | null; // 文章简介
}

```

// 组件内部状态

```

Interface EditState {}
article: Article; // 文章对象
content: string; // 编辑器内容
articleLabels: number[]; // 关联标签ID数组
reloadFlag: number; // 重加载标志
}

```

单一职责类设计: Editor 组件 (MyEditor)

职责: 管理Quill编辑器实例

接口:

```

interface MyEditorProps {
  content: string;
  onChange: (value: string) => void;
}

```

主要方法: 初始化编辑器、处理内容变化

职责: 管理文章元数据和标签

接口:

```

Interface SettingsProps {
  article: Article
  articleLabels: number[]
  onArticleChange: (article: Article) => {};
}

```

```
onLabelsChange: (labels: number[]) => void;
}
```

类图设计：

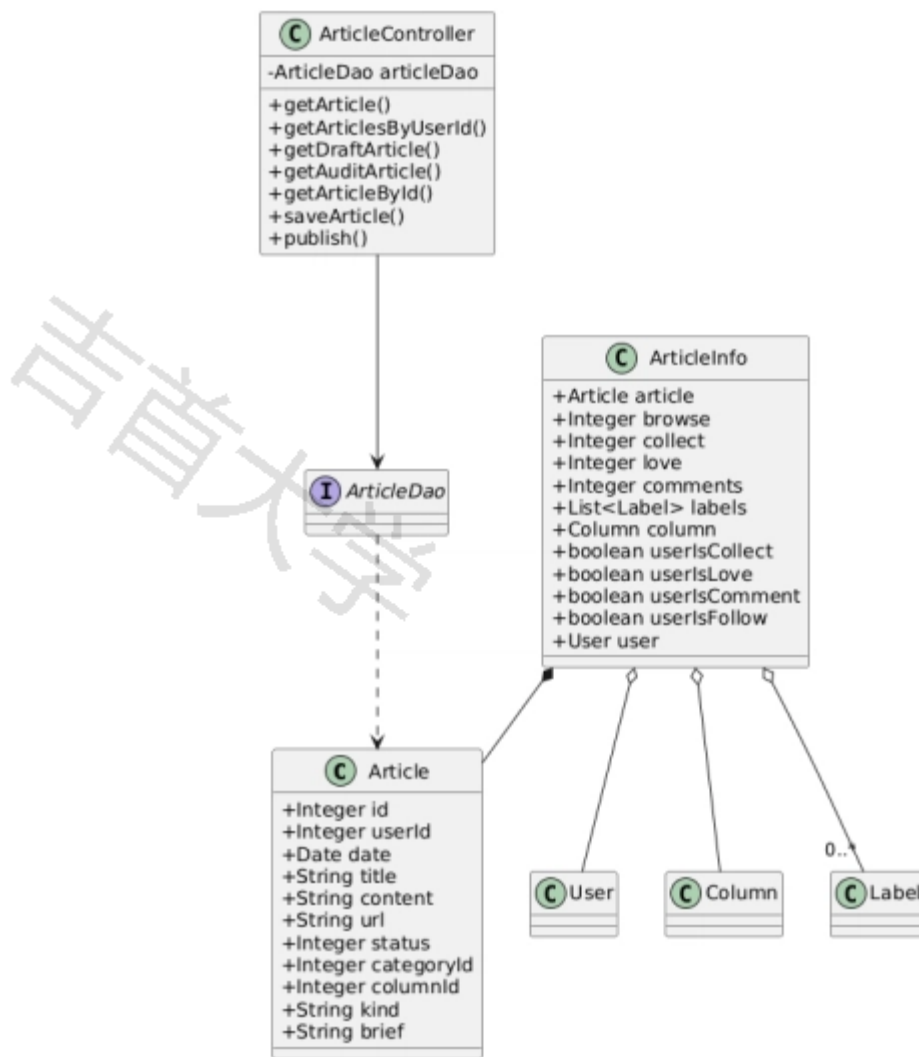


图5-3 文章编辑类图设计

本图清楚地表现出写作和发布的模块中核心实体Article以及它的前端展示辅助对象ArticleInfo。后者是文章的内容、作者的资料、专栏的分类、标签体系和统计指标等各个方面的数据属性。系统后台调用ArticleController的ArticleDao保存草稿，审核稿件，按状态筛选用户等具体的实现。

5.1.4 核心功能实现

实现代码流程：

```
// 在useEffect中执行
useEffect(() => {
  const init = async () => {
    if (param?.id) {
```

```

    const res = await getArticleAPI(parseInt(param.id));
    if (res.data.code === 200) {
    const { article, labels } = res.data.data;
    setArticle(article);
    setContent(article.content);

    addArticleLabels(labels.map((item: any) => item.id));
    }
    }
};
init();
}, [reloadFlag, param.id]);

```

5.2 协作编辑模块

5.2.1 模块概述

使用Yjs CRDT框架加上WebSocket技术，可以实现实时同步编辑多人共享文档的功能。系统主要是利用冲突检测和协调机制、编辑者光标位置的精确定位来保证数据的一致性、完整性。

主要特性：

27实时多人编辑

28自动冲突解决（CRDT算法）

29光标位置同步

30权限管理（编辑/查看权限）

31版本控制和自动保存

协同编辑模块时序图：

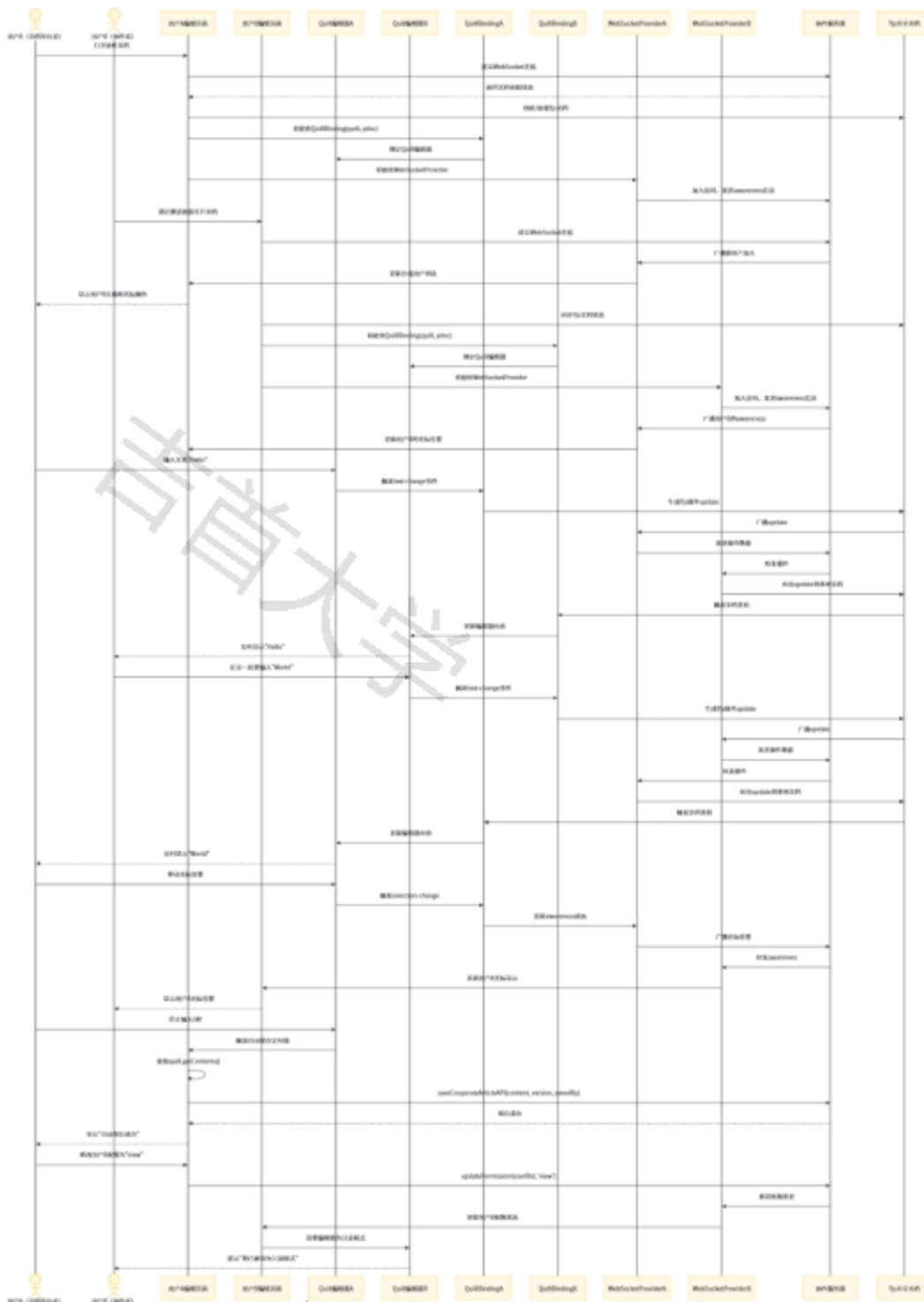


图5-4 协作编辑模块时序图

图5-4为按照UML建模标准绘制出来的协作编辑模块主要交互流程时序图。此模型有四个主要的交互对象，分别是用户A、用户B、协同服务、Node.js/WebSocket架构、Yjs库的文档管理系统等等。具体的交互过程如下——用户A打开文档文件，创建Websocket通信连接，接收协同服务返回的Y.Doc对象实例；协同服务给用户B提供相同的文档访问权限，启动实时同步；双方都按照CRDT算法把各自的输入内容事件化，并用Websocket协议来完成状态更新，用户A触发文本插入时，系统会产生相应的事件并推送到其他的在线成员上，所有的客户端用CRDT算法解析远程的变更并同步到本地的数据结构上，根据Awareness协议，系统会实时地保存鼠标光标的坐标和选中的范围这些元信息，并且

把它们传送给其他客户端，保证数据可靠，系统会定时地把当前版本的文档内容存入数据库里。本文从各方面对多用户并发编辑关键技术和应用进行了系统的分析。

5.2.2 界面设计与实现

协作编辑页面

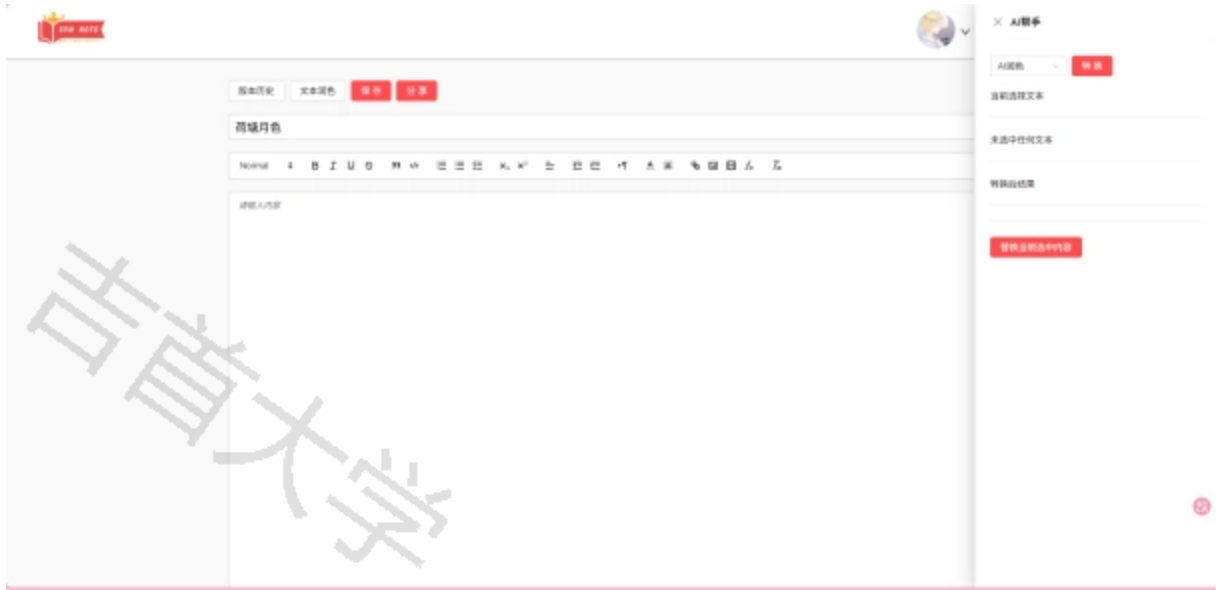


图5-5 协作编辑页面设计

图5-5为本节给出的协作编辑界面设计。该界面是用界面截图和原型图相结合的方式表现出来的，大致上由这四个大的部分组成，分别是顶部工具栏（文档标题、在线用户头像列表、分享按钮等），左侧主编辑区（用Quill框架创建的富文本编辑器可以实时展示内容），右侧辅助模块（包括在线用户名单、光标颜色标记、聊天窗口等），底部状态栏（显示同步进度、最近更新时间等）。本设计系统协同编辑，用多色光标区分不同的用户，并且配有内部通信区来提高团队成员之间互动的速度。

5.2.3 类设计与实现

核心数据结构

// 协作文章类型

```
Interface Cooperative Article
id: number; // 文档ID
time: string; // Creation Time
title: string; // 文档标题
content: string; // Document content
savedBy: number; // 最后保存者ID
version: string; // Version number
role: 'owner' | 'edit' | 'comment' | 'view'; // User Permissions
}
```

// 在线用户状态

```
interface OnlineUser {  
  id: number;  
  name: string;  
  avatar: string;  
  cursor: {  
    line: number;  
    column: number;  
  };  
  color: string;  
  role: string;  
}
```

主要模块组件: QuillBinding 绑定层

职责: 连接Quill编辑器与Yjs文档

核心属性:

32quill: Quill编辑器实例

33ydoc: Yjs共享文档

34ytext: Y.Text类型的共享内容

35WebsocketProvider 通信层

职责: 管理WebSocket连接与数据同步

核心功能:

```
Interface WebsocketProvider {  
  connect(): void; // Connect  
  disconnect(): void; // Disconnect  
  awareness: Awareness; // 用户感知信息  
}
```

```
Interface WebsocketProvider {  
  connect(): void; // Connect  
  disconnect(): void; // Disconnect  
  awareness: Awareness; // 用户感知信息  
}
```

Awareness 用户感知层

职责: 追踪在线用户和光标位置

状态信息:


```

{
user: { name, color, id }
cursor: { anchor, head }
}

```

类图设计:

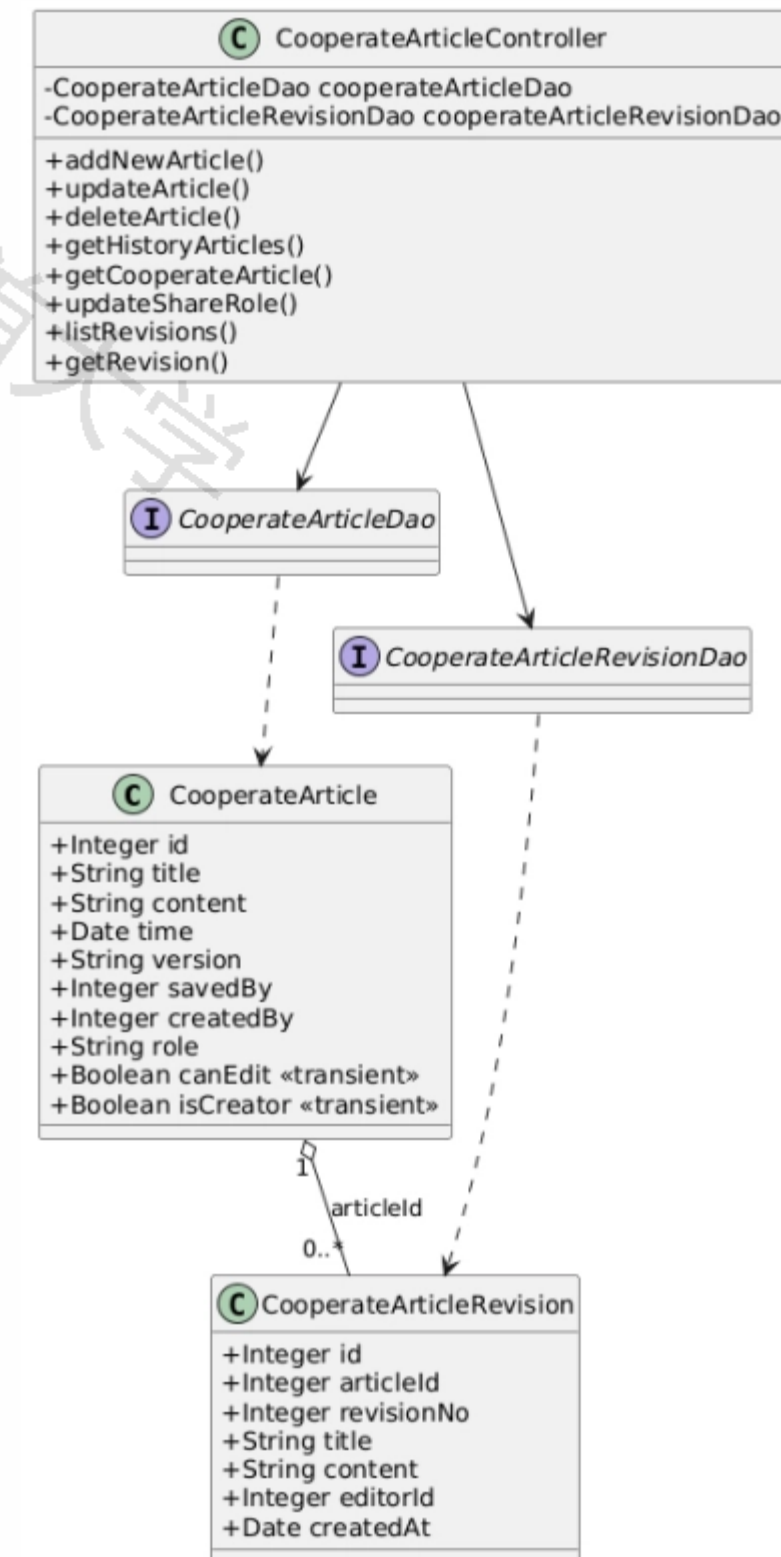


图5-6 协作编辑类图设计

图5-6该类图展示协作文档的核心数据模型 `CooperateArticle` 与版本快照 `CooperateArticleRevision` 的聚合关系（1 对多），以及后端通过 `CooperateArticleController` 调用 `CooperateArticleDao` / `CooperateArticleRevisionDao` 完成文档创建、保存、分享权限设置与版本查询的整体结构。

5.2.4 冲突处理机制与实例

本系统使用的是Yjs库中基于YATA算法实现的最后一致性。与传统的依靠中心化服务器、使用乐观锁或者时间戳顺序（OT）的并发操作不同，分布式可扩展的数据类型（CRDTs）依靠设计好的数据结构可以自主产生、有效地解决并发冲突问题。通过两个典型案例来证明。

例子一：两个人同时在一个位置插入文字

初始状态：文档内容是“AC”

同时发生的操作：

- 用户A在A后面插入B
- 用户B也在A后面插入D

冲突在哪：传统基于索引的方式会出现 `insert(1, 'B')` 和 `insert(1, 'D')` 的冲突，没法确定谁先谁后。

Yjs框架中CRDT（冲突检测与解决）的实现机制可以从以下几个方面进行解析，即全局唯一逻辑ID设计原则、全局唯一逻辑ID获取方法、全局唯一逻辑ID的更新操作、客户端唯一逻辑ID获取方式、服务端唯一逻辑ID获取方式等。其设计按照一定的规则，在冲突情况下用字典序规则来确定元素插入的顺序。客户端A的`clientId`小于客户端B的时候，合并得到的字符串序列可能会呈现成“A B D C”或者“B A D C”这样形式，都满足事先设定好的排序准则，利用这种办法可以达成多个客户端之间状态的同步，不需要再行安排协调。****YATA算法的应用分析****：以某典型操作示例为例——用户A执行`insert('B', left='A', right='C')`，而用户B则执行相同形式的操作——系统运用YATA算法对各事件进行有序化处理，并基于事件ID（如01.ID < 02.ID）实现所有副本的一致性聚合。虽然最终的结果是ABCD或者ADBC，但是该方法极大地提高了多客户端并发操作的可靠性，大大减小了产生冲突的风险。上述架构可以大大提高分布式合作环境下用户满意度和一致性的水平。

例子二：同时发生删除和插入（墓碑机制）

初始状态：文档内容是“Hello World”

同时发生的操作：

- 用户A删掉“World”
- 用户B把“World”替换成“Universe”（先删“World”，再插入“Universe”）

核心问题就是，用户A执行数据删除的时候，用户B正在进行数据插入，此时用传统的物理删除方法，会造成用户B后续插入的数据产生位置冲突、顺序乱套、丢失等状况。

由于CRDT（可复制分布式数据类型）在处理并发删除的时候会遇到一些问题，Yjs提出了墓碑这个概念。该机制用标记而不是实际的物理移除来应对删除指令，在目标实体上加上“已删除”状态标识来记录它的历史信息。已删除标记的存在使得系统在多人共同修改的时候也能够对内存的分配与回收有明显的看出。以世界(World)为更新目标

的时候，“World”的原生结构得到保留，“Universe”是用完整的过去时态信息来覆盖的。世界消失只影响当前的展现视角，不会破坏全局的一致性。两种操作模式互相独立又互相配合，在分布式环境当中改善了数据的同步以及冲突解决情况。

吉首大学



图5-7 协同编辑算法流程图

图5-7给出了该算法的全部实现步骤。该种冲突检测与处理机制依靠的是Yjs库的底层自动化运行方式，利用y-quill组件来实现事件绑定工作，有效地缩减了开发者需要手工书写繁杂冲突处理逻辑的任务。

5.2.5 核心功能实现

代码：

```
// 自动保存监听
useEffect(() => {
  if (!quill) return;
  let saveTimer: NodeJS.Timeout;
  const handleChange = () => {
    // 清除之前的定时器
    clearTimeout(saveTimer);
    // 设置新的保存定时器
    saveTimer = setTimeout(async () => {
      const content = quill.getContents();
      const delta = quill.getContents();
      await saveCooperateArticleAPI({
        id: docId,
        Content: JSON.stringify(delta),
        version: dayjs().format('YYYY/MM/DD HH:mm:ss'),
        savedBy: user.id,
        role: cooperateArticle.role
      });
    }, 5000); // 5秒延迟保存
  };
  quill.on('text-change', handleChange);
  return () => {
    clearTimeout(saveTimer);
    quill.off('text-change', handleChange);
  };
}, [quill, docId, user.id]);
```

5.2.6 Yjs文档状态持久化与恢复机制

根据协同编辑的架构设计思想，文档的状态信息保存在内存里的Y.Doc对象里。为应对服务重启、网络中断等不确定因素，并保证数据一致性、可靠性的实现，系统使用如下方式来对状态进行持久化存储以及后续的恢复操作

1. 定时自动保存（文档快照持久化）：

// 客户端自动保存逻辑（位于src/pages/yjs/components/quill/index.tsx）

在此系统架构下，ID值最小并且处于在线状态的用户就会得到执行自动备份任务的权力。

```
const electSaver = () => {
  const onlineUserIds = [...awareness.getStates().values()]
    .map(s => s.user?.id)
    .filter(Boolean)
    .sort((a, b) => a - b);
  return onlineUserIds[0]; // 最小ID用户为saver
};

// 仅当选的saver执行周期性保存
useEffect(() => {
  const interval = setInterval(async () => {
    const currentSaverId = electSaver();
    if (currentSaverId !== user.id) return; // Not the saver
    const ytext = ydoc.getText("quill");
    const content = ytext.toString(); // Y.Text → pure text
    await saveCooperateArticleAPI({
      id: docId,
      content: content,
      version: dayjs().format("YYYY/MM/DD HH:mm:ss"),
      savedBy: user.id,
    });
  }, 30000); // 每30秒检查一次
  return () => clearInterval(interval);
}, [ydoc, docId, user.id]);
```

2. Yjs服务端状态恢复：

// 服务端（Node.js y-websocket）启动时从数据库加载文档状态

// 伪代码描述y-websocket服务端持久化逻辑

```
const persistence = {
  // 从MySQL加载文档初始状态
  async bindState(docName, ydoc) {
    const row = await db.query(
      SELECT content FROM cooperate_article WHERE id = ? , docName
```

```

);
if (row) {
// 将数据库中的内容恢复到Y.Doc
const ytext = ydoc.getText("quill");
ytext.insert(0, row.content);
}
// 监听Y.Doc更新，定期持久化
ydoc.on("update", debounce(async (update) => {
const content = ydoc.getText("quill").toString();
await db.query(
    "UPDATE cooperate_article SET content = ?" WHERE id = ?
    [content, docName]
);
}, 2000)); // 2秒防抖，避免频繁写入
},
// 写入版本快照
async writeState(docName, ydoc) {
const content = ydoc.getText("quill").toString();
// 获取当前最大revision_no并+1
const [row] = await db.query(
    SELECT MAX(revision_no) AS max_rev FROM cooperate_article_revision WHERE article_id = ?
    docName
);
const nextRev = (row.max_rev || 0) + 1;
await db.query(
    INSERT INTO cooperate_article_revision
        (article_id, revision_no, title, content, editor_id, created_at)
    VALUES (?, ?, ?, ?, ?, NOW())
    [docName, nextRev, null, content, null]
);
},
};

```

5.2.7 协作成员权限校验

用户协作编辑的时候，系统按照在校验cooperate_member表的角色字段上的权限信息来创建起一种细致化的权

限控制手段。该机制的服务端、前端分别做一层逻辑处理任务。

服务端对于WebSocket连接早期的总体逻辑框架可以概括为如下内容。

获得认证凭证之后，解析用户标识，从 `cooperate_member` 数据库表中找出对应的权限信息。如果用户的权限标签是view，那么应该把有关联的资源配置成只读模式；如果之后出现编辑操作申请，马上拒绝并且终止该操作的执行。

前端这块根据role来控制界面：

owner或edit：编辑器可编辑，工具栏全亮

把编辑界面改为不能被修改的状态，在页面的顶部加上一句“当前处于只读模式”的提示信息，告知用户其目前所处的操作权限。

当用户没有相应的权限时，不能访问目标页面，用无权查看该文档的信息来告诉用户。

5.2.8 版本快照保存与回滚

快照保存有三个触发场景：

- (1) 用户手动点“保存”按钮，立刻存一条快照
- (2) Saver每30秒自动检查并保存，同样写快照
- (3) 快照超过50条时自动删掉最旧的那条

版本回滚的流程：

用户选择目标版本之后，触发了恢复到该版本的功能之后，前端系统就会弹出确认对话框来避免出现误操作的风险。如果用户选择了确定，则后面会依次做如下操作，即根据目前文档的状态生成新的快照记录作为回滚的依据，同步更新`cooperate_article.content`字段到指定版本内容，采用WebSocket做全量通讯，在所有的合作人员中发送文档变更通知。接收服务器收到数据之后，前端立马调用编辑器组件的重载方法来更新界面内容呈现的效果。

5.3 AI辅助模块

5.3.1 模块概述

该模块主要提高博客的创作和协作编辑效率，它的主要功能有对话式交互助手和文本智能化优化，即润色、翻译、内容扩充、校对和摘要生成。从技术架构视角分析，在前端 `myBlog` 模块中，通过本地部署的 AI 服务接口（例如 `http://localhost:5000`），借助 SSE（Server-Sent Events）事件流机制实现多维功能整合。经过处理的结果将立即传送到 Quill富文本编辑器上，给用户一个直观的操作界面。

本研究所建立的智能对话系统采用悬浮窗口的形式嵌入到用户的界面当中，使用WebSocket技术与大量的预训练语言模型进行高效率的协同交互。其主要的运作机制有两个主要部分，第一部分就是利用GET请求得到WebSocket服务器的基础URL，第二部分是根据得到的基础URL建立一个实时通信连接，再将接收到的数据按照分块的形式依次进行处理。上述功能模块的主要职责在于由`myBlog/src/pages/chat/index.tsx`中的`AIChatService`承担，具体涵盖会话状态管理及消息转发等任务逻辑的设计与实现；而针对WebSocket URL的动态生成，则交由位于`myBlog/src/apis/ws.ts`的专用代码模块完成。

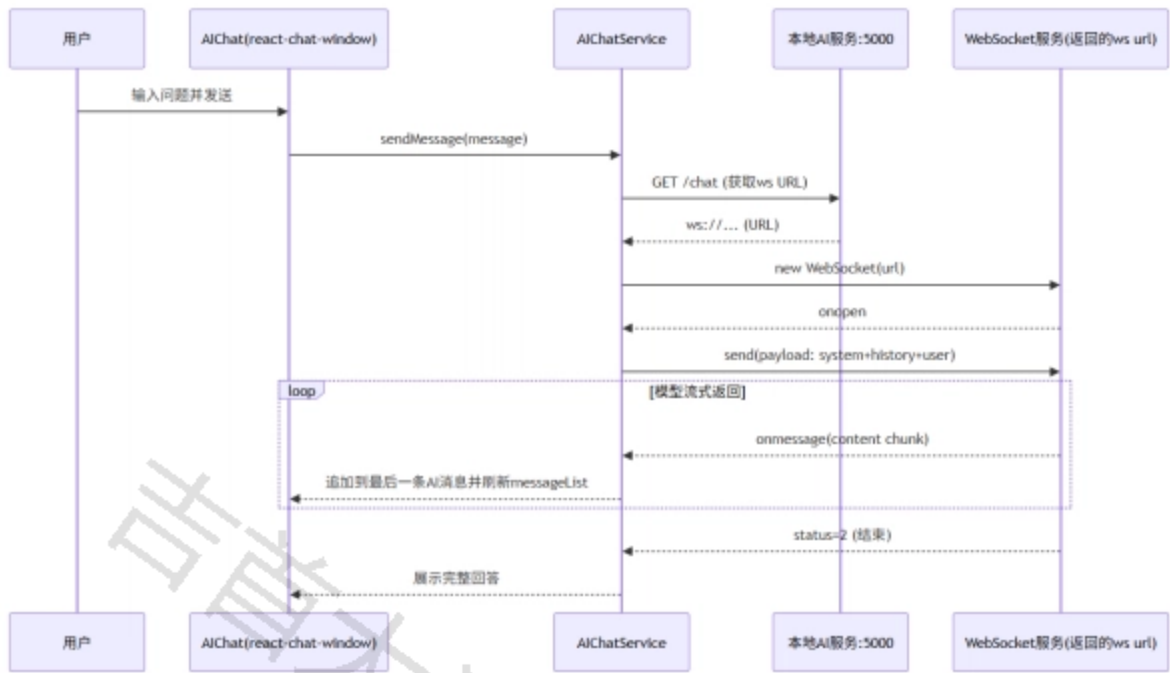


图5-8 AI对话助手时序图

这个流程图画的是AI聊天助手的完整流程：

用户登录之后，系统通过调用AIChat组件、建立WebSocket连接来完成初始化的功能。用户发送的信息被系统暂存到队列当中，同时加上一个占位符标识。当WebSocket连接没有建立的时候，就会立刻开始连接的创建过程；当WebSocket连接成功之后，系统就会得到包含系统默认提示语、历史对话内容以及新输入文本的一份完整的请求报文，再将其提交给服务端进行处理。当收到服务器的响应结果之后，对文本数据进行逐段解析，更新AI生成的回复记录，实时同步到界面中以达到对话结束或者出现异常中断的时候为止。

文本智能处理

统一入口是convert(type, text, setContent)，根据类型走不同的通道

润色（改写）：POST /text/rewrite

翻译：POST /text/translate

纠错：POST /text/correction

利用SSE(服务器到客户端)服务同第三方平台开展数据交流，以流式传送手段接收分片化数据。系统在此时还要对拼接好的数据进行实时处理，并且要实现对界面的动态更新，从而保证信息的即时性以及互动性的好转。

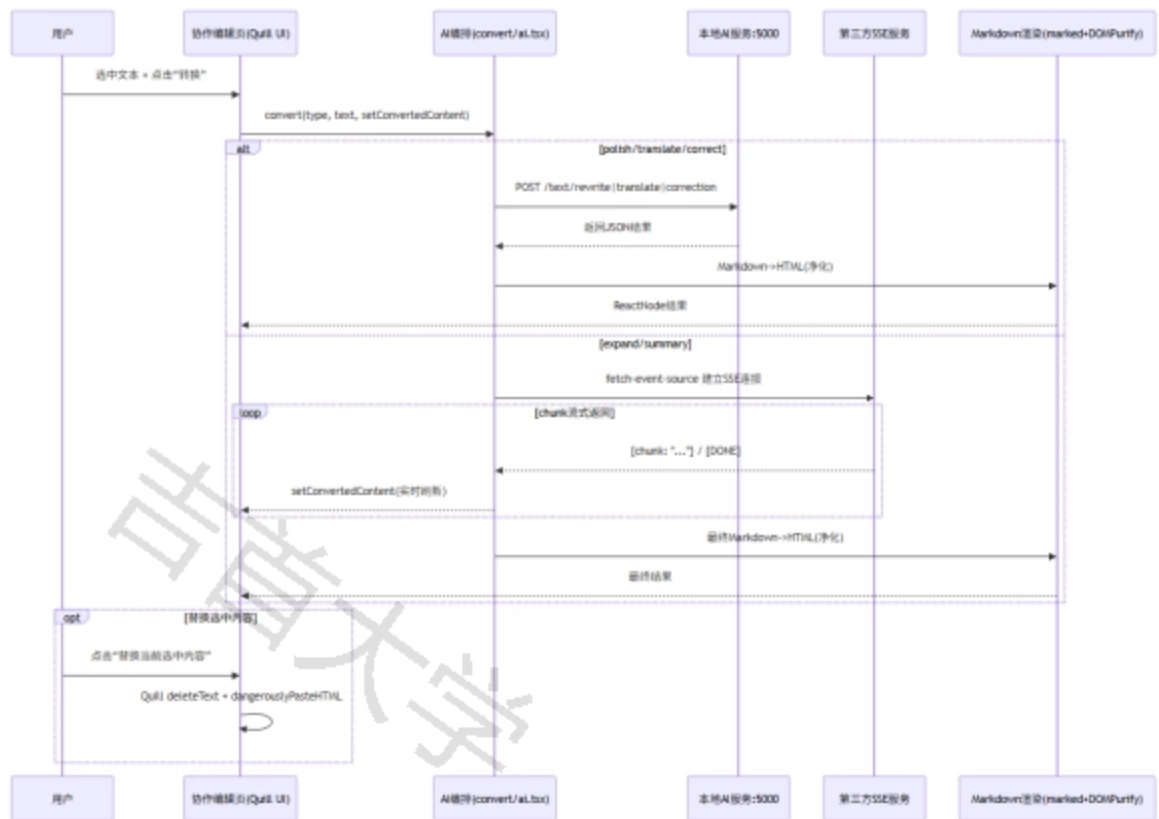


图5-9 AI文本智能处理时序图

这个时序图画的是AI辅助功能的调用流程：

用户选择某篇特定的文本之后，前端系统按功能类别把任务分成了润色、翻译、校正、扩写和摘要等种类，并且调用本地部署好的AI辅助服务的HTTP接口完成数据的处理工作，而对于复杂的扩写、摘要生成这些要求，使用Server-Sent Events（SSE）技术来创建一个长连接，然后依次接收由服务器传来的数据流。经过对结果的Markdown格式进行解析，并将解析结果用界面上的动态方式展示出来，让用户可以查看结果。如果用户输入正确，系统将会把数据库里指定的目的段落自动生成修改的内容。

5.3.2 界面设计与实现

本项目中 AI 辅助能力主要体现在两处 UI：

首页、写作页面都有全局悬浮交互助手AIChat的部署，在“Home”、“Edit”模块中呈现并且实现了对应的功能。



图5-10 AI小助手页面设计图

图5-10给出了AI虚拟助手的悬浮交互界面，用户可以使用下边的表情符号和文本输入框来完成信息的编写，在点击“发送”按钮之后就会展现出逐渐出现的内容。



图5-11 AI辅助页面设计图（含流式输出）

图5-11为“协作编辑”界面集成的“AI辅助”模块，给用户提供智能化文本处理服务。用户可以对想要优化的内容、翻译、扩展、校验、摘要生成等各种操作进行操作，并且可以使用“一键替换”功能来即时更新到当前编辑

的状态上。人工智能辅助模块含有润色、多语种翻译、内容延展、文档审核等多种功能，只需要对目标文件进行勾选之后点击转换就可得到该系统的成果。利用AI辅助支持可以把 Markdown文档转化为HTML、也可以实现富文本编辑器的自动显示与管理。

5.3.3 类设计与实现

核心数据结构

聊天消息类型（AI 对话）

// AI聊天消息类型

```
Interface Message {  
  type: "text"; // Message Type  
  author: "me" | "ai"; // Sender (User/AI)  
  data: {  
    text: string; // 消息内容（AI为流式拼接）  
  };  
}
```

文本转换类型（AI 文本工具）

// AI文本能力类型

```
type ConvertType = "polish" | "translate" | "expand" | "correct" | "summary";
```

SSE 流式事件结构

// SSE流式返回分片

```
interface SSEChunkEvent {  
  chunk?: string;  
  type?: "done";  
}
```

AIChatService 业务层

本模块的主要功能就是创建智能语音交互的核心结构，即WebSocket通信的即时保持，消息队列的及时分配，多轮对话的状态整合，连续文本回答的拼装。通过调用回调函数来对用户界面进行动态的改变并且保证交互的流畅性。

```
class AIChatService {  
  ws: WebSocket | null;  
  msgQueue: Message[];  
  connect: boolean;  
  retryCnt: number;  
  maxRetry: number;  
  appId: string;
```

```

domain: string;
onUpdate: (messages: Message[]) => void;
onConnChange: (connected: boolean) => void;
}

```

主要模块组件：ConvertService（文本能力编排层）

职责：对外提供统一入口 convert(type, text, setContent)，把“润色/翻译/纠错（HTTP一次性）”与“扩写/总结（SSE流式）”封装为同一种调用方式，最终输出可直接渲染的 ReactNode。

核心属性：

```

type: ConvertType: 能力选择 (polish/translate/expand/correct/summary)
text: string: 输入文本（来自编辑器选区/用户输入）
setContent(jsx): 流式场景实时刷新结果（扩写/总结用）

```

核心功能：

```

interface ConvertService {
  convert
  type: ConvertType,
  text: string,
  setContent?: (jsx: ReactNode) => void
): Promise<ReactNode> | ReactNode;
}

```

SSEService 通信层（扩写/总结）

核心功能就是创建SSE(Server Sent Events)通信通道，不断监视分片事件传输情况。对接收到的事件数据做解析、转发之后，就可对分片数据进行拼接处理，并且可以实现前端界面的动态更新任务。

核心功能：

```

Interface SSEService {
  connect
  url: string, // text-expander / text-summarizer
  method: "GET" | "POST",
  Body: Object.
  onMessageHandler: (event: any) => void,
  myHeader?: Record<string, string>,
  onHandleClose?: () => void
): Promise<void>;
}

```

本节主要对AI辅助设计数据特点进行分析，从而避免重复的说明。

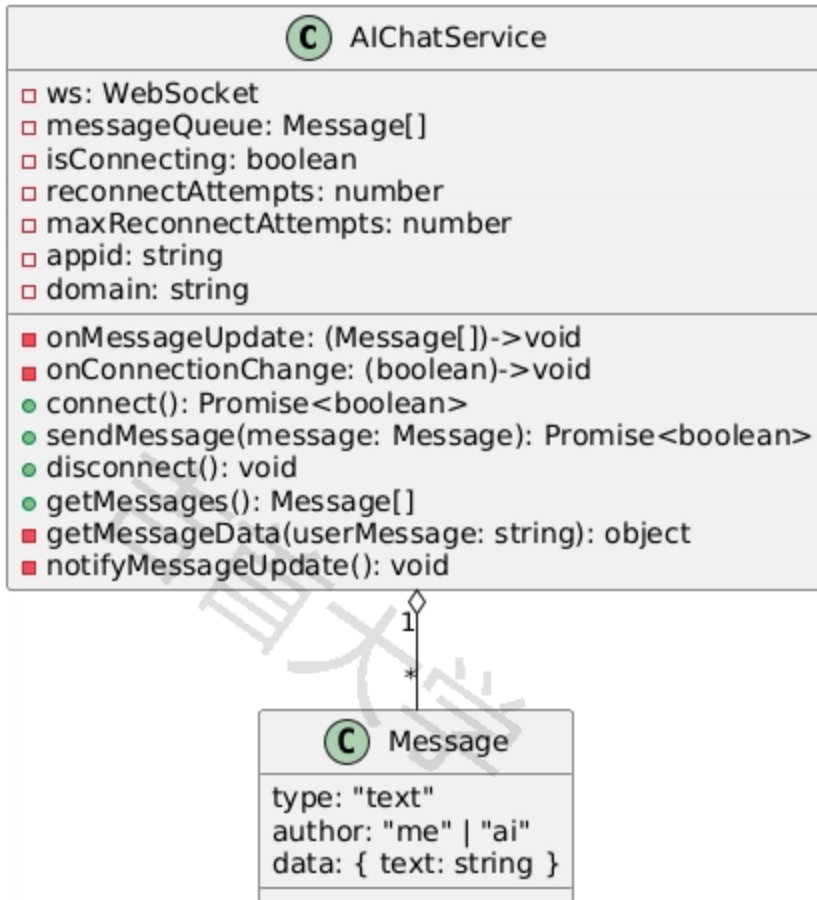


图5-12 AI对话类图设计

图5-12为AIChatService，它是系统的核心模块，主要完成业务逻辑层的工作。它依靠WebSocket连接以及消息队列(Message对象)的统一管理机制来达成客户端之间的高效双向通信，而且给用户交互界面赋予了连接创建，命令发出，通道结束以及历史记载展示等一系列的服务接口。数据传输时使用特殊的封装方式，把历史对话内容加上system标签变成payload格式的消息载体，而接收端则是按照逐条解析的方式将流式分片的数据流逐条提取出来，然后更新UI界面最新的AI回复。

5.3.4 核心功能实现

核心代码：

AI对话：

```

async connect(): Promise<boolean> {
    if (this.ws?.readyState === WebSocket.OPEN) return true;
    If this.isConnecting, then return false.
    this.isConnected = true;
    const res = await getWsUrlAPI();
    const url = res.data;
    return new Promise((resolve) => {
        this.ws = new WebSocket(url);
    });
}
  
```



```

this.ws.onopen = () => {
this.isConnected = false;
this.reconnectAttempts = 0;
this.onConnectionChange?.(true);
resolve(true);
};

this.ws.onmessage = (event) => {
const data = JSON.parse(event.data);
if (data.payload?.choices?.text?.[0]?.content) {
const chunk = data.payload.choices.text[0].content;
const i = this.messageQueue.length - 1;
if (i >= 0 && this.messageQueue[i].author === "ai") this.messageQueue[i].data.text += chunk;
else this.messageQueue.push({ type: "text", author: "ai", data: { text: chunk } });
this.notifyMessageUpdate();
}
};
});
}

```

AI辅助工具:

```

export const convert = (
type: string,
text: string,
setContent: (jsx: ReactNode) => void = () => {},
) => {
switch (type) {
case "polish":
return polishText(text);
case "translate":
return translateText(text);
case "expand":
return expandText(text, setContent);
case "correct":
return correctText(text);
case "summary":

```

```
return summaryText(text, setContent);
}
};
```

5.3.5 AI调用日志记录

三条AI路径都往ai_call_log表里写日志。

对于后端代理性能的检测机制为采用Python Flask框架搭建的后端服务，在请求到达以及响应回送之前都记录下时间戳，根据这些时间戳计算出整个处理过程所花的时间。另外把请求参数长度、响应数据大小和HTTP状态码等主要指标加入到日志中去；当发生异常的时候，用“error_msg”保存详细的错误信息以及具体的表现形式。

前端与Server-Sent Events(SSE)交互方式优化策略，客户端首次建立连接获取第一条消息时要主动向服务端发起请求来检验会话状况，在流式数据传输结束或者出现连接中断的时候需要发出事件通知并将总数据量、处理时间等主要指标作为回馈给服务端的信息。

AI小助手WebSocket：在onopen时发开始埋点，onclose时发结束埋点。

5.3.6 SSE流式中断与重试处理

扩写和总结的前端SSE连接加了下面几个容错：

网络中断的时候，系统需要保证已经保存的数据是完整的、一致的。界面要以“任务暂停，已经生成了X字内容，请重新启动”为提示信息，给用户做出操作指引，并且保证用户后续的操作权利。

非200响应给明确的错误提示

用户中断后可以重新点按钮触发，重新生成

关键代码大致是这样：

用fetch拿到response.body.getReader()，然后while(true)循环读chunk，每次读到就解析SSE格式，实时拼接到结果区域。catch到错误时把已生成的内容保留下来，不直接丢弃。

5.4 消息系统模块

5.4.1 模块概述

该模块主要是用户交互、信息推送两方面内容的实现模块，也就是私信交流、评论互动、点赞提醒等业务场景的实现模块。利用WebSocket通信方式可以实现即时的消息传输以及两向同步更新，还能对消息进行分类整理、未读标记设置以及历史记录查询等服务功能。

核心功能：

36私信管理（发送、接收、对话列表）

37消息通知（点赞、评论、粉丝提醒）

38实时推送（WebSocket）

39消息分类展示

40未读标记管理

消息模块时序图：



使用WebSocket进行双向实时通信，用户B接收到消息后显示出来，然后双方继续多轮对话交互，最后系统将已读的标记保存到数据库当中。在整个过程中用户可以调用getMessagesAPI接口得到完整的会话历史记录，根据需要执行删除操作。依靠WebSocket所具有的高效异步通信特点以及HTTP协议来实现静态资源的配置和状态管理的目的，进而创建出一个高度协调的信息传递体系。

5.4.2 界面设计与实现

消息系统页面：

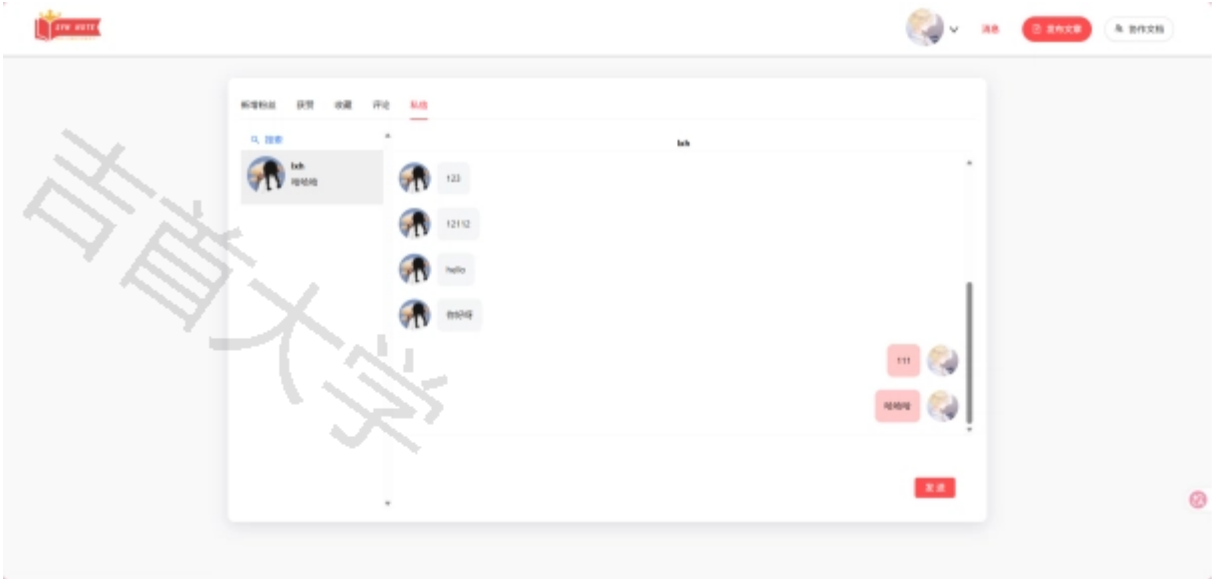


图5-14 消息系统页面

图5-14为本研究所用消息系统界面设计原型图，包括三大模块。左侧区域显示消息分类导航，包含私信、点赞通知、评论提示、粉丝互动等等内容，中部是消息列表区，用对话摘要、未读数量、最后更新时间这些指标来体现；右侧集中在一起的是完整的信息展示、文本输入框以及按钮这些主要的部件。此布局清楚地显示出系统对于信息的分类处理以及双向实时通讯所具有的明显优点。

消息分类视图：

表5-2 消息分类视图

消息类型	来源	显示内容	操作
私信 (Message)	用户1 → 用户2	对话气泡	删除、标记已读
点赞 (Like)	文章/评论	“XXX赞了你的文章”	返回原文、删除
评论 (Comment)	评论用户	“XXX评论了你的内容：xxx”	查看、回复
粉丝 (Fan)	关注用户	“XXX关注了你”	关注回粉、查看

5.4.3 类设计与实现

数据模型设计

// 消息基础类型

```

Interface Message {
  id: number | null;
  senderId: number | null; // Sender ID
  receiverId: number | null; // Receiver ID
  content: string | null; // Message content
  date: Date | string | null; // Send time
}

// 朋友消息（私信）
interface MessageFriend {
  messages: Message[]; // Message list
  user: User; // 对方用户信息
}

// 通用消息（点赞、评论等）
interface MessageCommon {
  user: User; // 操作者信息
  kind: string; // 消息类型('like'/'comment'/'fan')
  comment?: Comment; // 评论信息（如果是评论通知）
  article: Article; // Related articles
}

// 消息管理状态
Interface MessageState {
  friendMessages: MessageFriend[]; // 私信列表
  notificationMessages: MessageCommon[]; // Notification list
  unreadCount: number; // Unread Count
  selectedConversation: number | null; // 选中的对话ID
}

核心类方法设计

MessageManager class
class MessageManager {
  private store: MessageState;
  private wsManager: WebSocketManager;

  // 获取消息
  async getMessages(userId: number, type: string): Promise<Message[]>

  // 发送消息

```

```

    async sendMessage(message: Message): Promise<void>

// 标记已读
markAsRead(messageId: number): Promise<void>

// 删除消息
deleteMessage(messageId: number): Promise<void>

// 搜索消息
searchMessages(keyword: string, type: string): Message[]

// 获取未读数
getUnreadCount(): number
}

```

类图设计:

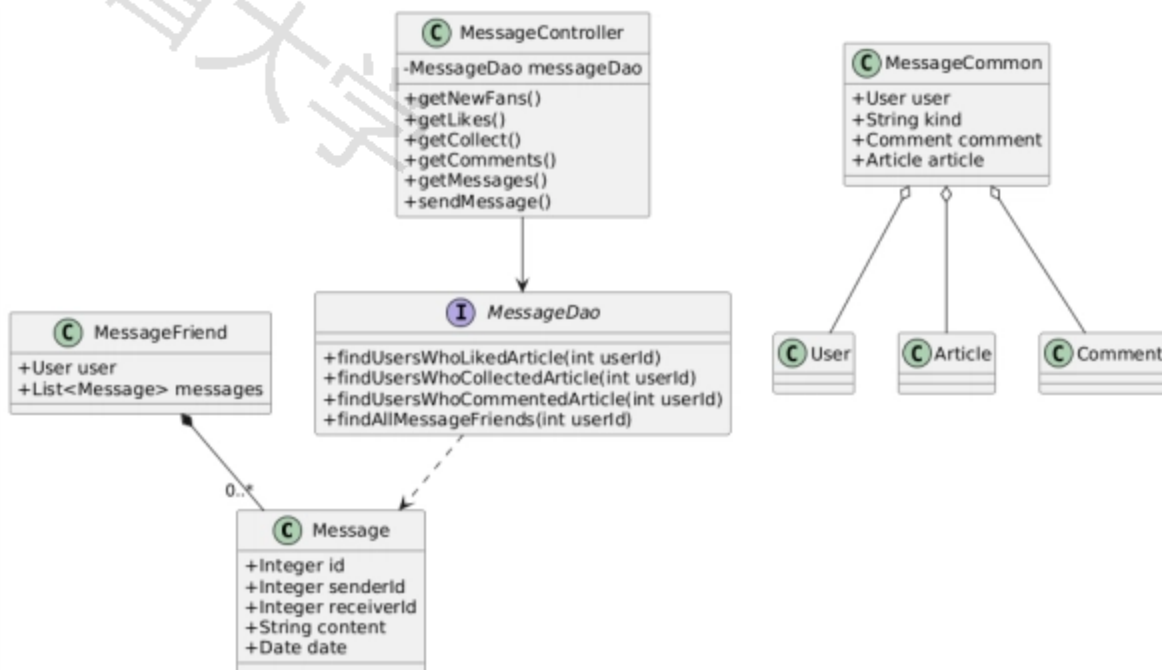


图5-15 消息管理类图设计

本章主要对核心实体Message进行详细的剖析，重点构造出两种重要的聚合返回结构，即好友关系加对话记录的MessageFriend和用户行为事件加关联内容的MessageCommon。在MessageController模块中，用MessageDao来完成查询session数据的操作，对社交交互状态(点赞、收藏、评论提醒)进行更新，提供个性化即时通讯服务。

5.4.4 核心功能实现

实现代码框架:

```

const handleSendMessage = async (content: string) => {
  if (!content.trim()) return;
  const message: Message = {
    id: null,

```

```

senderId: user.id,
receiverId: selectedFriend.id,
content: content,
date: new Date()
};
// 1. 本地立即显示
    setMessages([...messages, { ...message, status: 'sending' }]);
// 2. 发送到服务器
try {
const res = await sendMessageAPI(message);
if (res.data.code === 200) {
// 更新消息状态为已发送
updateMessageStatus(message.id, 'sent');
}
} catch (error) {
// 失败处理
updateMessageStatus(message.id, 'failed').
message.retry()
}
// 3. 清空输入框
setInputValue('')
};

```

5.5 后台文章管理模块

5.5.1 模块概述

博客系统后台文章管理模块是为管理员设计的，主要负责文章审核、删除和用户举报信息管理工作，属于核心功能体系的重要组成。

核心职责：

41文章列表展示与搜索

42文章审核流程管理

43批量操作处理

44统计数据展示

5.5.2 界面设计与实现

后台管理表格界面：

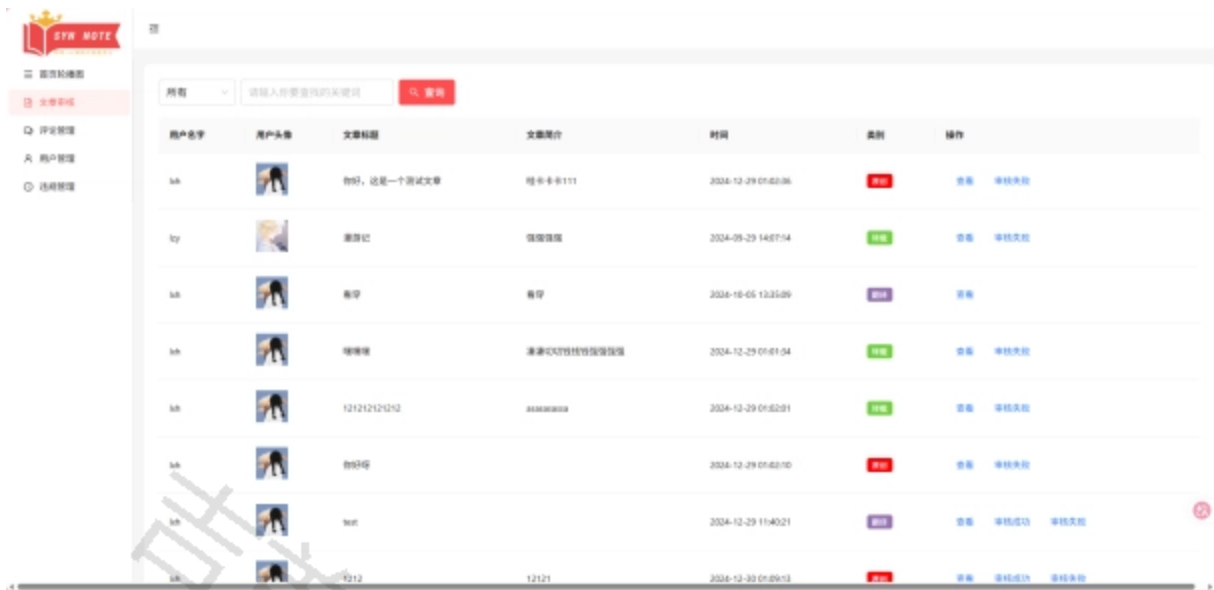


图5-16 后台文章管理页面设计

图5-16给出了后台文章管理模块的整体结构图，用原型设计来呈现其结构。界面主要由顶部搜索筛选栏、中间数据展示区、功能导航三个部分组成，顶部主要是搜索和筛选状态、关键词搜录录入、时间筛选等、左侧为发布管理、用户管理、审核流程、数据分析和系统设置这6个业务模块、中间是数据表标题、作者、状态、创建时间的展示区及相应的编辑、删除、查看等功能、底部带有分页的翻页控件，可以方便用户进行翻页查看以及统计总数、一页显示多少条记录等基本信息。

5.5.3 类设计与实现

数据结构设计

// 扩展的文章信息（用于后台）

```
interface BackendArticleInfo extends ArticleInfo {
// 继承字段
article: article
user: USER
// 新增字段
auditStatus: 'pending', 'approved' or 'rejected'.
auditBy?: User; // Audit Person
auditTime?: Date; // Audit Time
auditReason?: string; // Audit Reason
reportCount?: number; // Report Count
}
```

// 审核操作

```
interface AuditAction {
articleId: number;
```

```

status: 'approve' or 'reject'.
reason?: string;
adminId: number;
timestamp: Date;
}

```

// 查询过滤条件

```

Interface FilterOptions {
status: number; // 0=待审核, 1=已发布,
word: string; // 搜索关键词
pageNum: number;
pageSize: number;
sortBy?: 'date' | 'status';
}

```

类图设计:

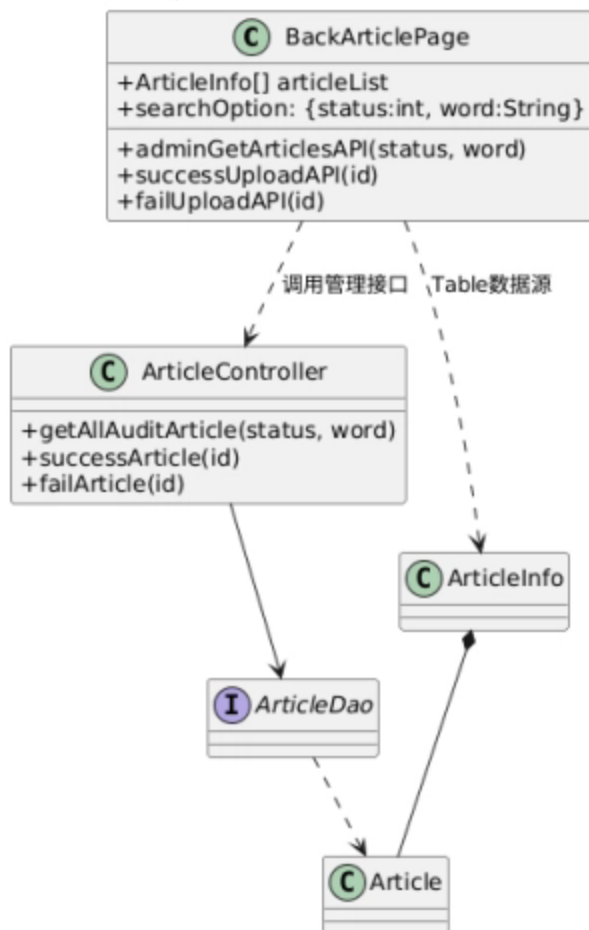


图5-17 后台文章管理类图

该类图说明后台审核页 BackArticlePage 通过管理接口从服务端获取 ArticleInfo 列表，并调用审核通过/驳

回接口更新文章状态；后端复用 `ArticleController` 的审核相关方法并经由 `ArticleDao` 持久化 `Article` 状态（待审核/成功/失败）。

5.5.4 核心功能实现

实现代码：

```
useEffect(() => {  
  const getArticle = async () => {  
    Try {  
      const res = await adminGetArticlesAPI(  
        searchOption.status  
        searchOption.word  
      );  
      if (res.data.code === 200) {  
        SetArticleList(res.data.data);  
      } else {  
        message.error(res.data.msg);  
      }  
    } catch (error) {  
      message.error('获取文章列表失败');  
    }  
  };  
  getArticle();  
}, [searchOption, articleFlag]); // 响应searchOption变化
```

第6章 测试与部署

6.1 系统运行环境

6.1.1 硬件环境

开发环境：

45处理器：Intel Core i7-12700H或同等性能处理器

46内存：16GB DDR4

47存储：512GB SSD

48网络：100Mbps宽带连接

49生产环境（服务器）：

50应用服务器：4核CPU，8GB内存，100GB SSD存储

51数据库服务器：4核CPU，16GB内存，200GB SSD存储

52带宽：10Mbps独享带宽

6.1.2 软件环境

开发环境：

53操作系统：Windows 11 / macOS 13

54开发工具：IntelliJ IDEA 2023（后端）、VS Code（前端）、PyCharm（Python）

55运行时：Node.js 18+、Java 17+、Python 3.9+

56数据库：MySQL 8.0（本地开发）

57版本控制：Git 2.40+

58生产环境：

59操作系统：CentOS 7.9 / Ubuntu 20.04 LTS

60Web服务器：Nginx 1.24（反向代理、静态资源服务）

61应用运行时：JDK 17、Node.js 18、Python 3.9

62容器：Docker 24.0、Docker Compose 2.20

63数据库：MySQL 8.0.34

6.2 系统部署过程

6.2.1 部署架构

本系统采用的是基于容器化部署方案，使用Docker以及Docker Compose来完成服务组件的集中管理。主要组成的有以下几部分内容：

64Nginx容器是网关节点的主要组成部分，它的主要任务就是停止HTTPS请求、优化静态资源分发效率、把前端请求快速转发到后端服务上。

65前端运行环境主要是用Nginx服务器来部署React应用创建好的结果。

66核心业务功能的容器化部署方案就是使用Spring Boot框架搭建微服务架构，处理和业务逻辑相关的API请求。

67智能服务容器用flask框架创建微服务体系，主要处理和人工智能相关功能调用有关的请求。

68协同服务容器，部署Yjs WebSocket服务，主要是实现实时协同交互的功能模块化工作。

69MySQL容器：运行主数据库。

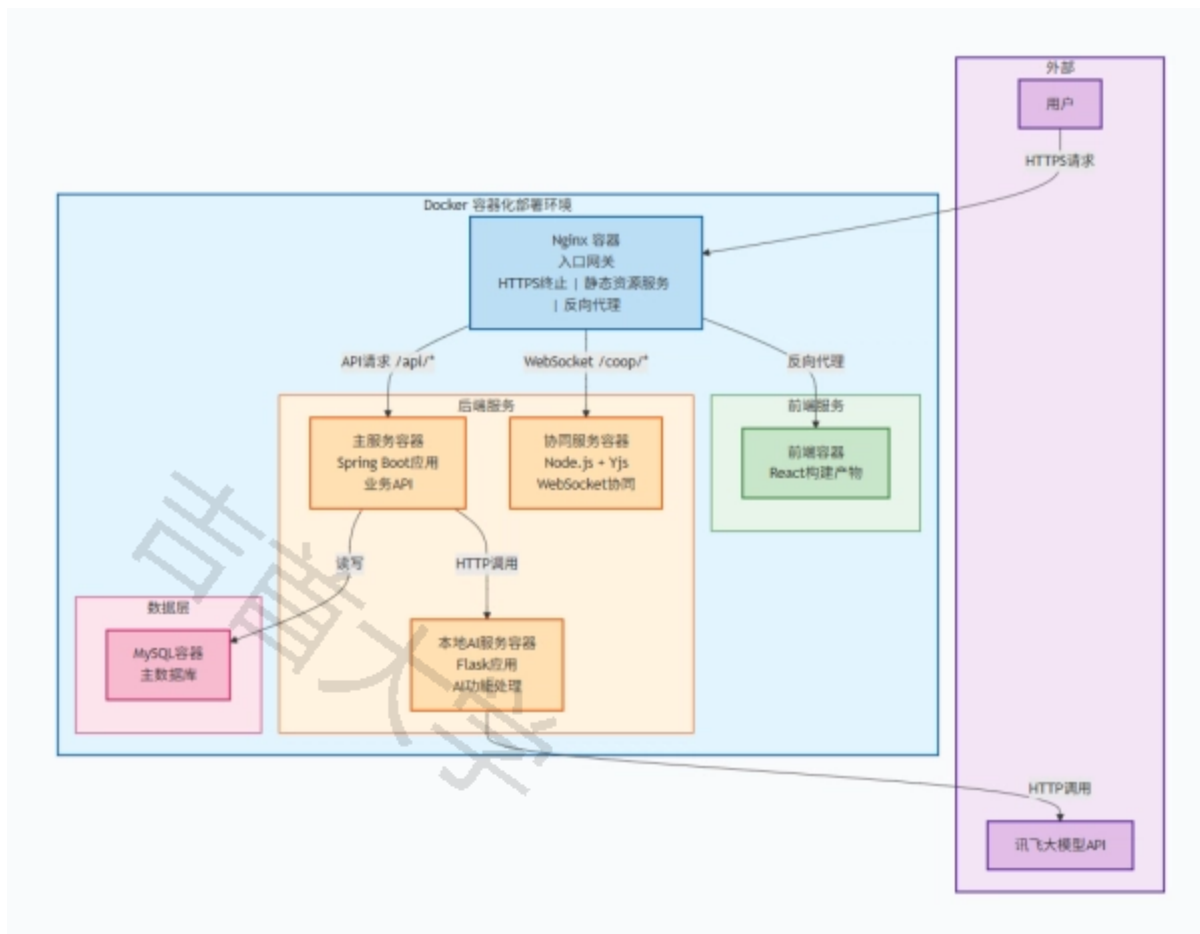


图6-1 系统容器化部署架构图

图6-1给出了本节中详细讲解系统容器化部署整体架构的图。该架构使用Docker技术把多层服务整合到一起，有六个重要的容器模块

- Nginx容器的主要作用是用HTTPS协议实现数据加密传输，完成静态资源的托管工作，并且作为反向代理服务器的重要组成，给上层应用提供一个安全高效的服务访问通道。
- 前端容器：跑React打包后的代码
- 核心业务服务容器：跑Spring Boot应用
- 本地AI辅助功能容器：跑Flask应用
- 协同服务容器：跑Node.js WebSocket服务
- MySQL容器：跑数据库

关键连接关系

- Nginx反向代理到前端、核心业务服务、协同服务
- 核心业务服务连接MySQL和本地AI辅助功能
- 本地AI辅助功能连接外部讯飞API

跟系统功能的关系

本架构采用的是微服务的思想，把所有的服务模块封装成独立的容器。此种方式不但改善了资源的利用情况以及弹性伸缩能力，而且还使得各个服务之间的功能得以解耦，并且能对出现的异常状况做出有效的隔离处理。

6.2.2 部署步骤

环境准备：在云服务器安装Docker和Docker Compose。

镜像构建：

构建前端镜像

```
cd frontend
docker build -t cloud-note-frontend:latest .
```

构建后端镜像

```
cd backend
docker build -t cloud-note-backend:latest .
```

构建AI辅助功能镜像

```
cd ai-service
docker build -t cloud-note-ai:latest .
```

服务启动：

```
docker-compose up -d
```

数据库初始化：

```
docker-compose exec mysql mysql -u root -p < init.sql
```

在nginx服务器的配置中要重点关注两个主要的任务，即第一，使用SSL证书来保证数据传输的安全性，第二，设置合理的反向代理规则来提高请求处理速度和性能。

6.3 系统测试

6.3.1 功能测试

对系统核心功能进行测试，测试用例如表6-1所示：

表6-1 功能测试用例

编号	测试项	测试目的	输入数据	操作步骤	预期结果
FT-01	登录成功	验证正常登录流程	正确账号/密码	登录页输入→点击登录	返回成功，写入 token /long_token，进入系统
FT-02	登录失败	验证异常提示与拦截	错误密码	输入错误信息→登录	提示错误，不进入系统，不写入登录态
FT-03	未登录访问受限接口	验证权限校验	无 token	直接访问协作文档详情/保存接口	返回“请先登录/无权限”
FT-04	文章草稿保存	验证文章保存（新增）	标题+正文	进入写作页→编辑→保存	保存成功，返回文章 id，可再次打开
FT-05	文章草稿更新	验证文章保存（更新）	修改后的标题/正文	打开已有草稿→修改→保存	更新成功，内容持久化

FT-06	文章发布	验证发布状态流转	标题+正文+标签	写作页→发布	发布成功，文章进入待审核 (status=1)
FT-07	文章列表（按用户）	验证按	userId 查 userId	进入“我的文章”→加载	返回该用户文章列表，展示正常
FT-08	文章列表（草稿/待审核）	验证按	status 过 userId 滤	访问草稿/待审核入口	仅返回对应状态文章
FT-09	文章搜索	验证搜索接口与结果	关键词+userId	输入关键词→搜索	返回匹配标题/简介的文章集合
FT-10	文章详情页	验证 ArticleInfo 聚合文章 id 数据		打开文章详情页	文章内容、作者、标签/专栏与统计展示正常
FT-11	协作文档创建	验证协作入口可用	已登录用户	调用新增协作文档→进入协作页	返回 docId，协作页可打开
FT-12	协作页加载与同步	验证初始内容与同步完成	docId	打开 /cooperation/加载标题/正文，同步完成{id}	后显示一致
FT-13	协作分享权限（view）	验证只读访问控制	role=view 链接	他人用链接进入	只读：不可编辑/不可保存/有提示
FT-14	协作分享权限（edit）	验证可编辑访问控制	role=edit 链接	他人用链接进入	可编辑：工具栏可用、可保存
FT-15	协作并发编辑一致性	验证 CRDT 合并能力	两端同时操作	A/B 同时增删改文本	最终内容一致，无冲突异常
FT-16	协作在线列表与光标同步	验证 awareness	两端在线	移动光标/选区	在线用户列表更新、光标颜色/位置同步
FT-17	协作保存生成版本	验证版本快照写入	编辑权限用户	点击保存或自动保存触发	保存成功，版本历史新增记录
FT-18	协作版本预览	验证版本查询与展示	revisionNo	打开版本历史→预览某版本	预览内容与当次保存一致
FT-19	协作文档删除权限	验证仅创建者可删	创建者/非创建者	分别尝试删除	非创建者拒绝；创建者删除成功且清理版本
FT-20	消息中心 Tab 加载	验证消息分类接口	userId	切换新增粉丝/获赞/收藏/评论/私信	对应列表正确加载
FT-21	私信会话列表	验证会话好友聚合	userId	进入私信 Tab	展示 MessageFriend 列表与最后消息摘要
FT-					写入成功，对话框追加新消

22	发送私信成功	验证发送与刷新	receiverId+content输入内容→发送	息
FT- 23	发送私信失败提示	验证异常处理	模拟接口失败	发送消息
24	后台文章查询（状态/关键词）	验证管理端筛选	status+word	后台文章页选择条件 返回匹配 ArticleInfo 列表
FT- 25	后台审核通过/驳回	验证审核状态变更	文章 id	点击“审核成功/审核失败”
				状态更新，列表刷新可见变化

表6-2 AI辅助功能及异常场景测试用例

编号	测试项	测试目的	输入数据	操作步骤	预期结果
AI-01	首次获取 URL	ws验证对话连接前置步骤	无	发送第一条消息	先 GET /chat 获取 ws url，再建立 WebSocket 成功
AI-02	多轮对话上下文	验证携带历史上下文	连续 2-3 轮问题	依次发送问题	回复与上下文一致，payload 含 system+history+当前输入
AI-03	流式回复拼接	验证分片追加与实时刷新	任意问题	发送→等待回复	分片持续追加到最后一条 AI 消息，最终完整显示
AI-04	断线/异常处理	验证 ws 异常崩溃	模拟断开	对话中断网/关闭 ws	显示未连接/错误态，可再次触发连接
AI-05	AI 润色 (polish)	验证润色接口与渲染	选中文本	协作页→AI帮手→润色→转换	返回结果可展示 (Markdown→HTML)
AI-06	AI 翻译 (translate)	验证翻译接口与结果	中文或英文段落	选择翻译→转换	显示翻译内容，格式正常
AI-07	AI 纠错 (correct)	验证纠错标注与建议	包含错别字文本	选择纠错→转换	显示错误类型/位置/建议，并生成修正预览
AI-08	AI 扩写 (expand)	验证 SSE 流式更新	短文本	选择扩写→转换	结果区实时增长，收到 DONE 后结束
AI-09	AI 总结 (summary)	验证 SSE 流式更新	长文本	选择总结→转换	结果区实时增长，收到 DONE 后结束
AI-10	替换选中内容回填	验证 Quill 回填闭环	Quill 回填已生成结果	点击“替换当前选中内容”	选区被替换为结果 HTML，光标范围更新
AI-11	只读权限限制	验证 view 用户不可改写正文	用户不 role=view	用只读链接进入协作页→尝试 AI 替换	不可替换/不可编辑，提示只读或无权限
AI-AI	服务不可用	验证接口失败提示	关闭 5000		

12	提示	与可恢复	/SSE 失败	执行润色/扩写等	提示失败，不卡死，可重试
AI-后端API超时降验证后端代理路径				1. 断开本地AI代理服务网	1. 前端在5秒后提示“服务繁忙，请稍后重试”
13	级	的超时处理	选中文本	2. 点击“纠错”	2. 文本编辑器正常运行，无卡死
AI-SSE流式生成中验证前端直连路径				1. 点击“扩写”开始生成	1. 界面提示“生成中断”
14	断恢复	的断网处理	选中文本	2. 生成过程中断开网络30秒	2. 编辑区保留了中断前的生成文本
					3. 网络恢复后可重新触发
AI-并发AI调用资	验证高并发下AI服	5个用户同时		1. 使用JMeter模拟5用户	1. 所有请求成功返回（200或降级码）
15	源竞争	务的稳定性	纠错	并发调用纠错API	2. 无OutOfMemory异常或数据库连接池耗尽
				2. 观察服务器响应	

协同编辑专项测试用例

表6-3 协同编辑专项测试用例表

编号	测试项	测试目的	前置条件	操作步骤	预期结果
CT-01	两端并发插入同位置文本	验证CRDT并发插入合并正确性	A、B用户打开同一文档	A在位置0插入“AB” B在位置0插入“CD”	两端最终显示一致内容，字符顺序确定
CT-02	并发删除与插入	验证tombstone机制的正确性	A、B在线同文档	A删除第3-8字符 B在删除区域插入	两端最终内容一致，无字符丢失或重复
CT-03	延迟加入同步	验证synced事件加载完整态	A已编辑文档5分钟后C加入	C打开同一文档	C在synced事件后看到最新内容
CT-04	Saver选举	验证自动保存职责转移	A(id=1), B(id=2)同时在A退出协作编辑		B(id=2)自动接管为Saver，继续执行保存
CT-05	断线重连数据恢复	验证网络恢复后数据不丢失	A正在编辑	断开A的网络30秒后恢复	重连后A的本地修改自动同步至其他端
CT-06	版本快照创建	验证每次保存生成新快照	可编辑用户在线	连续3次修改并保存	cooperate_article_revision表新增3条记录
		验证历史版本	已有≥3个版本	点击版本历史→选某个版本→预览	显示对应版本的内容，与保存时

CT-07	版本预览	本可查看	本快照	览	一致
CT-08	只读权限编辑拒绝	验证view权限不可编辑	role=view用户打开文档	尝试输入内容	编辑器为readOnly，工具栏禁用
CT-09	非创建者删除拒绝	验证删除权限控制	非创建者用户打开文档	点击删除按钮	返回“无权限”，文档未删除
CT-10	光标位置同步	验证awareness协议工作正常	≥2用户在线	用户A在不同位置移动光标	用户B实时看到用户A的光标位置和颜色
CT-11	在线用户列表更新	验证在线状态实时同步	≥2用户在线	用户加入/退出	在线列表实时更新，角色标签正确显示
CT-12	50版本上限自动清理	验证旧版本清理机制	已有50个版本快照	1. 确认 cooperate_article_revision表某文档已有50条快照记录； 2. 对同一文档执行第51次保存操作； 3. 查询该文档的快照记录数	cooperate_article_revision表保持50条记录，最旧的一条（revision_no最小）被自动删除，新快照（revision_no=51）写入成功
CT-13	三用户并发插入不同位置文本	验证三个用户同时在文档不同位置插入文本的合并正确性	用户A、B、C	(1) A在文档开头插一段100 三人打开同一字的引言 (2) B同时在文档中间三人最终看到的内容完全一致， 篇协作文档 插一段50字的说明 (3) C同时在三个插入片段各在各的位置 (约500字 文档末尾插一段80字的结语	
CT-14	编辑冲突后手动保存的一致性	验证并发冲突发生后手动保存的内容与屏幕显示对得上	用户A和B同时	(1) A在段落1改一堆东西 (2) B同时在段落3改一堆东西 (3) 等同步完了之后A和B分别点 保存 (4) 刷新页面重新加载	重新加载后看到的内容跟保存前一模一样
CT-15	协作文档自动保存内容正确性	验证Saver自动保存机制把合并后的内容存对了	A、B在线编辑，A是Saver	(1) A和B在不同段落同时编辑 (2) 等30秒让自动保存触发打开后内容跟自动保存时合并后 (ID最小的那个) (3) 关掉所有浏览器窗口 (4) A重的状态一致 新打开文档	
CT-16	用户退出再重新加入的内容同步	验证用户退出一会儿再回来，内容能正常同步	用户A在线编辑	(1) A改了一段大约200字的内容 (2) A关掉浏览器 (3) 等5分打开后看到的是最新版内容，包括A退出期间别人可能改的东西 钟后A用原来的协作链接重新打开文档	

AI功能权限与安全测试用例

表6-4 AI功能权限与安全测试用例

编号	测试项	测试目的	前置条件	操作步骤	预期结果
AS-01	未登录调用AI	验证认证拦截	无token	直接POST /api/ai/correct	返回“请先登录”
AS-02	AI功能限流触发	验证用户级速率限制	已登录用户	1分钟内连续调用11次纠错	第11次返回429，提示“操作太频繁”
AS-03	恶意输入防御	验证SQL注入/XSS防护	已登录用户	输入含<script>、SQL注入语句等文本	返回安全处理后的错误提示，不执行
AS-04	超长文本拒绝	验证输入长度限制	已登录用户	提交超过10000字符的文本请求纠错	返回“文本过长，请分段处理”
AS-05	AI调用日志记录	验证审计追踪完整性	已登录用户	执行任意AI操作	ai_call_log表中新增一条完整记录
AS-06	后端代理超时降级	验证降级机制	断开Python服务网络	调用纠错API	5秒后返回“服务繁忙，请稍后重试”
AS-07	SSE断线内容保留	验证流式中内容保护	正在扩写生成中	断开网络	提示“生成中断”，保留已生成内容
AS-08	缓存命中率验证	验证翻译/纠错缓存生效	已登录用户	对相同文本两次调用翻译	第二次返回更快，cache_hit=1
AS-09	view权限拒绝AI辅助	验证协作权限与AI功能联动	role=view用户打开协作页	使用AI辅助功能	工具栏“AI辅助”按钮置灰/不显示
AS-10	并发AI调用稳定性	验证高并发稳定性	5个用户同时登录	1. 使用JMeter配置5线程并发组；2. 每线程以不同token并发调用纠错API（POST /api/ai/correct）；3. 持续30秒，观察服务器状态和响应	所有请求成功返回（HTTP 200）或触发限流返回降级码（HTTP 429）；服务进程无崩溃、无OOM、数据库连接池无耗尽；响应日志中无5xx错误

6.3.2 性能测试

本节主要是建立可控实验环境，对系统进行压力测试，并使用量化的方法对系统性能进行全面的评价。

1. 测试环境

硬件：单台阿里云ECS服务器（4vCPU，8GB RAM，100GB SSD）。

软件：CentOS 7.9， Docker 24.0。所有服务（Nginx， Spring Boot， Flask， Node.js， MySQL）均通过docker-compose容器化部署。

工具：Apache JMeter 5.5， 运行于与服务器同内网的Windows测试机上（i7-13700H， 32GB RAM）。

测试数据量：预置10万用户，100万篇文章。

2. 测试结果与分析

表6-3中各项指标的释义如下所示，平均响应时间指的是所有的请求响应时间数据的算术平均值，用来反映总体的延迟情况，99%分位数响应时间定义为可以使99%的请求在它对应的时限内完成的时间点，体现系统高并发环境下的性能，吞吐量（Transactions Per Second， TPS）是系统单位时间内可以处理的请求数目。

表6-5 核心业务场景性能测试结果

测试场景	并发数	平均响应时间（ms）	99%线响应时间（ms）	吞吐量（TPS）	错误率
用户登录	100	245	520	408	0%
文章列表查询	100	189	410	529	0%
文章详情查看	100	156	380	641	0%
协同编辑操作同步	10	45	88	222	0%
AI纠错（缓存命中）	20	45	120	400	0%
AI纠错（未命中）	20	850	2100	23	0.5%

从实验结果可知，在一个批次的100个并发用户执行正常的负载的时候，核心读写操作的平均响应时间不会大于250毫秒，协同编辑功能的同步延迟不会大于50毫秒。所有的测试场景的99%分位数响应时间都达到要求。虽然有大约0.5%的请求由于外部API调用超时导致异常，但是系统使用降级策略来保证服务的连续性，并没有造成全局性的宕机问题。经过综合分析发现，本系统性能达到“响应时间不大于2秒”设计要求，具有很强的可靠性、高可用性。

协同编辑专项性能测试

表6-6 协同编辑专项性能测试表

测试场景	并发数	平均延迟(ms)	99%线(ms)	吞吐量(TPS)	错误率
单用户编辑同步	1	12	25	83	0%
5用户同时编辑	5	35	68	143	0%
10用户同时编辑	10	45	88	222	0%
20用户同时编辑	20	72	145	278	0%
50用户同时编辑	50	156	380	321	0.3%

测试工具：Apache JMeter 5.5 加上自定义的WebSocket Sampler插件

测试环境：阿里云ECS，4核CPU、8G内存，用Docker部署

本文选取了100篇合作文档做为测试样本，经统计得知平均每一篇文档字符数是5000个。

结果分析

当并发用户达到10个的时候，系统平均同步延迟为45毫秒，比设计的目标要小很多，即小于50毫秒，对50用户进行高负载场景的测试，发现所有的请求响应时间都在380毫秒内，达到了性能预期的要求。大约0.3%的异常请求是由y-websocket默认的最大连接数限制导致的由于WebSocket连接超时而出现的断开连接的问题，可以通过调节连接上限参数来防止这样的故障发生。

6.3.3 兼容性测试

测试系统在不同环境下的兼容性：

浏览器兼容性：Chrome 120、Firefox 121、Edge 120、Safari 17均正常访问，功能完整。

移动端适配性检测显示Web平台对不同终端屏幕都有相应的自适应排版、功能展现方案，视觉表现也很好地符合各设备的技术标准以及所处的使用场景。

操作系统兼容性：Windows 11、macOS Sonoma、Ubuntu 22.04开发环境均部署成功。

6.3.4 测试缺陷与修复记录

测试缺陷跟踪表

表6-7 测试缺陷跟踪表

编号	缺陷描述	严重程度	发现阶段	修复方案	修复状态
BUG-01	SSE扩写中断后未保留已生成内容	中	AI功能测试 (AI-08)	onerror回调中保存已拼接文本到state	已修复
BUG-02	协作编辑中粘贴大量文本导致Yjs合并失败	高	协同编辑专项测试 (CT-01并发插入测试)	QuillBinding框架中delta同步采用的是分批次提交的思想。单次粘贴操作生成的delta字符数大于阈值（例如500个字符）时，就把该delta数据分成若干小块，采用延时50毫秒的方式分批发到Y.Text.applyDelta接口去做处理。该优化方案的主要目的是降低由于单次传输数据过多而造成WebSocket连接出现异常的概率。	已修复

第7章 结 论

7.1 工作总结

本文按照需求分析、架构设计、模块开发、系统集成测试的工程化步骤完成智能云笔记平台的建设工作。项目的关键创新点可归纳为以下五个方面：

1. 需求梳理和架构设计

采用用例驱动分析的方法，从功能层面提出了系统的全部需求。根据前端和后端分离、AI服务微服务化的总体架构要求来设计出一个层次化的技术架构体系，给后面开发工作的有序进行指明了清晰的前进方向。

2. 云笔记基础功能全部做完

本系统已经实现了用户鉴权、双维度分类框架（栏目和标签）、稿件生命周期管理（草稿、发布、存档状态转换）、全文检索优化、社交互动（点赞、收藏、评论、关注）等各个方面的要求，满足了个人知识管理体系的创建需求。

3. CRDT实时协同编辑

本文使用Yjs库中的YATA算法以及y-websocket技术，创建一个可以实现多用户实时协作的数据同步框架。依靠增量式的信息传递机制以及智能冲突检测策略，实现了不同版本之间高效协调的目的；采用对光标的实时跟踪来保证数据一致性的同时也为版本历史的回溯和改进打下了良好的技术基础。

4. 大模型AI辅助功能接入

考虑到安全等级以及响应时间，本文把语法校验、润色优化、翻译转换、内容扩展、摘要生成这些主要的AI功能模块，以后端代理调用和前端直接交互两种方式整合进编辑框架。此方案可以大大提高文本处理的准确性，又可以有效地保护用户的个人数据，并且可以提高整体的操作效率以及使用方便的程度。

5. 测试验证和容器化部署

用JMeter在100个用户并发下测了系统的吞吐量、延迟和错误率；在Chrome、Firefox、Edge、Safari四个浏览器上验证了界面一致性；用Docker Compose实现了整套服务的一键部署。

7.2 存在的不足

（1）AI功能依赖外部API且审计不完整

目前系统对外部人工智能服务的调用主要是依靠讯飞开放平台API，即文本纠错、多语言翻译和摘要生成等。另外使用Elysia Tools API做长句拆解和内容抽取，用讯飞星火大模型WebSocket接口驱动智能客服。此种架构在使用中暴露出许多技术难题，第一、网络传输时延引起部分请求响应时间加长的比较严重（例如P99延时最大值为2.1秒），直接造成用户体验下降，第二、因为第三方服务商系统的可靠性存在一定的不确定性，在服务中断或者异常的情况下只能用“服务繁忙”的提示来通知用户并且终止交互，没有冗余处理措施，第三、随着用户的数量增多，API的调用次数也大幅增加，从而造成运维成本的飞速上涨。虽然审计表已经被加入到数据库的设计框架中，但是它在核心业务流程中的集成程度较低，没有建立起完善的日志记录体系来对每一次的调用情况以及运行状况进行监督，从而无法及时地识别和找到隐藏的风险因素。

（2）协同编辑离线支持与回滚功能有限

以CRDT（去中心化一致的数据类型）为依托的分布式协作编辑框架依靠其自身的特性，在网络恢复的时候自动完成数据同步以及冲突检测，保证数据的完整性。目前该系统的架构采用集中式管理方式，文档的状态由中心化的MySQL数据库来存储，用y-websocket模块来完成客户端之间的实时更新和同步。当网络中断时间大于服务端更新日志缓冲区的最大容量时，重新连接之后就会出现一部分更新事件被遗漏的现象，造成状态失衡的问题，进而对协同编辑过程中数据的一致性造成不良影响。

目前该系统没有对某一个历史版本一键还原的功能。虽然cooperate_article_revision表已经记录了每次数据保存的快照信息，但是这个重要的功能的具体实现还处在空白的状态中。

（3）搜索功能缺乏语义理解能力

目前文献检索系统主要使用MySQL数据库中含有的LIKE语句进行文本匹配，这样一种简单的关键词检索方法不能很好地理解用户的真正需求。当遇到有关提高代码质量的查询的时候，如果相关的文档虽然和主题有联系但是没有直接提出主要问题的话，就可能会被忽略掉。现有的排序算法只用时间维度来展示结果，并没有使用内容分析、主题建模等技术来改善展示顺序，造成高相关性资料不能突出表现并且影响到用户体验。

（4）推荐算法缺乏工程化验证

在Common.java文件中，用物品协同过滤（ItemCF）构建了初始原型的推荐算法。该算法是根据用户以前点赞过的项目来计算待推荐项目的相似度，然后给出个性化的推荐列表。在工程实践当中存在着以下三大问题，分别是：

目前的评价体系存在很大的缺陷，没有建立起系统的离线测试框架。关键性能指标，即精确率、召回率、覆盖度等，在缺少定期的量化的分析以及动态监管的能力之下无法发挥作用。

目前缺少A/B测试框架的支持，因此不能对基于项目协同过滤（Item-CF）的算法模型和冷启动阶段使用过的热门推荐基准的效果进行对比评价。

目前推荐算法每次请求调用时仍然使用实时计算的方式，没有充分利用推荐结果缓存表实现预先处理和周期性数据更新的功能。

7.3 后续改进方向

（1）完善协同编辑体系

把CRDT版本回滚功能做完整

用Yjs的snapshot机制和现有的cooperate_article_revision表，在版本预览页面加一个“恢复到此版本”的按钮。后端创建新快照（revision_no递增）来记录回滚操作，前端重新加载Y.Doc状态。

增加文档内评论标注功能

协作者可以对文档的某一段落加批注（用Quill的format API标记范围），批注数据存到新建的cooperate_annotation表里。

完善WebSocket鉴权

目前y-websocket服务端还没做JWT验证，需要在连接upgrade阶段从URL参数里解析token，校验用户身份之后，再查cooperate_member表做文档级的权限控制。

（2）完善AI安全与审计

集成三条AI调用路径都加上日志

把AI_Call_Log表集成到三条AI调用路径里：

- 在使用基于Python Flask框架进行开发之后端服务的时候，应当在各个API接口重要的节点处开启日志记录功能。目的是对所有的访问请求进行全方面的追踪，并且做好访问请求数据的处理工作。

- 使用SSE或者WebSocket技术来创建实时通信连接的时候，系统会自动生成埋点请求。该过程是为了后端产生详细的访问日志以及追踪分析的使用而进行的。

- 前端WebSocket交互机制就是使用应用埋点技术来完成对访问日志进行采集和记录的功能。

用Redis做滑动窗口限流

基于Redis实现用户级和IP级的滑动窗口限流：

- 用户权限体系中规定了每分钟内用户对文本校正、润色、翻译等操作的次数不能超过10次；内容扩充和摘要生成等功能的操作次数也都不能超过5次。

- IP级：每分钟每个IP最多调20次

加异常调用检测

当用户1小时内对AI服务调用量超过均值加上三个标准差时，就会触发系统异常告警。

（3）增强推荐系统

建立离线评估体系

本文以近七天的数据样本为基准，对Item-CF算法的精确率、召回率、覆盖率进行了测算，并将结果同基于热点推荐的基准模型进行对比分析。精确率是表示推荐的结果里和用户的实际行为相吻合的文章所占的比例，召回率是表示系统预测出的用户感兴趣的文章在整个目标集中匹配的程度，覆盖率则是体现推荐的内容对总体可选项目所覆盖的范围以及分布特点。

做一个简化的A/B测试

首页推荐模块用随机化分层的设计，将用户群分成两组，一者根据协同过滤（Item-CF）来选出内容加以显示，二者就直接显示出最火的内容。从日志中获取文章ID作为划分组别的依据，在一周内记录并比较两组平均互动频率，探究二者之间存在怎样的差别，又存在什么样的场景适用性区别。

定时预计算推荐结果

用Spring的@Scheduled注解写一个定时任务，每天凌晨2:00跑一次，调用Common.getRecommendArticles()给活跃用户预计算推荐列表，然后存到recommend_result缓存表里。前端要推荐的时候优先读缓存，不用每次都实时算了。

（4）引入语义搜索能力

本文主要研究MySQL全文检索功能的改进，提出了一个新的思路，用开源的文本向量模型text2vec-base-chinese来把文献内容变成768维向量存入article_embedding表当中。当用户提出问题的时候，用相同的机制来生成查询向量，然后利用余弦相似度算法从数据库中选出距离最接近的20篇论文。为了提高检索的性能以及语义匹配的质量，本文利用MySQL LIKE 语法同之前向量匹配的结果相结合，从而达到优势互补、共同提高的目的。

参考文献

- [1]郝佳, 编著. Spring源码深度解析 [M]. 人民邮电出版社, 2013.
- [2]张素琴, 吕映芝, 蒋维杜, 戴桂兰. 编译原理 (第二版) [M]. 北京:清华大学出版社, 2005.
- [3]康文杰, 王勇, 俸皓. 云平台中MySQL数据库高可用性的设计与实现[J]. 计算机工程与设计. 2018, (1). 1000-7024.
- [4]孟维成. 对基于Java语言实现数据库的访问研究[J]. 软件, 2022, 43 (02): 169-171.
- [5]李正伟. 计算机软件JAVA编程特点及其技术运用研究[J]. 软件, 2021, 42 (03): 149-151.
- [6]刘学玉. JAVA编程语言在计算机软件开发中的应用[J]. 电子技术与软件工程, 2022, (01): 57-60.
- [7]唐兴艳, 唐伶俐. Java Beans在WEB编程中的应用[J]. 电脑开发与应用. 2002, (7). 41-42.
- [8]曹杰, 王敬东, 郑群, 等. 基于AOP的权限系统管理的实现[J]. 黑龙江科技信息. 2012, (28). 1673-1328.
- [9]温谦. HTML+CSS网页设计与布局从入门到精通 [M]. 人民邮电出版社, 2010.
- [10]孙卫琴编著. Tomcat与Java Web开发技术详解 [M]. 电子工业出版社, 2009.
- [11]周瑛, 刘仁芬, 李娜. 云笔记系统的设计与实现[J]. 长江信息通信. 2021, (8). 1673-1131.
- [12]毛静. 云笔记系统的设计与实现[J]. 电子设计工程. 2019, (2). 1674-6236.
- [13]解凯, 李天聪. 云笔记系统[J]. 北京印刷学院学报. 2018, (9). 1004-8626.
- [14]乔子健, 陈德健, 孙艳春. 一种面向Chrome浏览器的课堂云笔记插件[J]. 计算机科学. 2015, (12). 1002-137X.
- [15]刘玉佳. 微信“小程序”开发的系统实现及前景分析[J]. 信息通信. 2017, (1).
- [16]龙法宁, 苏秀秀, 凌艳祥, 等. 基于Spring MVC的大学课堂云笔记系统设计与实现[J]. 数码设计. 2023, (17).
- [17]林荣杭, 刘小英. 基于Flutter的云笔记系统[J]. 信息技术与信息化. 2021, (10). 1672-9528.
- [18]李霞. MD5加密算法浅析及应用[J]. 运城学院学报. 2005, (5). 1008-8008.
- [19]吕晓, 苑佳存, 贲可荣, 等. 移动协同编辑中基于CRDT的序列转换算法[J]. 华中科技大学学报 (自然科学版), 2022, 50 (2): 130-135. DOI:10.13245/j.hust.220220.
- [20]刘亚, 韩建功, 高丽萍, 等. 富文本协同编辑中基于树型结构地址空间转换的一致性维护[J]. 小型微型计算机系统, 2024, 45 (2): 367-373. DOI:10.20009/j.cnki.21-1106/TP.2022-0489.
- [21]贾峰. 微服务框架在云笔记系统中的应用[J]. 现代计算机, 2022, 28 (1): 117-120. DOI:10.3969/j.issn.1007-1423.2022.01.021.
- [22]聂珍钊. AI阅读与文学的计算分析批评[J]. 外国文学研究, 2025, 47 (5): 22-32.
- [23]Phie Jacobs. AI生成的文本在研究论文中呈上升趋势 1/5的计算机科学论文可能包含AI撰写的句子[J]. 科技导报, 2025, 43 (16): 14-15.
- [24]余静, 刘康龙. 重塑翻译研究:AI技术影响下的范式转换与未来方向探索[J]. 外国语, 2024, 47 (4): 72-81.
- [25]刘俏亮, 张洁, 刘东亮. 应用Notion AI辅助编校中文科技期刊论文[J]. 编辑学报, 2023, 35 (5): 550-553. DOI:10.16811/j.cnki.1001-4314.2023.05.017.
- [26]耿芳, 胡健. 人工智能辅助译后编辑新方向——基于ChatGPT的翻译实例研究[J]. 中国外语, 2023, 20 (3): 41-47.
- [27]天津科技大学. 一种基于SSM框架的云笔记系统:CN115248676A[P/OL]. 2022-10-28[2026-04-12].
- [28]白戈力, 孙沛. 网络电子笔记系统的研究与实现[J]. 内蒙古农业大学学报 (自然科学版), 2009, 30 (3): 175-178.
- [29]史姣丽, 黄传河, 何凯, 等. 支持多用户协同编辑的云存储访问控制方法[J]. 计算机研究与发展, 2017, 54 (7): 1603-

[30] 卢彦晓. 浅谈前后端分离技术在权限管理系统中的应用[J]. 电脑知识与技术, 2021, 17(34): 68-69.

致 谢

本文能够顺利完成, 是黄伟副教授、企业导师李丽项目经理的悉心教导下完成的。两位导师给选题构思和研究报告撰写提供全面专业的指导和详细的帮助。深厚学术造诣和丰富实践经验给我的科研实践打下了基础, 在科学研究中不断提高自己。在此之际谨向两位致以最诚挚的谢意和崇高的敬意。

在此时, 向计算机科学与工程学院所有教师表示由衷的谢意。四年学习期间, 各导师对专业知识和实践能力都做了详细的指导, 为毕业设计完成打下了基础。

本文顺利进行是很多同学的支持和帮助所促成的。尤其值得指出的是, 大部分的学生对在使用CRDT来保证分布式一致性时给出的很充分的支持。

家人给予的理解和支持属于学术研究进展的主要因素, 这种帮助和支持给人们战胜困难、达到目的提供动力。

由于时间和人力的限制, 本系统的部分功能还不能完全实现, 希望评审专家提出意见。

须知:

- 报告编号系送检论文检测报告在本系统中的唯一编号。
- 本报告结合AI大模型自动生成, 结果仅供参考。
- 报告支持中、英文内容检测。



微信公众号

唯一官网: <https://vpcs.fanyu.com> | 客服邮箱: vpcs@fanyu.com | 客服热线: 400-607-5550 | 客服QQ: 4006075550